

The Proof Analysis Problem^{*}

Noel Arteche[†] Albert Atserias[‡] Susanna F. de Rezende[§] Erfan Khaniki[¶]

Abstract

Atserias and Müller (JACM, 2020) proved that for every unsatisfiable CNF formula φ , the formula $\text{REF}(\varphi)$ —stating that “ φ has small Resolution refutations”—does not have subexponential-size Resolution refutations. Conversely, when φ is satisfiable, Pudlák (TCS, 2003) showed how to construct a polynomial-size Resolution refutation of $\text{REF}(\varphi)$ given a satisfying assignment of φ . A question that had remained open is: *do all short Resolution refutations of $\text{REF}(\varphi)$ explicitly leak a satisfying assignment of φ ?*

We answer this question affirmatively by providing a polynomial-time algorithm that extracts a satisfying assignment for φ given any short Resolution refutation of $\text{REF}(\varphi)$. The algorithm follows from a new feasibly constructive proof of the Atserias–Müller lower bound, formalizable in Cook’s theory PV_1 of bounded arithmetic. This implies that Extended Frege can efficiently prove (a suitable formalization of the statement) that automating Resolution is NP-hard.

Motivated by this algorithm, we introduce a new meta-computational problem concerning Resolution lower bounds: the *Proof Analysis Problem* (PAP). For a fixed proof system Q , the Proof Analysis Problem for Q asks, given a CNF formula φ and a Q -proof of a Resolution lower bound for φ , encoded as $\neg\text{REF}(\varphi)$, whether φ is satisfiable. In contrast to the Proof Analysis Problem for Resolution, which is in P, we prove that PAP for Extended Frege (EF) is NP-complete. In particular, EF can prove Resolution lower bounds on satisfiable formulas without necessarily revealing a satisfying assignment.

Our results yield new insights into proof search and the meta-mathematics of Resolution lower bounds: (i) for every proof system that simulates EF as well as for Resolution, the system is (weakly) automatable if and only if it can be (weakly) automated exclusively on formulas stating Resolution lower bounds; (ii) we provide explicit REF formulas that are exponentially hard for bounded-depth Frege systems; and (iii) for every strong enough theory of arithmetic T we construct explicit unsatisfiable CNF formulas that are exponentially hard for Resolution but for which T cannot prove even a quadratic Resolution lower bound. This latter result applies to arbitrarily strong theories like PA or ZFC, and does not require any complexity-theoretic assumptions.

^{*}An extended abstract appeared in the 66th Annual Symposium on Foundations of Computer Science (FOCS 2025) [AAdRK25].

[†]Lund University and University of Copenhagen, noel.artech@cs.lth.se

[‡]Universitat Politècnica de Catalunya and Centre de Recerca Matemàtica, atserias@cs.upc.edu

[§]Lund University, susanna.rezende@cs.lth.se

[¶]University of Oxford, erfan.khaniki@cs.ox.ac.uk

Contents

1	Introduction	1
1.1	Contributions	2
1.2	Technical overview	7
1.3	Related work	11
1.4	Open problems	11
1.5	Structure of the paper	12
2	Preliminaries	12
2.1	Levin reductions	13
2.2	Proof complexity	13
2.3	Bounded arithmetic	16
2.4	The REF formulas	18
3	The Proof Analysis Problem: definitions and basic facts	22
4	The extraction algorithm	25
4.1	The block-width reduction algorithm	26
4.2	The block-width analysis algorithm	29
4.3	Putting it together	32
4.4	Assignment extraction as information efficiency	33
5	The Proof Analysis Problem for Extended Frege is NP-complete	35
5.1	Hardness proof	35
5.2	Does analyzability require lower bounds?	37
6	The Atserias–Müller lower bound in PV_1	39
6.1	The restriction argument	40
6.2	The block-width lower bound	43
6.3	Formalization of the final lower bound statement	46
7	Pudlák’s upper bound in Resolution	47
7.1	Description of the construction	47
7.2	The construction as a low-depth circuit	49
7.3	Correctness of the construction in Resolution	52
8	Extended Frege proves that automating Resolution is NP-hard	55
9	Universality of REF formulas	57
9.1	Propositional fragments of the Atserias–Müller lower bound	58
9.2	Automatability in terms of REF formulas	59
9.3	Unprovability of Resolution lower bounds	61
	Acknowledgements	64
	References	64

1 Introduction

The most natural computational problem arising in proof complexity is that of *proof search*: what is the complexity of finding proofs? In the late 90s, the notion of *automatability*, defined by Bonet, Pitassi, and Raz [BPR00], emerged as a central concept in the theory of propositional proof complexity. A proof system Q is *automatable* if there is a deterministic algorithm that finds a Q -proof of a formula φ in time polynomial in the shortest one available. Except for Tree-like Resolution, which is automatable in quasi-polynomial time [BP96], no other non-trivial proof system is known to be automatable in polynomial or quasi-polynomial time.

Krajíček and Pudlák [KP98] and Bonet, Pitassi, and Raz [BPR00] proved that under standard worst-case number-theoretic assumptions in cryptography, strong proof systems like TC^0 -Frege and Extended Frege are not automatable. These results can be transferred to AC^0 -Frege under slightly stronger hardness assumptions [BDG+04], but it seems hard to push them further. Their proof techniques require some amount of basic number theory to be formalized in the system, something that is likely unworkable for Resolution. Since then, efforts focused on showing the hardness of automating Resolution and related weak systems [Pud03; AR08; GL10; AM11; HP11; Ats13; BPT14; MPW19], culminating in the final answer by Atserias and Müller [AM20], who proved that Resolution is not automatable unless $P = NP$. This is the optimal hardness assumption since $P = NP$ implies the automatability of any proof system.

The technique used in [AM20] relies on the insight that Resolution *cannot reason about its own lower bounds*. To every CNF formula φ , they associate a new formula $REF_s(\varphi)$ that encodes the statement “there is a size- s Resolution refutation of φ ”. As a tautology, $\neg REF_s(\varphi)$ is a natural propositional encoding of a Resolution lower bound. (We postpone to the preliminaries the details of the encoding of the REF formula we use, where we also discuss previously studied variations.)

Pudlák [Pud03] had shown already in 2003 that whenever φ is satisfiable, the formula $REF_s(\varphi)$ is easily refutable by Resolution. On the other hand, Atserias and Müller [AM20] proved that whenever φ is unsatisfiable, Resolution will require exponential size to refute $REF_s(\varphi)$, for s being some fixed polynomial in the number of variables of φ , which we omit in the subscript for the rest of this introduction for the sake of clarity.

As a consequence, an automating algorithm running on formulas of the form $REF(\varphi)$ can be used to decide SAT in polynomial time: if $\varphi \in SAT$, then the algorithm must find a short refutation of $REF(\varphi)$ that Pudlák guarantees must exist; on the other hand, if $\varphi \notin SAT$, then there are no short refutations of $REF(\varphi)$, so we can stop the automating algorithm after a polynomial number of steps and be certain that φ is unsatisfiable.

The proof strategy behind the Resolution lower bound on REF formulas was soon adapted to a variety of weak proof systems (those where unconditional size lower bounds are known) [Bel20; BY24; Gar24; GKMP20; dRGN+21; IR22; Pap24], although the REF-like formulas used in these spin-off results are no longer natural lower-bound statements for these systems. In general, as pointed out by Pudlák, the question of whether a proof system can prove any of its own lower bounds “is widely open, except for Resolution, and we consider it more important than automatability” [Pud20, p. 3]. It is currently open, for example, whether systems like constant-depth Frege have polynomial-size proofs of any of their own lower bounds.

The feat of the Resolution lower bound on REF formulas, combined with the upper bound for satisfiable formulas, implies that Resolution can *only* reason about “trivial” Resolution lower bounds (i.e., lower bounds on satisfiable formulas, which do not have refutations of any size). This highlights the upper bound construction as something even more remarkable, given that Resolution cannot efficiently prove its own soundness [AB04]. Intriguingly, the known upper bound for $REF(\varphi)$ for satisfiable φ crucially relies on Resolution guessing a satisfying assignment and using it as the backbone of the refutation. It is then natural to ask whether this is necessary:

(Q₁) *Is it the case that whenever there is a short Resolution refutation of $\text{REF}(\varphi)$, the proof must “leak” a satisfying assignment?*

By “leaking” we mean that a satisfying assignment is always readable in polynomial time from the given refutation. It is important to note that given a refutation π of $\text{REF}(\varphi)$, one cannot simply restrict π in a way that corresponds to $\text{REF}(\varphi|_{x_1=0})$ and $\text{REF}(\varphi|_{x_1=1})$ to extract a satisfying assignment. This is because the variables of φ are *not* variables of $\text{REF}(\varphi)$. In principle, such a self-reducibility trick seems to require access to an automating algorithm, so that one could successively look for refutations of $\text{REF}(\varphi|_{x_1=0})$ or $\text{REF}(\varphi|_{x_1=1})$, then $\text{REF}(\varphi|_{x_1=b_1, x_2=0})$ or $\text{REF}(\varphi|_{x_1=b_1, x_2=1})$, and so on for all variables. Without access to an automating algorithm, it is not at all clear whether satisfying assignments can be extracted efficiently.

Yet another way of phrasing the lower bound on REF formulas is to see it as the correctness proof of a *lower bound analysis algorithm*. Namely, the result proves that there is an algorithm that given a Resolution refutation π of $\text{REF}(\varphi)$ decides whether φ is satisfiable. The algorithm consists simply of checking whether π is correct and short enough. The correctness of this procedure requires the proof of the lower bound, and this framing naturally leads to the following second natural question regarding REF formulas:

(Q₂) *Is there an algorithm that given an Extended Frege proof π of a Resolution lower bound $\neg\text{REF}(\varphi)$ decides in polynomial time whether φ is satisfiable?*

If the answer were affirmative, this would settle the long-standing open problem of the NP-hardness of automating Extended Frege: given a CNF formula φ , construct the formula $\text{REF}(\varphi)$ and run the automating algorithm to find a short Extended Frege refutation. If an algorithm as the one asked for in (Q₂) existed, then we could apply it on this refutation to *analyze* whether φ is satisfiable. This distills the main idea in [AM20], and the framing of the question in terms of algorithm design suggests that such an algorithm might well be possible *without the need for unconditional Extended Frege lower bounds*.

Overall, the two questions (Q₁) and (Q₂) above hint at the central role of meta-mathematical lower bound statements in the theory of efficient proof search. We believe this calls for a deeper structural understanding that could lead to much-needed conceptual insights in automatability.

1.1 Contributions

Motivated by questions (Q₁) and (Q₂) above, we introduce a new meta-computational problem relating proofs and computation: the *Proof Analysis Problem*.

For every propositional proof system Q , the *Proof Analysis Problem for Q* (PAP_Q) consists in analyzing Resolution lower bounds proven by Q . More formally, given a CNF formula φ and a Q -proof of the Resolution lower bound encoded by the formula $\neg\text{REF}(\varphi)$, the task is to decide whether φ is satisfiable.

The Proof Analysis Problem for Q (PAP_Q)	
Input	A CNF formula φ , a size parameter s in unary and a Q -proof π of the formula $\neg\text{REF}_s(\varphi)$.
Output	Is φ satisfiable?

The problem can be seen as the computational task of distinguishing “true” Resolution lower bounds (those where φ is actually unsatisfiable) from “trivial” ones (those where the lower bound trivially holds because φ is satisfiable and there is therefore no Resolution refutation, of any size). For those proof systems for which $\text{PAP}_Q \in \text{P}$, we say that Q is *analyzable*. We remark that the REF formula in the definition of PAP_Q is always referring to Resolution refutations, while the proof system Q where $\text{REF}(\varphi)$ is being derived can be arbitrarily strong.

For the case of Resolution itself, the problem PAP_{Res} is easy to compute thanks to the lower bound on REF formulas [AM20]: if π is a correct refutation of $\text{REF}(\varphi)$ and it is small, then φ must be satisfiable. Until now, however, to the best of our knowledge this was the extent of what could be said about PAP-like problems. In particular, we are not aware of any other upper or lower bounds on this problem for proofs systems other than Resolution.

In the language of PAP, questions (Q_1) and (Q_2) can be neatly rephrased as follows:

(Q_1) Does PAP_{Res} admit a search-to-decision reduction?

(Q_2) Is Extended Frege analyzable? Namely, is PAP_{EF} in P ?

In this work we kick-start the systematic study of these Proof Analysis Problems and settle questions (Q_1) and (Q_2) above. This in turn yields a series of interesting consequences for the meta-mathematics of proof complexity lower bounds as well as proof search. We outline our results next.

1.1.1 An algorithm for assignment extraction

On the topic of question (Q_1) , our main result is that the search version of PAP_{Res} can be solved deterministically in polynomial time.

Theorem 1.1 (Assignment extraction algorithm, informal). *The search version of the Proof Analysis Problem for Resolution can be solved in deterministic polynomial time whenever the size parameter s is at least n^3 . That is, there is an algorithm that, given a CNF formula φ over n variables and $\text{poly}(n)$ clauses and a Resolution refutation π of $\text{REF}_s(\varphi)$ with $s \geq n^3$, extracts a satisfying assignment for φ in time polynomial in n , s and the size $|\pi|$ of π , whenever φ is satisfiable.*

The question can be stated more formally in terms of *Levin reductions*. A Levin reduction between search problems R_1 and R_2 is a Karp-style many-one reduction that maps instances of R_1 to instances of R_2 , with the additional property that it also maps solutions of R_1 to solutions of R_2 , and back. The reduction $\varphi \mapsto \text{REF}(\varphi)$ showing that SAT reduces to the Proof Size Problem for Resolution with an exponential gap is clearly Levin in one direction: given a satisfying assignment of φ , Pudlák’s construction can craft a refutation of $\text{REF}(\varphi)$. However, it had remained open whether this Levin reduction could be made two-way: given a refutation π of $\text{REF}(\varphi)$, can one always extract a satisfying assignment to φ in polynomial time?

For most if not all natural NP -complete languages, the corresponding search problems tend to be complete under Levin reductions. However, the same decision problem could admit different search problems associated to it, and it is known that if $\text{P} \neq \text{NP} \cap \text{coNP}$, then there are NP search problems that do not reduce to each other under Levin reductions, while their decision versions are NP -complete (and hence do reduce to each other) under Karp reductions (see, for example, [KM00; FFNR03]). To the best of our knowledge, until now the only natural examples of candidates to be NP -hard search problems without Levin reductions were precisely certain problems arising in the context of meta-complexity. One is the Minimum Circuit Size Problem (MCSP), for which Mazon and Pass [MP24] recently proved that a certain gap version is *not* NP -complete under Levin reductions, assuming the existence of indistinguishability obfuscation (iO). The other candidate was precisely the reduction from SAT to the Proof Size Problem for Resolution. Theorem 1.1 settles this, giving a two-way Levin reduction.

The existence of the extraction algorithm answers question (Q_1) in the affirmative: Resolution refutations of $\text{REF}_s(\varphi)$ *must leak a satisfying assignment*. This has a certain information-theoretic flavor: the fact that satisfying assignments can *always* be efficiently extracted implies that the most succinct description of a refutation of $\text{REF}_s(\varphi)$ must include the description of a satisfying assignment for φ . We can make this precise in the language of Kolmogorov complexity using the framework of *information efficiency* of Krajíček [Kra22], who studied the minimum time-bounded Kolmogorov complexity (Kt) of propositional proofs.

Theorem 1.2 (Assignment extraction as information efficiency, informal). *For every satisfiable CNF formula φ over n variables and $\text{poly}(n)$ clauses,*

$$\text{info}_{\text{Res}}(\neg\text{REF}(\varphi)) \approx \min\{\text{Kt}(\alpha \mid \varphi) \mid \varphi(\alpha) = 1\},$$

where $\text{info}_Q(\psi) := \min\{\text{Kt}(\pi \mid \psi) \mid \pi : Q \vdash \psi\}$ is Krajíček’s information efficiency function.

To the best of our knowledge, this is one of the first applications of Krajíček’s framework.

1.1.2 The Proof Analysis Problem for strong proof systems

Motivated by (Q_2) , we ask whether PAP is in **P** for strong proof systems. We conclude that the answer is likely negative by proving optimal conditional lower bounds in the form of **NP**-hardness for every proof system that p -simulates Extended Frege (EF).

Theorem 1.3 (NP-hardness of PAP_{EF} , informal). *For every propositional proof system S that p -simulates Extended Frege, the Proof Analysis Problem for S is NP-complete.*

This means that, unlike Resolution, strong proof systems are seemingly able to prove “trivial” Resolution lower bounds on satisfiable formulas without having to first prove that the underlying formula is satisfiable. In particular, this means Extended Frege is strong enough to *obfuscate* the satisfying assignments. As a consequence, for strong proof systems like Extended Frege, one cannot hope to prove they are **NP**-hard to automate following a strategy similar to that of [AM20].

The idea behind the hardness reduction highlights that the question of whether a proof system is analyzable boils down to whether it can efficiently prove any non-trivial Resolution lower bounds. In fact, the situation is rather dramatic: as soon as a strong enough system can prove *at least one* non-trivial Resolution lower bound, then the system immediately becomes hard to analyze.

Theorem 1.4 (Equivalence of analyzability and REF lower bounds, informal). *Let S be a reasonably strong propositional proof system. Assuming that there is some $\delta > 0$ such that $3\text{SAT} \notin \text{ioSIZE}[2^{\delta n}]$, the following are equivalent:*

- (i) *the proof system S is analyzable;*
- (ii) *for every unsatisfiable 3-CNF formula φ over n variables, every S -proof of the formula $\neg\text{REF}_{\text{poly}(n)}(\varphi)$ requires size at least $2^{\Omega(n)}$.*

1.1.3 Formalization of the Atserias–Müller lower bound in PV_1

The inspiration for why the extraction algorithm in Theorem 1.1 might exist in the first place comes from *witnessing theorems* in bounded arithmetic. We work here with Cook’s theory PV_1 and Buss’s S_2^1 , which are first-order theories of arithmetic formalizing polynomial-time reasoning. In these theories, if a statement of the form $\forall x \exists y \varphi(x, y)$ with a low-complexity $\varphi(x, y)$ is provable in the theory, then there exists a polynomial-time algorithm that *witnesses* y given x . This implies, in particular, that if a problem is proven **NP**-hard in one of these theories, then the reduction will be a Levin reduction.

The key observation for us is that the statement of the lower bound is itself of this form, a $\forall \Sigma_1^b$ sentence:

“for every formula φ and every Resolution refutation π of $\text{REF}(\varphi)$,
there exists a satisfying assignment for φ , or else π is large.”

Thus, if the previous statement were provable in PV_1 , we would get a polynomial-time function extracting satisfying assignments given φ and π .

While the extraction algorithm presented in Theorem 1.1 is given directly in natural language, it is still worth formalizing the lower bound in bounded arithmetic to obtain a variety of applications.

Theorem 1.5 (Atserias–Müller lower bound [AM20] in PV_1 , informal). *The theory PV_1 proves the statement that for every CNF formula φ over n variables and every size parameter $s \in \mathbb{N}$, if φ is unsatisfiable and π is a correct Resolution refutation of $\text{REF}_s(\varphi)$, then $|\pi| \geq 2^{\Omega(s/n^2)}$.*

Formalizations in bounded arithmetic tend to be particularly interesting when they lead to new proofs of known statements. This has been the case, for example, with Razborov’s formalizations of circuit lower bounds leading to the now-famous proof of Håstad’s switching lemma via a simpler counting argument [Raz95]. Remarkably, the method introduced by Razborov to formalize the switching lemma is recognized for enabling proofs to at least two major conjectures in combinatorics [ALWZ21; PP24]. Another example is a recent new proof of the Schwartz–Zippel lemma [AT24], proven via a hybrid argument formalizable in S_2^1 . Our formalization of the lower bound on REF formulas also relies on a new proof. We elaborate on this in the technical overview.

We remark that our bound of the form $2^{\Omega(s/n^2)}$ is slightly worse than the original one, which we state here for convenience.

Theorem 1.6 (Atserias–Müller lower bound [AM20]). *For every CNF formula φ over n variables and every size parameter $s \in \mathbb{N}$, if φ is unsatisfiable and π is a correct Resolution refutation of $\text{REF}_s(\varphi)$, then $|\pi| \geq 2^{\Omega(s/n)}$.*

The difference in the bound means that we can only show that PV_1 proves hardness of $\text{REF}_s(\varphi)$ for $s \geq n^3$. We leave it open whether PV_1 can achieve the original $2^{\Omega(s/n)}$ bound via a different argument. In any case, this is not particularly important for our applications. We comment on this further in the technical overview.

1.1.4 Formalization of Pudlák’s upper bound in Resolution

We complement the formalization of the lower bound with a formalization of the upper bound [Pud03], showing that there are short refutations of $\text{REF}(\varphi)$ whenever φ is satisfiable. This construction can be carried out by a constant-depth circuit and could be formalized in S_2^1 , but certainly also in much weaker theories. We prove the somewhat surprising fact that the construction can be proven correct in Resolution itself.

Theorem 1.7 (Pudlák’s upper bound [Pud03] in Resolution, informal). *There is a polynomial-size depth-2 Boolean circuit $P(\alpha, \varphi, s)$ of fan-in 2 that given a CNF formula φ , a satisfying assignment α , and $s \in \mathbb{N}$, outputs a Resolution refutation π of $\text{REF}_s(\varphi)$. Furthermore, the correctness of this circuit P has polynomial-size proofs in Resolution.*

That is, not only Resolution has short refutations of $\text{REF}(\varphi)$ when φ is satisfiable: Resolution can show that the circuits generating these refutations from satisfying assignments are correct. This, again, is in striking contrast with the fact that Resolution does not have small proofs of its own soundness [AB04].

1.1.5 Propositional fragments of Atserias–Müller: automatability in terms of REF formulas

The main consequence of the extraction algorithm together with its formalization in bounded arithmetic is the following precise characterization theorem relating the provability of a formula $\neg\varphi$ to the provability of the formula $\neg\text{REF}(\text{REF}(\varphi))$. (For the sake of clarity, we ignore for now the exact size parameters of the REF formulas, which are always some fixed polynomials; in general, when we write $S \vdash_{\text{poly}} \varphi$ we mean that S has polynomial-size proofs of φ , and by “reasonable proof system” we mean essentially that the system is closed under *modus ponens*.)

Theorem 1.8 (Propositional fragments of Atserias–Müller, informal). *Let S be a reasonable propositional proof system that simulates Extended Frege. Then, for every sequence $\{\varphi_n\}_{n \in \mathbb{N}}$ of unsatisfiable CNF formulas,*

$$S \vdash_{\text{poly}} \neg\varphi_n \quad \text{if and only if} \quad S \vdash_{\text{poly}} \neg\text{REF}(\text{REF}(\varphi_n)).$$

The lower bound on REF formulas says that for every unsatisfiable φ , the corresponding $\text{REF}(\varphi)$ is hard for Resolution, making $\text{REF}(\text{REF}(\varphi))$ unsatisfiable. The latter encodes the statement “ $\text{REF}(\varphi)$ is hard for Resolution”, and our theorem shows that when restricted to the reasoning power of a specific proof system S , such a lower bound has small proofs if, and only if, S has short proofs of the unsatisfiability of φ in the first place. That is, the *fragment* of the Atserias–Müller lower bound that has short proofs in S is precisely the one corresponding to the formulas that S can prove unsatisfiable with short proofs.

This characterization is surprisingly tight and has consequences for automatability and proof search. Since we can relate the proof size of φ in S to the proof size of $\text{REF}(\text{REF}(\varphi))$, this means that looking for proofs of $\text{REF}(\text{REF}(\varphi))$ can be a proxy for searching for proofs of φ .

Theorem 1.9 (Automatability in terms of REF formulas, informal). *For every reasonable proof system S that simulates Extended Frege as well as for Resolution itself,*

- (i) *S is automatable if, and only if, S is automatable exclusively on REF formulas;*
- (ii) *S is weakly automatable if, and only if, S is weakly automatable exclusively on REF formulas.*

We remark again that these REF formulas are always talking about Resolution, not about S . That is, for every strong enough proof system, *efficient proof search over all tautologies is equivalent to efficient proof search over Resolution lower bounds.*

Until now no such general structural result was known that related proof search generally to proof search for a particular class of formulas. This goes in line with a question of Pich and Santhanam [PS22], who asked whether automating a proof system on truth-table tautologies (i.e., formulas stating circuit lower bounds) implies the automatability of the system on all tautologies. We have proved that this is the case for the class of formulas stating Resolution lower bounds in place of truth-table tautologies.

1.1.6 Unprovability of Resolution lower bounds

Theorem 1.7, together with Theorem 1.6, further imply that true Resolution lower bounds can be essentially arbitrarily hard to prove. Namely, if S is a propositional proof system where $\{\varphi_n\}_{n \in \mathbb{N}}$ is a sequence of formulas that S cannot refute in polynomial size, then S cannot refute $\{\text{REF}(\text{REF}(\varphi_n))\}_{n \in \mathbb{N}}$ either.

Theorem 1.10 (Propositional unprovability of Resolution lower bounds, informal). *Let Q be a reasonable propositional proof system that simulates Resolution. If $\{\varphi_n\}_{n \in \mathbb{N}}$ is a sequence of hard unsatisfiable CNF formulas for Q , where φ_n has n variables and size $|\varphi_n| = \text{poly}(n)$, then*

- (i) *the formulas $\text{REF}_{n^2}(\varphi_n)$ over $N = \text{poly}(n)$ variables are all unsatisfiable and require size $2^{N^{\Omega(1)}}$ to be refuted in Resolution;*
- (ii) *yet, Q does not have polynomial-size refutations of the formulas $\text{REF}_{N^2}(\text{REF}_{n^2}(\varphi_n))$ stating quadratic lower bounds on $\text{REF}_{n^2}(\varphi_n)$.*

There is nothing special about quadratic lower bounds being unprovable—one can get arbitrarily small polynomial lower bounds by tweaking the encoding. See the discussion after Theorem 1.12.

We note that Iwama showed in 1997 that the Proof Size Problem for Resolution is NP-complete [Iwa97]. This means, in particular, that its complement is coNP-complete and hence, unless $\text{NP} = \text{coNP}$, no propositional proof system can efficiently derive *all* tautological REF formulas (i.e., all true Resolution lower bounds), or else there would be a polynomially bounded proof system. While this has a similar flavor to our result, our theorem is different in at least two aspects. First, from an explicit family of hard tautologies we obtain an explicit family of hard REF formulas for the system, in a generic way. Second, the parameters are essentially optimal: we identify a sequence of unsatisfiable formulas for which an exponential (and hence

maximal) Resolution lower bound holds —while the system Q in question cannot even prove a quadratic lower bound.

As a corollary of Theorem 1.10, for example, we get the first explicit lower bounds for REF formulas in bounded-depth Frege systems.

Corollary 1.11 (Hard REF formulas for bounded-depth Frege, informal). *For every $d \leq O(\log n / \log \log n)$, the formulas $\text{REF}(\text{REF}(\text{PHP}_n))$ are all unsatisfiable but require size $\exp(\Omega(n^{1/(2d+1)}))$ to be refuted in depth- d Frege systems.*

We build here on the best-known PHP lower bounds by Håstad [Hås23], but even better bounds could be obtained by applying the same argument to Tseitin formulas using the results of Håstad and Risse [HR22]. This corollary contrasts again with the open question of Pudlák [Pud20] about whether constant-depth Frege proves any of its own lower bounds.

Finally, using similar ideas, we obtain unconditional independence results for first-order theories of arithmetic.

Theorem 1.12 (First-order unprovability of Resolution lower bounds, informal). *Let T be a consistent first-order theory extending Robinson Arithmetic by a set of polynomial-time recognizable axioms. Then, there exists a sequence of unsatisfiable propositional formulas $\{\psi_N\}_{N \in \mathbb{N}}$ described uniformly by a polynomial-time algorithm, where ψ_N has N variables, such that*

- (i) *Resolution refutations of the formula ψ_N require size $2^{N^{\Omega(1)}}$;*
- (ii) *there exists $c > 0$ such that the theory T cannot prove $\Omega(N^c)$ lower bounds on the Resolution size of these refutations; that is, there is $N_0 \in \mathbb{N}$ such that the lower bound expressed by the first-order sentence $\forall N \forall \pi (N > N_0 \wedge \text{Ref}_{\text{Res}}(\psi_N, \pi) \rightarrow |\pi| > N^c)$ is unprovable in T .*

We remark that the theory T in this theorem can be arbitrary strong. This implies that, unconditionally, theories like Peano Arithmetic (PA) cannot prove all true Resolution lower bounds. The same ideas apply to Zermelo–Fraenkel Set Theory (ZFC) and similarly powerful formal systems.

We also note that the constant $c > 0$ in the exponent of the unprovable lower bound depends on the definition of ψ_N . In general, one can alter ψ_N to get an unprovable lower bound of the form $\Omega(N^c)$ for any fixed constant $c > 0$.

1.2 Technical overview

Next we provide a technical overview of the main proof ideas and how these are combined to yield our main results and corollaries.

1.2.1 Assignment extraction: techniques

We obtain the extraction algorithm in Theorem 1.1 by derandomizing the proof of the Resolution lower bound for the REF formulas. The original proof revolves around the concept of *block-width* (called *index-width* in [AM20]) in Resolution refutations of $\text{REF}_s(\varphi)$. The variables of the formula are arranged into s blocks, each encoding a clause in the purported refutation of size s . The block-width of a refutation π is then the largest number of blocks mentioned in a clause of the refutation π . The proof proceeded in two steps:

1. derive a *block-width lower bound*, showing that if $\varphi \notin \text{SAT}$, then the block-width of any refutation of $\text{REF}_s(\varphi)$ must be large;
2. by a *random restriction argument*, argue that if the refutation π is small, there exists a restriction that makes the block-width of the restricted refutation small, contradicting the previous point.

Our algorithm works by following these steps in reverse. First, given a refutation π from which we want to extract a satisfying assignment, instead of sampling a restriction at random from a specific distribution, we construct a restriction *deterministically* in a greedy fashion, tailored to the specifics of π . This is reminiscent of the kind of greedy deterministic restrictions used by Cook and Pitassi [CP90] to formalize Haken’s lower bound for the pigeonhole principle in bounded arithmetic, and more broadly in the style of Beame and Pitassi [BP96], Clegg, Edmonds, and Impagliazzo [CEI96] and Ben-Sasson and Wigderson [BW01]. Our algorithm runs in deterministic polynomial time and always succeeds in finding a restriction that reduces the block-width to $O(\sqrt{s \log |\pi|})$.

In the second step, we look at the proof of the block-width lower bound and interpret it as a Prover-Delayer game in the style of Atserias and Dalmau [AD08]. The Prover issues queries about the values of the variables of $\text{REF}_s(\varphi)$, or forgets previously recorded such values, and the Delayer replies following a concrete strategy that allows them to keep playing until a large number of blocks appear queried. Our algorithm traverses the Resolution refutation guided by the Delayer’s strategy in the Prover-Delayer game. We can then prove that this Delayer’s strategy will, after a polynomial number of steps, reach either

- (a) a clause of *high block-width*, or
- (b) a clause encoding a *satisfying assignment* to φ .

Since the greedy deterministic restriction in the first step made sure the block-width is small, the Delayer will be guaranteed to find a satisfying assignment.

We note that our deterministic restriction only achieves a reduction of block-width to $O(\sqrt{s \log |\pi|})$, while using a random restriction one could achieve up to $O(\log |\pi|)$. In fact, if one allows randomness in the extraction algorithm, then an argument similar to the random restriction of [AM20] yields a zero-error probabilistic polynomial-time extraction algorithm that works even when the refutation π being analyzed is for the formula $\text{REF}_{n^2}(\varphi)$. In contrast, the price to pay for determinism is that the size parameter s should be at least n^3 .

1.2.2 NP-hardness of the Proof Analysis Problem for Extended Frege

The idea behind the hardness proof in Theorem 1.3 boils down to the following crucial insight: as soon as a proof system S proves *at least one* true Resolution lower bounds for some specific formula ψ , then S can also prove lower bounds on formulas of the form $\varphi \vee \psi$ for any φ by completely ignoring φ . However, since φ might be satisfiable, $\varphi \vee \psi$ might be overall satisfiable, yet S will prove a lower bound “agnostically”, without revealing a satisfying assignment.

In more detail, suppose the proof system S (in our case, Extended Frege) can uniformly prove Haken’s lower bound for the pigeonhole principle formulas [Hak85]. That is, there is a polynomial-time algorithm that given a number n and t outputs a S -proof π such that

$$\pi : S \vdash \neg \text{REF}_t(\text{PHP}_n).$$

Now, let φ be a propositional formula over n variables different from those of PHP_n . If the system S is strong enough, it will be able to uniformly construct a proof π' such that

$$\pi' : S \vdash \neg \text{REF}_t(\varphi \vee \text{PHP}_n).$$

The proof π' goes roughly as follows: if φ is satisfiable then so is $\varphi \vee \text{PHP}_n$, but S being strong enough to prove the soundness of Resolution, it can conclude that there will be no refutation of any size, let alone of size t ; if, on the other hand, φ is unsatisfiable and there is a size- t refutation of $\varphi \vee \text{PHP}_n$, then any restriction to the variables of φ necessarily reduces $\varphi \vee \text{PHP}_n$ to just PHP_n . Assuming S is strong enough

to prove that Resolution is closed under restrictions, S would conclude that there is a size- t refutation of PHP_n —contradicting Haken’s lower bound, which we assumed can itself be proven in S .

Observe how the previous argument performs a proof by cases that works regardless of the satisfiability of φ . In particular, even in the case that a particular φ is satisfiable, an explicit satisfying assignment never appears in the proof—we only reason about a hypothetical one. For the argument to go through for $S = \text{EF}$, everything we need is for EF to prove Haken’s lower bound, which was already formalized in this system by Cook and Pitassi [CP90].

1.2.3 Formalization of the upper and lower bounds

A large part of our technical contribution consists in formalizing the proofs leading to the NP-hardness of automating Resolution. We summarize below some of the challenges encountered, and the solutions devised.

The Atserias–Müller lower bound in PV_1 . Different proofs of the lower bound exist in the literature, but none of them seem directly formalizable in PV_1 . The original proof has the caveat of the random restriction argument, which might be formalizable in Jeřábek’s theory APC_1 , but likely not in PV_1 . In addition, the block-width lower bound is proven by relating small refutations to the canonical exponential-size tree-like refutation of any formula, which could be hard to reason about in bounded theories.

In the work of de Rezende, Göös, Nordström, Pitassi, Robere, and Sokolov [dRGN+21] two alternative proofs were presented. The first proof consists of a random restriction followed by a block-width lower bound proven via a reduction to the retraction weak pigeonhole principle. The random restriction presents the same formalization issues as the original proof, and the block-width reduction is equally problematic: the decision tree reduction they use has low depth, which is necessary to transfer the lower bounds on (block-)width, but the size of the decision tree itself seems to be super-polynomial, and hence cannot be reasoned about in PV_1 . The second proof of de Rezende et al. uses this same block-width lower bound followed by the size-width trade-offs of Ben-Sasson and Wigderson [BW01]. The block-width lower bound is still problematic, of course, but in addition to this, the statement of Ben-Sasson and Wigderson—“for every small Resolution refutation, there exists another Resolution refutation in small width”—is itself impossible to formalize in bounded arithmetic. The reason for this is that, as demonstrated by Thapen [Tha16], in general these narrow proofs can require super-polynomial size and therefore the statement of Ben-Sasson and Wigderson cannot possibly be a bounded formula.

Finally, Garlík [Gar19] has proven lower bounds on the REF formulas for the so-called non-relativized encoding. Unfortunately for us, his proofs encounter the same barrier: they rely on random restriction arguments, which are in any case more involved than the original ones.

We resolve these issues by coming up with new proof, inspired by the extraction algorithm, that modifies both ingredients in the original proof, yielding Theorem 1.5. The random restriction argument is replaced by a greedy deterministic restriction just like the one used in the extraction algorithm. For the block-width lower bound, we show that the argument can be completely described by a Prover-Delayer game without referring to the exponential-size canonical tree-like refutation, making the entire proof formalizable in PV_1 .

We remark that moving from the random restriction to the deterministic one comes at the cost of a slightly worse lower bound. The original size bound on REF formulas is of the form $2^{\Omega(s/n)}$ and hence yields $2^{\Omega(n)}$ Resolution size lower bounds for all REF_s formulas with $s \geq n^2$. Our deterministic restriction, in line with the parameters of the extraction algorithm, achieves a lower bound of $2^{\Omega(s/n^2)}$ which is exponential in $\Omega(n)$ for $s \geq n^3$. It seems reasonable that the original proof with the random restriction can be formalized in APC_1 , but we have not carried out this formalization.

Pudlák’s upper bound in Resolution. For the upper bound in Resolution (Theorem 1.7), our proof is based on a careful analysis of the construction that makes it possible to describe the construction by a low-depth circuit. Carrying out the proof of the correctness of this circuit in Resolution is tedious, but ultimately clear once the right description of the circuit is provided.

1.2.4 Consequences

Characterization of the propositional fragments of Atserias–Müller. Our characterization theorem (Theorem 1.8) is a consequence of the formalization of the lower bound in PV_1 (Theorem 1.5) and the upper bound in Resolution itself (Theorem 1.7). Those results, in the propositional setting, imply that

1. for the extraction algorithm E , we have $EF \vdash \text{REF}(\text{REF}(\varphi), \pi) \rightarrow \text{SAT}(\varphi, E(\varphi, \pi))$; and
2. for Pudlák’s algorithm P , we have $\text{Res} \vdash \text{SAT}(\varphi, \alpha) \rightarrow \text{REF}(\text{REF}(\varphi), P(\varphi, \alpha))$.

Here, we highlight the variables of the REF formula encoding a refutation π as the second argument of REF . In particular, the Resolution proof of the correctness of P is also possible in EF . Then, simple use of contraposition allows us to go from $\neg\varphi$ to $\neg\text{REF}(\text{REF}(\varphi))$, and vice versa. That is, if EF can prove $\neg\varphi$, then it can also prove it in the encoding $\neg\text{SAT}(\varphi, \alpha)$, and when substituting $E(\varphi, \pi)$ for α , where π are the only free variables, contraposition on item (1) gives us that EF derives $\neg\text{REF}(\text{REF}(\varphi), \pi)$. The other direction is analogous.

Automatability in terms of REF formulas. For the characterization of automatability in Theorem 1.9 to go through we build on Theorem 1.8 and additionally show that the characterization given there is not only in terms of proof size, but it is actually constructive. Given a proof of $\neg\varphi$ in S we can efficiently construct a proof of $\neg\text{REF}(\text{REF}(\varphi))$, and vice versa. In this way, searching for proofs of $\text{REF}(\text{REF}(\varphi))$ is a proxy for the proofs of φ .

Remarkably, our proof techniques fail for proof systems strictly between Extended Frege and Resolution. The upper bound in Resolution does imply that from a refutation of $\text{REF}(\text{REF}(\varphi))$ we can obtain a refutation of φ . Unfortunately, it is our extraction algorithm (Theorem 1.1) what guaranteed that if φ has a refutation of size t , then $\text{REF}(\text{REF}(\varphi))$ has a refutation of size $\text{poly}(t)$. In Extended Frege this is true thanks to the extraction algorithm, but it seems conceivable that weaker systems might be able to easily prove $\neg\varphi$ without being able to prove $\neg\text{REF}(\text{REF}(\varphi))$ efficiently. (For Resolution itself this result does go through, for the more ad-hoc reason that REF formulas talk about Resolution itself).

Unprovability of Resolution lower bounds. For Theorem 1.10 we exploit Theorem 1.7: if there is a short refutation of $\text{REF}(\text{REF}(\varphi))$, then there is a short refutation of φ . Since we formalized the upper bound construction in Resolution, the result applies to any proof system that contains Resolution (and behaves naturally in the sense that it is closed under *modus ponens*). Then, if φ is a hard formula for Q and Q simulates Resolution, we have that $\text{REF}(\varphi)$ is unsatisfiable. By the lower bound on REF formulas (Theorem 1.6), this formula is exponentially hard for Resolution, making $\text{REF}(\text{REF}(\varphi))$ unsatisfiable as well—but hard to refute for Q .

In the first-order setting, Theorem 1.12 relies again on the formalization of the upper bound on REF formulas. This time, instead of starting from a sequence of hard propositional formulas, we can leverage Gödel’s second incompleteness theorem to start from a sentence (the consistency of T) that is unconditionally unprovable in T . From this follows that T cannot prove the soundness of a certain propositional system based on T (the so-called *strong proof system of T* [Pud20]). We then consider the $\text{REF}(\cdot)$ formula around these soundness statements. We conclude that if T could derive the lower bound on the $\text{REF}(\cdot)$ formulas in question, it would also be able to prove the soundness of the strong proof system of T and, as a consequence,

T would derive its own consistency. Since Gödel’s incompleteness theorem gives us sentences that are unconditionally independent of T , the corresponding Resolution lower bounds are also unconditionally unprovable in T . This works essentially for any theory of arithmetic subject to Gödel’s incompleteness phenomenon, and does not rely on any complexity-theoretic assumptions.

1.3 Related work

Our work fits into a trend in complexity theory concerned with the meta-mathematics of computational complexity, which has gained remarkable momentum in recent years. Most of this work has been primarily concerned with the formalization of cornerstone results of computational complexity in bounded arithmetic and establishing unprovability and logical independence as barrier results. The literature is too vast to review here, so refer the reader to the recent survey of Oliveira [Oli25]. In parallel, there has been a growing body of work deploying tools and ideas from mathematical logic to prove complexity-theoretic statements (see, e.g., [Gay21; Mül21; Kha22a; Kha22b; PS22; Nar22; PS23; Kra23; Gay23; Kha24; ACG24; AKPS24; Kra24; Nar24; Gay24]). Our work continues in this direction.

Two recent works conceptually related to our investigations on REF formulas merit further discussion. Santhanam and Tzameret [ST21] initiated a general study of REF formulas for arbitrarily strong proof systems. In particular, they studied *iterations* of these formulas, which are reminiscent of the nested $\text{REF}(\text{REF}(\varphi))$ formulas that feature in our work. Their REF formulas are not limited to Resolution, and they consider the iterated version of REF^Q when REF^Q talks about an arbitrarily strong proof system Q . While we are unable to connect our work on analyzability to their results, our characterization of proof size in terms of REF formulas (Theorem 1.8) has conceptual ties to their Iterated Lower Bounds Hypothesis.

The other relevant work is the research of Li, Li, and Ren [LLR24], who studied the provability of Resolution lower bounds in relativized theories of bounded arithmetic in the context of TFNP . Until their work, the only formalization of proof complexity lower bounds that we are aware of is that of Cook and Pitassi [CP90]. Li, Li, and Ren studied so-called *refuter problems* in proof complexity: given a purported Resolution refutation of, say, PHP_n , which is smaller than the known lower bounds, find a mistake in the proof (which must certainly exist, due to these very lower bounds). They connect the provability of lower bounds to the complexity of solving these refuter problems in subclasses of TFNP . While their results yield formalizations of some proof complexity lower bounds, our results are essentially incomparable. First, their provability results are for *relativized* theories of bounded arithmetic, where the given Resolution refutation is accessed through an oracle, while our proofs are in the non-relativized theories, where we can quantify over the objects in question. Second, they consider the provability of lower bounds for explicit families of tautologies like the pigeonhole principle or the Tseitin formulas. In contrast, the lower bound we are concerned is a sort of *meta lower bound*: it tells us that the $\text{REF}(\varphi)$ formulas are hard whenever φ is unsatisfiable. We believe, however, that the TFNP perspective on analyzability might shed light on some of our open questions.

1.4 Open problems

Analyzability of constant-depth Frege and other weak proof systems. Similar techniques to those of [AM20] have been employed to prove the NP-hardness of automating other weak proof systems like Regular and Ordered Resolution [Bel20; BY24], k -DNF Resolution [Gar24], Cutting Planes [GKMP20], Nullstellensatz and Polynomial Calculus [dRGN+21], the OBDD proof system [IR22] and, more recently, even AC^0 -Frege [Pap24]. All proof systems weaker than Resolution are analyzable just because Resolution is (i.e, their corresponding PAP problems are in P), since analyzability is downwards closed under p-simulations. For the stronger systems, the question remains open. Are these systems analyzable? What about their search versions?

We highlight the analyzability of constant-depth Frege as a particularly interesting problem. While we have proven some unconditional lower bounds on REF formulas here, it is open whether AC^0 -Frege can prove any true Resolution lower bounds at all. It has been conjectured in the past that the PHP lower bound might be formalizable in these systems, at least in quasi-polynomial size. If this was possible, the NP-hardness of PAP_{EF} in Theorem 1.3 could be improved all the way to these systems.

FP-completeness of the search version of PAP_{Res} . While we have shown that assignment extraction can be performed in polynomial time, our algorithm does not seem to be possible anywhere below P . The algorithm seems hard to parallelize, which raises the question of whether the search version of PAP_{Res} is in NC or even below. This is related to the question of whether the formalization of the lower bound on REF formulas is provable in theories weaker than PV_1 . If the statement was provable in, say, VNC^1 , witnessing theorems would give us an extraction algorithm in FNC^1 . We conjecture that this improvement is impossible, and that the search problem of PAP_{Res} is complete for FP, but we are unable to prove it. The reduction, if true, likely requires some new technical idea. This would imply, among other things, that V^0 does not prove the lower bound on REF formulas, unconditionally.

On the weak automatability of Resolution. Recall that a proof system is *weakly automatable* if there exists a proof system that p-simulates it and is automatable. By our Theorem 1.9, the weak automatability of Resolution is equivalent to a proof system Q simulating Resolution and being automatable on REF formulas. If $Q \geq EF$, then our theorem would imply that Q itself would be automatable on all formulas, hitting cryptographic hardness results [KP98; BPR00; BDG+04; ACG24]. However, if Q is strictly weaker than EF, our statement does not apply and the automatability of Q on REF formulas does not imply automatability on all formulas. This does not seem to contradict any hardness assumptions. Of course, no such Q is known to be efficiently automatable on Resolution lower bound statements, but this raises again the question of whether some non-trivial algorithm weakly automating Resolution might be plausible.

1.5 Structure of the paper

The paper is structured as follows. After the preliminaries in Section 2, we dedicate Section 3 to formally defining the Proof Analysis Problem and stating some basic facts about it. Section 4 proves Theorem 1.1, describing the algorithm for the search version of PAP_{Res} and Theorem 1.2. Section 5 proves Theorem 1.3, giving NP-hardness of PAP_{EF} and stronger systems. In Section 6 and Section 7 we formalize, respectively, the Resolution lower bound and upper bound on REF formulas that yield Theorem 1.5 and Theorem 1.7. Section 8 translates this to the propositional setting to show that Extended Frege has polynomial-size proofs of the NP-hardness of automating Resolution. Finally, Section 9 gives the proof of the characterization in Theorem 1.8 and proves Theorem 1.9 on the equivalence of automatability of REF formulas, while Section 9.3 proves Theorem 1.10 and Theorem 1.12 on the unprovability of Resolution lower bounds.

2 Preliminaries

We assume the reader to be familiar with the central concepts of computational complexity theory. Below, we review the essential definitions and facts involving proof complexity and bounded arithmetic that feature in the paper. For a more comprehensive treatment of proof complexity, we refer to Krajíček [Kra19]. For bounded arithmetic, the recent survey of Oliveira [Oli25] covers all the necessary material in the style of the meta-mathematics of computational complexity, which aligns with the style of our work. Other classical texts in logic and bounded arithmetic also cover these contents (see, e.g., [HP93; Kra95; Bus97; Bus98]).

2.1 Levin reductions

For decision problems $A, B \subseteq \{0, 1\}^*$, we use the notation $A \leq_m^P B$ to express that A *many-one reduces* (or *Karp reduces*) to B , meaning that there is a polynomial-time computable function f such that for all $x \in \{0, 1\}^*$ we have $x \in A$ if and only if $f(x) \in B$. We will be concerned with a strengthening of Karp reductions for *search problems*. A search problem for us is a relation $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$. A search problem R is in **FP** if there exists a polynomial-time function that on input x , outputs some y such that $(x, y) \in R$, if such a y exists.

We will say that R_1 *Levin reduces* to R_2 and write $R_1 \leq_{\text{Levin}}^P R_2$ if there exists a triple of polynomial-time computable functions (f, g, h) such that for all $x, w \in \{0, 1\}^*$, it holds that (i) if $(x, w) \in R_1$, then $(f(x), g(x, w)) \in R_2$, and (ii) if $(f(x), w) \in R_2$, then $(x, h(x, w)) \in R_1$. Whenever membership in R_1 and R_2 is checkable in polynomial time, it holds that the sets $\text{dom } R_1$ and $\text{dom } R_2$ are in **NP** and $\text{dom } R_1 \leq_m^P \text{dom } R_2$.

2.2 Proof complexity

Following the classical definition of Cook and Reckhow [CR79], a *propositional proof system* S for the set **TAUT** of propositional tautologies is a polynomial-time function $S : \{0, 1\}^* \rightarrow \text{TAUT}$ whose range is exactly **TAUT**. We think of S at the polynomial-time verifier mapping proofs to the statements they prove; i.e., if $S(\pi) = \varphi$, then we say π is an S -proof of φ . It is often convenient to think of a proof system as establishing *unsatisfiability*; thus, if φ is an unsatisfiable formula and π is an S -proof of the tautology $\neg\varphi$, then we say that π is an S -*refutation* of φ , or an S -proof of the unsatisfiability of φ . Since we deal exclusively with classical logic, here and below we tacitly gloss over the distinction between the formulas $\neg\neg\varphi$ and φ ; this is particularly useful for literals ℓ , where $\neg\ell$ is sometimes used to denote the complementary literal.

For a tautology φ and a proof system S , we denote by $\text{size}_S(\varphi) := \min_{\pi: S(\pi)=\varphi} |\pi|$ the *size* of its smallest S -proof. A proof system S is *polynomially bounded* if there exists a constant $c \in \mathbb{N}$ such that for all $\varphi \in \text{TAUT}$ we have $\text{size}_S(\varphi) \leq |\varphi|^c$. For a sequence $\varphi = \{\varphi_n\}_{n \in \mathbb{N}}$ of tautologies, we write $S \vdash_{\text{poly}} \varphi$ or simply $S \vdash_{\text{poly}} \varphi_n$ to express that $\text{size}_S(\varphi_n) = |\varphi|^{O(1)}$ as n grows. When we want to emphasize that it is via a specific proof π that S proves φ_n , we write $\pi : S \vdash \varphi_n$. More generally, for $s \in \mathbb{N}$, we write $S \vdash_s \varphi_n$ to express that there exists an S -proof π of size $|\pi| \leq s$ such that $\pi : S \vdash \varphi_n$.

We say that a proof system S *simulates* another system Q , written $S \geq Q$, if there exists a constant $c \in \mathbb{N}$ such that for every $\varphi \in \text{TAUT}$ we have $\text{size}_S(\varphi) \leq \text{size}_Q(\varphi)^c$. We additionally say that S *p-simulates* Q and write $S \geq_p Q$ if there exists a polynomial-time computable function sending Q -proofs to S -proofs of the same formula; i.e., there exists a polynomial-time computable function f such that for every $\varphi \in \text{TAUT}$ and every $\pi : Q \vdash \varphi$, we have $f(\pi) : S \vdash \varphi$. We say that two proof systems S and Q are *polynomially equivalent* if $S \geq_p Q$ and $Q \geq_p S$. A proof system S is *optimal* if $S \geq Q$ for every propositional proof system Q , and respectively *p-optimal* if $S \geq_p Q$ for every propositional proof system Q .

A *literal* is a propositional variable or its negation. Given a formula $\varphi(x_1, \dots, x_n)$, a *literal substitution* is a mapping of the form $\rho : \{x_1, \dots, x_n\} \rightarrow \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n, 0, 1\}$ that replaces variables by other literals or substitutes constants in their place. We denote by $\varphi_{\upharpoonright \rho}$ the substituted formula $\varphi(\rho(x_1), \dots, \rho(x_n))$, with the convention that every resulting occurrence of $\neg\neg x_i$ is replaced by x_i . A *restriction* is a particular case of a variable substitution, where all variables are mapped to either 0, 1, or themselves. We say that a proof system S is *closed under substitutions* (respectively, *closed under restrictions*) if there exists a constant $d \in \mathbb{N}$ such that for every tautology φ and every literal substitution (respectively, restriction) ρ , it holds that $\text{size}_S(\varphi_{\upharpoonright \rho}) \leq \text{size}_S(\varphi)^d$. All the explicit proof systems dealt with in this work (i.e., the ones described below, like Resolution or Frege or Extended Frege systems) are closed under literal substitutions. In these cases, a proof of the substituted formula can be obtained directly by applying the substitution line by line to every formula appearing in a proof π of φ , and we hence denote by $\pi_{\upharpoonright \rho}$ the corresponding substituted proof of $\varphi_{\upharpoonright \rho}$.

2.2.1 Resolution

A central proof system in this work is Resolution (Res). We usually see this as a refutation system for CNF formulas. Accordingly, we sometimes write $\pi : \text{Res} \vdash \neg\varphi$ for a CNF formula φ , to mean that π is a Resolution refutation of φ , hence a proof of the tautology $\neg\varphi$. In this way, Res is a Cook–Reckhow proof system for the fragment of TAUT made of the formula of the form $\neg\varphi$, where φ is an unsatisfiable CNF formula. Through the standard Tseitin transformation of an arbitrary propositional formula into equisatisfiable CNF form, Res can also be seen as a Cook–Reckhow proof system for TAUT itself; we do not need the details of this in this paper.

A *literal* is a propositional atom or its negation, a *clause* is a disjunction of literals, and a *CNF formula* is a conjunction of clauses. We see clauses as sets of literals, and write simply $C \subseteq D$ to express that C is a subclause of D . A *Resolution refutation* of an unsatisfiable CNF formula $\varphi = C_1 \wedge \dots \wedge C_m$ over variables $x_1, \dots, x_n$ is a sequence $D_1, \dots, D_s$ of clauses over $x_1, \dots, x_n$ such that $D_s = \perp$, denoting the empty clause, and for every $i \in [s - 1]$, the clause D_i either (a) is one of the clauses $C_1, \dots, C_m$ of φ , or (b) is a *weakening* of a previous clause, meaning that $D_i \supseteq D_j$ for some $1 \leq j < i$, or (c) has been obtained from two previous clauses $D_j = A \vee x$ and $D_k = B \vee \neg x$, for $j, k < i$, by an application of the *Resolution rule*:

$$\frac{A \vee x \quad B \vee \neg x}{A \vee B} \quad (\text{Res})$$

We say that $A \vee B$ is obtained by *resolving* over x . The *length* of π , denoted by $\text{length}(\pi)$, is s . We often write $\varphi \vdash_{\text{Res}}^s \perp$ to indicate that there exists a Resolution refutation of φ in length s .

To every Resolution refutation π we can associate a directed acyclic graph in a natural way, and we often do so implicitly. We denote by $\text{depth}(\pi)$ the length of the longest path in the dag, starting from the root labeled by the empty clause $\perp$. The number of vertices in this graph is precisely $\text{length}(\pi)$.

Resolution is *implicationally complete*, meaning that for every pair of CNF formulas φ and ψ , if $\varphi \models \psi$, there exists a derivation of each clause of ψ from the clauses of φ . We write $\varphi \vdash_{\text{Res}} \psi$ to express that such a collection of derivations exist.

We will also deal with a mild extension of the Resolution system, known as *k-DNF Resolution* [Kra01], denoted $\text{Res}(k)$ for $k \geq 1$. The system $\text{Res}(k)$ is also a refutational system, but lines are *k-DNF* formulas, which are unbounded fan-in disjunctions of *k-terms*, conjunctions of up to k literals. A clause is a 1-DNF. The system consists of a weakening and an introduction rule,

$$\frac{A}{A \vee B} \quad (\text{Weak}) \quad \frac{A \vee \ell_1 \quad B \vee (\ell_2 \wedge \dots \wedge \ell_s)}{A \vee B \vee (\ell_1 \wedge \dots \wedge \ell_s)} \quad (\wedge\text{-Intro})$$

together with a Cut rule that generalizes the Resolution rule,

$$\frac{A \vee (\ell_1 \wedge \dots \wedge \ell_s) \quad B \vee \neg\ell_1 \vee \dots \vee \neg\ell_s}{A \vee B} \quad (\text{Cut})$$

where A and B are *k-DNF* formulas and $s \leq k$.

It is easy to see that Resolution (Res) corresponds to $\text{Res}(1)$.

2.2.2 Frege systems

Throughout this work we reason about Resolution refutations within much stronger systems for propositional logic. A *Frege system* [CR79] consists of a finite set of axiom schemas and inference rules that are sound and implicationally complete for the language of propositional tautologies built from the Boolean connectives negation ($\neg$), conjunction ($\wedge$), and disjunction ($\vee$). A *Frege proof* is then a sequence of formulas where each formula is obtained by either substitution of an axiom schema or by application of an inference

rule on previously derived formulas. The specific choice of rules does not affect proof size up to polynomial factors, as long as there are only finitely many rules and these are sound and implicational complete [CR79]. We refer to Cook and Reckhow [CR79] or Krajíček [Kra19, §2.1] for specific examples of choices for these rules and axioms. One can alternatively define Frege systems in the formalism of Natural Deduction or the Sequent Calculus for classical propositional logic, but we will not be concerned with these syntactic details.

Of central importance for us is the *Extended Frege* (EF) system [CR79], in which proof lines can be succinctly written as Boolean circuits rather than formulas [Jeř05]. In general, for a circuit class C , one can consider the proof system C -Frege, in which lines are restricted to be Boolean circuits of that type. We are particularly interested in the AC_d^0 -Frege systems, in which lines are restricted to be Boolean circuits of unbounded fan-in and constant depth d . We also consider more generally *bounded-depth Frege systems*, where the depth d is bounded, but not necessarily a constant.

For bounded-depth Frege systems, we have strong lower bounds available. The most famous such lower bound is the one for the *Pigeonhole Principle* (PHP). For every $m \in \mathbb{N}$ and $n \in \mathbb{N}$ such that $m > n$, the formula PHP_n^m stands for the CNF formula over variables $p_{i,j}$ for $i \in [m]$ and $j \in [n]$ consisting of the clauses

$$\bigvee_{j \in [n]} p_{i,j} \quad \text{for all } i \in [m], \quad (\text{PHP-1})$$

$$\neg p_{i,j} \vee \neg p_{i',j} \quad \text{for all } i, i' \in [m], i \neq i' \text{ and } j \in [n] \quad (\text{PHP-2})$$

We sometimes denote by PHP_n the formula PHP_n^{n+1} .

Strong lower bounds are known on the proof complexity of the pigeonhole principle for bounded-depth Frege systems [Ajt94; PBI93; KPW95; FU96]. Here we state only a simplified version of the best such lower bound, proven by Håstad [Hås23].

Theorem 2.1 ([Hås23]). *For every $d \leq O(\log n / \log \log n)$, depth- d Frege systems require size at least $\exp(\Omega(n^{1/(4d-2)}))$ to prove $\neg \text{PHP}_n$.*

Finally, we often consider extensions of Extended Frege by sets of additional axioms. For a set $A \subseteq \text{TAUT}$ of tautologies that is recognizable in polynomial time, the system $\text{EF} + A$ refers to Extended Frege extended with the axiom schemas that allow (formula) substitution instances of any formula in A .

2.2.3 Automatability and proof search

The notion of automatability, introduced by Bonet, Pitassi, and Raz [BPR00], is a natural formalization of efficient proof search in propositional proof system. We say that a proof system S is *automatable* if there exists a constant $c \in \mathbb{N}$ and an algorithm that given a propositional tautology φ , outputs an S -proof of φ in time $(|\varphi| + \text{size}_S(\varphi))^c$, meaning that the proof search algorithm succeeds in finding a proof of size polynomial in the size of the shortest one.

Even when a system might not be automatable, it seems natural to ask whether there exists a system Q that p -simulates S and is itself automatable. In this case, we say that S is *weakly automatable* [AB04]. Weak automatability is equivalent to the existence of an automating algorithm where the output proof belongs to a system $Q \geq_p S$ rather than S itself. In particular, weak automatability is closed downwards under p -simulation.

A more restrictive notion of proof search is given by the *Proof Size Problem*. Associated to any propositional proof system S we can define the *Proof Size Problem for S* (PSP_S), defined as the language

$$\text{PSP}_S := \{(\varphi, 1^s) \mid \text{there is an } S\text{-proof of } \varphi \text{ in size } s\}.$$

Automating S entails approximating minimum proof-size to a polynomial, in polynomial time.

2.3 Bounded arithmetic

We heavily rely on the connection between propositional proof complexity and (weak) theories of arithmetic. We assume familiarity with basic knowledge of first-order logic and introduce the main theories we are concerned with.

2.3.1 The theories PV_1 and S_2^1

Theories of bounded arithmetic capture various forms of feasible reasoning and act as a uniform counterpart of propositional proof systems. The main tool to capture feasibility in mathematical reasoning is to bound the complexity of formulas over which one can apply induction.

Cook's PV_1 . Cook's theory PV_1 [Coo75; KPT91] is an attempt to make precise the idea of polynomial-time reasoning. It is a universal theory whose vocabulary $\mathcal{L}_{PV}$ consists of a function symbol for each polynomial-time function, and the axioms are precisely the recursive definitions of these functions via composition and limited recursion on notation, in the style of Cobham's functional definition of FP [Cob65]. The theory further admits induction on quantifier-free formulas, which define precisely polynomial-time predicates.

The formal definition of PV_1 is rather technical and the details are not particularly relevant to our proofs, so we refer the reader to Krajíček's textbook [Kra95, Definition 5.3] for the details. The reason we rarely care about the technicalities of PV_1 is that we often work instead in the theory S_2^1 of Buss, which happens to be conservative over PV_1 for the classes of formulas we are interested in. We discuss this next.

Buss's S_2^1 . We see S_2^1 as a theory sitting in between Robinson's Arithmetic Q and Peano Arithmetic PA. Let $\mathcal{L}_{PA}$ denote the language of Peano Arithmetic, $\mathcal{L}_{PA} := \{0, 1, +, \cdot, <\}$. The axioms of PA consist first of the axioms of Robinson's arithmetic Q, which define the basic behavior of the symbols of $\mathcal{L}_{PA}$ (see, for example, [Kra19, §7.4.3] for a definition), together the *Induction Schema*

$$(\varphi(0) \wedge \forall x(\varphi(x) \rightarrow \varphi(x+1))) \rightarrow \forall x\varphi(x), \quad (\text{IND}_\varphi)$$

available for every formula φ .

The language of S_2^1 is the first-order language of bounded arithmetic, $\mathcal{L}_{BA} := \{0, 1, +, \cdot, <, \lfloor \cdot \rfloor, \lfloor \cdot / 2 \rfloor, \#\}$. This extends the language of Peano Arithmetic $\mathcal{L}_{PA}$ above by the symbols $\lfloor x \rfloor$, $\lfloor x/2 \rfloor$ and $x\#y$. The standard interpretation of $\lfloor x/2 \rfloor$ is clear. The notation $\lfloor x \rfloor$ denotes the length of the binary encoding of the number x , $\lceil \log(x+1) \rceil$, while the *smash symbol* $x\#y$ stands for $2^{\lfloor x \rfloor \cdot \lfloor y \rfloor}$.

For a term t in the language of bounded arithmetic and a variable x that does not appear in t , a formula of the form $\forall x(x < t \rightarrow \varphi(x))$ or $\exists x(x < t \wedge \varphi(x))$ is called a *bounded formula*. The quantifiers guarded by the bounds on x are called *bounded quantifiers* and we simply write $\forall x < t(\varphi(x))$ and $\exists x < t(\varphi(x))$. If the bounded quantifiers are of the form $\forall x < \lfloor s \rfloor$ or $\exists x < \lfloor s \rfloor$ for some term s , then they are called *sharply bounded quantifiers*. The *hierarchy of bounded formulas* consists of the classes Σ_n^b (and Π_n^b), for $n \geq 1$, which are defined by counting the alternations of bounded quantifiers ignoring the sharply bounded ones, starting with an existential (respectively, universal) one. The class Δ_n^b consists of all formulas that admit an equivalent definition in both Σ_n^b and Π_n^b . In particular, the class Δ_0^b stands for all formulas with sharply bounded quantifiers only.

The theory S_2^1 of Buss [Bus86] extends Robinson's arithmetic Q by a set BASIC of simple axioms for the new function symbols (see, e.g., [Kra95, Definition 5.2.1] for the complete list). On top of this, the theory

has the *Polynomial Induction* schema (PIND) for Σ_1^b -formulas: for every $\varphi \in \Sigma_1^b$, the theory contains the axiom

$$\varphi(0) \wedge \forall x(\varphi(\lfloor x/2 \rfloor) \rightarrow \varphi(x)) \rightarrow \forall x\varphi(x). \quad (\text{PIND}_\varphi)$$

When working over S_2^1 , we often invoke instead the schema for *Length Induction*,

$$\varphi(0) \wedge \forall x(\varphi(x) \rightarrow \varphi(x+1)) \rightarrow \forall x\varphi(|x|), \quad (\text{LIND}_\varphi)$$

made available for all Σ_1^b -formulas. This form of induction is provable from (PIND_φ) for $\varphi \in \Sigma_1^b$ [Kra95, Lemma 5.2.5].

Unlike $\mathcal{L}_{PV}$, the language $\mathcal{L}_{BA}$ of bounded arithmetic does not contain a function symbol for every function in $\mathbf{FP}$. However, every such $f \in \mathbf{FP}$ is Σ_1^b -definable in S_2^1 , meaning that there exists a Σ_1^b formula whose interpretation over the standard model $\mathbb{N}$ defines f and such that S_2^1 proves the totality of this definition. Thus, in the rest of the paper we choose to use the theory $S_2^1(\mathcal{L}_{PV})$, which is the theory S_2^1 in the language of bounded arithmetic extended by all PV function symbols, meaning that we have a fresh symbol for each function in $\mathbf{FP}$, and induction is now available for all $\Sigma_1^b(\text{PV})$ formulas. The theory $S_2^1(\mathcal{L}_{PV})$ is fully conservative over S_2^1 . In what follows we abuse notation and denote this simply as S_2^1 .

The final key fact for us is that all $\forall\Sigma_1^b$ formulas provable in S_2^1 are already provable in PV_1 . That is, the theory S_2^1 is $\forall\Sigma_1^b$ -conservative over PV_1 [Bus86]. We use this in some of the formalizations, where we carry out arguments in S_2^1 but later appeal to its proof in PV_1 . For a proof of this fact, see, for example Krajíček's textbook [Kra95, Thm. 5.3.4 and Cor. 7.2.4].

2.3.2 Exact counting in PV_1 and S_2^1

When working in theories of bounded arithmetic, counting the size of different sets requires a great deal of care. We use the wide-spread *Log-notation* $n \in \text{Log}$ as a short-hand for the formula $\exists x(|x| = n)$. A set X is called a *bounded definable set with parameter y* if there exists an arithmetic formula $\varphi(x, y)$ and some term $t(y)$ such that $X = \{x < t(y) \mid \varphi(x, y)\}$. We also adopt the standard set-theoretic notation denoting the interval $[0, a)$ directly by a . Then, a Boolean circuit $C : 2^k \rightarrow 2$ naturally defines a bounded definable set $X_C = \{x < 2^k \mid C(x) = 1\}$ with parameter C from which $k \in \text{Log}$ can be extracted, and there exists a PV function Count which counts Log-sized initial segments of circuit-definable sets; i.e., in the standard model, $\text{Count}(C, a)$ is the cardinality of the set $X_C \cap [0, |a|)$.

Importantly, if we know that $2^k \in \text{Log}$, then we are able to *exactly count* the size of X_C when working in PV_1 . This proves crucial when carrying out different combinatorial arguments. As an example, arguments involving the pigeonhole principle or different averaging arguments are all possible in PV_1 thanks to exact counting, as long as the sizes of the sets in question are in Log .

2.3.3 Buss's witnessing theorem

Witnessing theorems are a central tool in the field of bounded arithmetic connecting proofs and computation. Roughly speaking, they show that existential quantifiers can be turned into explicit functions computing witnesses for these quantifiers, such that the computational complexity of such a function depends tightly on the logical strength of the theory proving the statement. We mainly rely on Buss's witnessing theorem for S_2^1 (and hence also for PV_1), capable of witnessing one level of existential quantifiers via polynomial-time functions.

Theorem 2.2 (Buss's witnessing theorem [Bus86]). *Let $\varphi(x, y)$ be a Σ_1^b formula. If $S_2^1 \vdash \forall x \exists y \varphi(x, y)$, then there exists a PV function w such that $S_2^1 \vdash \forall x \varphi(x, w(x))$.*

Buss's original argument uses proof theory. We refer to Hájek and Pudlák [HP93, Theorem 4.32] for a model-theoretic proof.

2.3.4 Cook's propositional translation

Following Krajíček [Kra19, §8.6], we say that a theory of arithmetic T *corresponds* to a propositional proof system S if (i) T can prove the soundness of S and (ii) every universal consequence $\forall x\varphi(x)$ of T , where φ is quantifier-free, admits polynomial-size proofs in S when suitably grounded into a sequence of propositional formulas. (Pudlák alternatively says that S is a *weak system* of the theory T [Pud20].)

We are interested in the correspondence between PV_1 and Extended Frege (EF). In this case, the process used to turn first-order formulas into propositional ones is known as (Cook's) *propositional translation*, introduced in his seminal paper on PV [Coo75]. Given a quantifier-free formula $\varphi(x)$, Cook's translation is a polynomial-time construction sending φ to a sequence of polynomial-size propositional formulas $\{\llbracket \varphi \rrbracket_n\}_{n \in \mathbb{N}}$ such that for every $n \in \mathbb{N}$, the formula $\llbracket \varphi \rrbracket_n \in \text{TAUT}$ if and only if $\mathbb{N} \models \varphi(n)$. See [Kra19, §12.3] or [Bus97, §6.1] for a complete definition of the construction. Cook then observed that, under this translation, PV_1 and EF do indeed correspond to each other.

Theorem 2.3 (Cook's correspondence theorem [Coo75]). *The theory PV_1 and the proof system Extended Frege correspond to each other. That is,*

- (i) PV_1 proves the soundness of EF;
- (ii) if $\varphi(x)$ is a quantifier-free formula in the language $\mathcal{L}_{BA}(PV)$ and $PV_1 \vdash \forall x\varphi(x)$, then there exists a polynomial-time computable function f such that for every $n \in \mathbb{N}$, it holds that $f(1^n) : \text{EF} \vdash \llbracket \varphi \rrbracket_n$.

A proof of the theorem can be found in Krajíček's textbook [Kra19, Theorem 12.4.2].

2.3.5 Strong proof systems of theories of arithmetic

Let T be a consistent theory extending Robinson Arithmetic Q by a set of axioms that is decidable in polynomial time (i.e., T extends Q and is *polynomial-time axiomatizable*). We denote by P_T the propositional proof system in which a proof of the propositional tautology φ is given by a T -proof of the first-order sentence $\text{Taut}(\ulcorner \varphi \urcorner)$ which states that the propositional formula φ is made true by every truth-assignment for its variables; here $\ulcorner \varphi \urcorner$ stands for the code of φ in a standard arithmetization of propositional formulas. Following Pudlák [Pud20], we call P_T the *strong proof system* of T .

We will use two key facts of these proof systems. First, if T is a theory for which P_T is defined and Q is a propositional proof system that corresponds to T in the sense of Section 2.3.4, then $P_T \geq_p Q$ [Pud20, §4.2, Fact 2]. Second, by Gödel's second incompleteness theorem, T does not prove the soundness of P_T [Pud20, §4.2, Fact 3].

2.4 The REF formulas

The main character in this paper is the so-called *REF formula*. Given a propositional CNF formula φ and a size parameter $s \in \mathbb{N}$, the formula $\text{REF}_s(\varphi)$ states that φ has a Resolution refutation consisting of at most s clauses.

It is important to choose an encoding that is simultaneously natural from a modeling point of view while not making the formulas artificially hard to refute. At a basic level, the main property that such an encoding should satisfy is that for a concrete φ and s , the formula $\text{REF}_s(\varphi)$ should be satisfiable if and only if there is a Resolution refutation of φ in at most s clauses. It also seems natural to require that a Resolution refutation should be readable in polynomial time from a satisfying assignment to REF.

While different encodings have appeared in the literature, they tend to agree on a few basic ideas. The formula $\text{REF}_s(\varphi)$ consists of s so-called *blocks* of variables, each representing a clause in the purported Resolution refutation. Each block has variables to represent the literals that appear in this block, how it was obtained (resolved or weakened from an axiom), and it contains *pointer variables* to indicate from which blocks it was derived.

The unary encoding of Pudlák. Pudlák [Pud03] uses the seemingly most standard encoding, which we refer to as the *unary encoding* for REF. He used it to prove that the canonical pair of Resolution is symmetric. This encoding employs pointers in unary, meaning that for every block $B \in [s]$, there are up to s additional variables to point at the blocks from which B was derived.

The relativized unary encoding of Atserias and Müller. Atserias and Müller [AM20] start by studying Pudlák’s encoding. They proved suitable so-called *index-width* lower bounds for it in Resolution, but they were unable to prove a *size* lower bound for it. They then introduced a *relativized* version, in which each block can be possibly *enabled* or *disabled*. If it is disabled, then the block is not used towards the refutation, and its associated clauses are immediately satisfied. These additional *enabling variables* made it possible to prove the size lower bound from the index-width lower bound for the unrelativized encoding. We refer to this second encoding as the *relativized unary encoding*.

We note, however, that the change of encoding is not the source for hardness. Garlík [Gar19] proved that even when using the original encoding of Pudlák, the formulas are hard for Resolution whenever the underlying CNF is unsatisfiable.

The binary encoding of de Rezende et al. In their alternative proof of the lower bound on REF formulas, de Rezende et al. [dRGN+21] introduce an encoding of REF where pointers are encoded in binary. Informally, for every block $B \in [s]$, there are $O(\log s)$ variables used to encode the value $B' \in [s]$ of the block(s) from which B was derived. We refer to this as the *binary encoding*. While this encoding also includes the enabling variables of the relativized encoding, these are inessential, since one can always assign the pointers in a dummy fashion to effectively disable a block.

We contend that the unary relativized encoding is both the most natural as well as the most versatile. We see three reasons for this:

1. thanks to the enabling variables, one can naturally turn a Resolution refutation of $t < s$ clauses into a satisfying assignment to $\text{REF}_s(\varphi)$ simply by disabling $s - t$ blocks that are not needed, while in the relativized encoding one needs to fill in the remaining $s - t$ clauses with some dummy content;
2. the enabling variables make the random restriction argument leading to the size lower bound much simpler to prove, and Garlík has shown that the hardness does not come from this change in syntax;
3. when using a unary encoding rather than the binary one of de Rezende et al., one can easily restrict some pointers to get an instance of $\text{REF}_t(\varphi)$ for every $t < s$, while in the binary encoding, after disabling a block, the binary pointers might still be able to point to it, making the formulas more delicate to handle after applying a restriction.

We remark that the choice between unary and binary encodings is ultimately inessential, and all the results in this paper can be reproven for the binary encoding. We choose the unary encoding mainly for reason (3) above, which simplifies the write-up.

We now define the formula in detail.

The variables of $\text{REF}_s(\varphi)$. Here, we assume φ is a CNF formula over n variables $x_1, \dots, x_n$ and m clauses and define the following variables, where $\text{Lit}_n := \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}$.

a-lit_ℓ^A	: literal $\ell \in \text{Lit}_n$ is present in the clause $A \in [m]$ of φ ;
enable^B	: block $B \in [s]$ is enabled;
derived^B	: block $B \in [s]$ is obtained by a Resolution step;
weak_A^B	: block $B \in [s]$ is obtained by weakening from clause $A \in [m]$ of φ ;
lit_ℓ^B	: literal $\ell \in \text{Lit}_n$ is present in the block $B \in [s]$;
$\text{res}_{x_i}^B$	: block $B \in [s]$ is obtained by resolving over the variable x_i ;
$\text{lpoint}_{B'}^B$	: block $B \in [s]$ is resolved on the left from block $B' \in [s]$, $B' < B$;
$\text{rpoint}_{B'}^B$	: block $B \in [s]$ is resolved on the right from block $B' \in [s]$, $B' < B$.

Building on these variables, the $\text{REF}_s(\varphi)$ formula is defined as follows. We write the clauses as implications for the sake of readability.

Definition 2.4 (The REF formulas). Let $n, m, s \in \mathbb{N}$, and let $\text{Lit}_n := \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}$ denote the of possible literals over n variables. The $\text{REF}_s(\varphi)$ formula is built from the variables defined above, together with the conjunction of the following clauses:

$$\left(\text{enable}^B \wedge \text{res}_{x_i}^B \wedge \text{lpoint}_{B'}^B \wedge \text{lit}_\ell^{B'} \right) \rightarrow \text{lit}_\ell^B \quad \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{x_i\}, \quad (\text{REF-1})$$

$$\left(\text{enable}^B \wedge \text{res}_{x_i}^B \wedge \text{rpoint}_{B'}^B \wedge \text{lit}_\ell^{B'} \right) \rightarrow \text{lit}_\ell^B \quad \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{\neg x_i\}, \quad (\text{REF-2})$$

$$\left(\text{enable}^B \wedge \text{weak}_A^B \wedge \text{a-lit}_\ell^A \right) \rightarrow \text{lit}_\ell^B \quad \text{for } B \in [s], A \in [m], \ell \in \text{Lit}_n, \quad (\text{REF-3})$$

$$\left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{i \in [n]} \text{res}_{x_i}^B \quad \text{for } B \in [s], \quad (\text{REF-4})$$

$$\left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{lpoint}_{B'}^B \quad \text{for } B \in [s], \quad (\text{REF-5})$$

$$\left(\text{enable}^B \wedge \text{derived}^B \right) \rightarrow \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{rpoint}_{B'}^B \quad \text{for } B \in [s], \quad (\text{REF-6})$$

$$\left(\text{enable}^B \wedge \neg \text{derived}^B \right) \rightarrow \bigvee_{A \in [m]} \text{weak}_A^B \quad \text{for } B \in [s], \quad (\text{REF-7})$$

$$\left(\text{enable}^B \wedge \text{lpoint}_{B'}^B \right) \rightarrow \text{enable}^{B'} \quad \text{for } B, B' \in [s], B' < B \quad (\text{REF-8})$$

$$\left(\text{enable}^B \wedge \text{rpoint}_{B'}^B \right) \rightarrow \text{enable}^{B'} \quad \text{for } B, B' \in [s], B' < B \quad (\text{REF-9})$$

$$\neg \text{lit}_\ell^s \quad \text{for } \ell \in \text{Lit}_n, \quad (\text{REF-10})$$

$$\text{enable}^s. \quad (\text{REF-11})$$

Remark 2.5. Our encoding of REF has fewer axioms than that of [AM20]. For example, we do not require that if a block B is resolved on the left by variable x from block B' , then B' should contain x , or we do not require that for every resolution step there is a unique resolved variable. We remark that soundness still holds and REF is satisfiable if and only if there exists a refutation of length at most s , which can easily be read from the satisfying assignment to the REF formula. We also remark that the lack of these axioms does not affect the extraction algorithm or the lower bound in any way: while removing axioms could in principle make the lower bound easier to prove, the algorithm works just as well if we added the missing axioms, and our lower bound proof still goes through with the additional axioms. This more succinct encoding, however, makes it easier to formalize the upper bound construction in Resolution. ◀

Remark 2.6 (Number of variables). The formula $\text{REF}_s(\varphi)$ is defined over $N = \Theta(s^2 + sm + sn + mn)$ variables and $M = \Theta(s^2 n^2 + smn)$ clauses. For the case when the a-lit variables are restricted to encode a k -CNF formula over n variables and $s = n^c$ for some constant $c \geq 1$, we have $m = O(n^k)$ and $N = O(n^{\max\{2c, c+k\}})$. ◀

Blocks and block-width. If a variable is part of a block B_i , we say that it *mentions* B_i . An important measure for us will be the *block-width* of a given clause C over the variables of $\text{REF}_s(\varphi)$. This is defined as the number of different blocks mentioned by the variables of the literals in C , not counting the root block $B_\perp$. We denote this measure by $\text{bw}(C)$, and generalize it to refutations by taking $\text{bw}(\pi)$ to be the maximum block-width over all the clauses in π .

2.4.1 The REF formulas for other proof systems

We will also be interested in the REF formulas for proof systems other than Resolution. In general, for a Cook–Reckhow system Q , we denote by $\text{REF}^Q(\varphi, \pi)$ the formula stating that π is a correct Q -refutation of an unsatisfiable φ . For convenience, in this context we always consider all proof systems as refutational systems. The formula $\text{REF}^Q(\varphi, \pi)$ is simply the propositional formula that verifies that π is accepted as a Q -refutation of φ by the Boolean circuit that checks Q -refutations. This can be obtained by writing the computation of the circuit as a Boolean formula using the usual Tseitin encoding.

Remark 2.7 (Notation). By default, the REF formula stands for the formula as defined in Definition 2.4 for Resolution refutations. If we want to refer to the REF formula for a different proof system, we explicitly write REF^Q for the system Q in question. ◀

2.4.2 The SAT formula and reflection principles

We also have the SAT(φ, α) formula, encoding that a CNF formula φ is satisfied by an assignment α . The variables we consider are

- α_i : value assigned by α to variable x_i ;
- a-lit_ℓ^A : literal $\ell \in \text{Lit}_n$ is present in clause $A \in [m]$;
- sat_ℓ^A : clause $A \in [m]$ is satisfied because literal $\ell \in \text{Lit}_n$ evaluates to 1 under α .

We use a-lit instead of lit to distinguish between these variables and the lit variables of $\text{REF}(\varphi, s)$.

Definition 2.8 (The SAT formulas). Let $n, m \in \mathbb{N}$, let φ denote the set of variables of the form a-lit_ℓ^A as above and let α denote the set of variables α_i as above. The formula $\text{SAT}(\varphi, \alpha)$ is the CNF formula over the variables in φ, α and additionally all the variables sat_ℓ^A above consisting of the conjunction of the following clauses,

$$\neg \text{sat}_\ell^A \vee \text{a-lit}_\ell^A \quad \text{for all } A \in [m] \text{ and } \ell \in \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}, \quad (\text{SAT-1})$$

$$\neg \text{sat}_{x_i}^A \vee \alpha_i \quad \text{for all } A \in [m] \text{ and } i \in [n], \quad (\text{SAT-2})$$

$$\neg \text{sat}_{\neg x_i}^A \vee \neg \alpha_i \quad \text{for all } A \in [m] \text{ and } i \in [n], \quad (\text{SAT-3})$$

$$\bigvee_{i \in [n]} \text{sat}_{x_i}^A \vee \text{sat}_{\neg x_i}^A \quad \text{for all } A \in [m]. \quad (\text{SAT-4})$$

One can similarly write a SAT formula for evaluating DNF formulas in the obvious ways. It is always clear from context which version of SAT we are using, so we use the same notation for both.

Proposition 2.9. *For every CNF formula φ , the following statements hold:*

- (i) if φ has a Resolution refutation of length s , then $\text{SAT}_{\upharpoonright\varphi}$ has a Resolution refutation of length $O(s)$;
- (ii) if $\text{SAT}_{\upharpoonright\varphi}$ has a Resolution refutation of length s , then φ has a Resolution refutation of length $O(s)$.

Proof. Assume φ has n variables $x_1, \dots, x_n$ and m clauses $C_1, \dots, C_m$. We first show how to go from a refutation of $\text{SAT}_{\upharpoonright\varphi}$ to a refutation of φ . After fixing φ , the only variables left in SAT are of type sat_ℓ^A or α_i . Since Resolution is closed under literal substitutions, it suffices to define a substitution σ that replaces all the original variables by literals of φ , and argue that all the axioms of $\text{SAT}_{\upharpoonright\varphi, \sigma}$ follow from clauses of φ . For every $i \in [n]$, every $\ell \in \{x_1, \dots, x_n, \neg x_1, \dots, \neg x_n\}$, and every $A \in [m]$, the substitution σ maps

$$\sigma(\alpha_i) := x_i \quad \text{and} \quad \sigma(\text{sat}_\ell^A) := \begin{cases} \ell & \text{if } \ell \in C_A \\ 0 & \text{if } \ell \notin C_A. \end{cases} \quad (2.1)$$

Let us inspect the axioms of $\text{SAT}_{\upharpoonright\varphi, \sigma}$. For an axiom $\neg \text{sat}_\ell^A \vee \text{a-lit}_\ell^A$ of type (SAT-1), if the restriction of this axiom is present in $\text{SAT}_{\upharpoonright\varphi}$ it is because $\ell \notin C_A$, or else a-lit_ℓ^A would be satisfied. Then, $\sigma(\text{sat}_\ell^A) = 0$ and the substitution is satisfied. If the axiom is of type (SAT-2), then either $x_i \notin C_A$, in which case $\sigma(\text{sat}_{x_i}^A) = 0$ and $\neg \text{sat}_{x_i}^A \vee \alpha_i$ is satisfied, or $\sigma(\text{sat}_{x_i}^A) = x_i$ and $\sigma(\alpha_i) = x_i$, which gives the trivial clause $\neg x_i \vee x_i$. The case for (SAT-3) is analogous. Finally, for a clause of type (SAT-4), it is easy to see that the substitution σ maps the clause precisely to the clause C_A itself, which is a clause of φ .

To go from a refutation of φ to a refutation of $\text{SAT}_{\upharpoonright\varphi}$, we do the following. First, rename all the variables α_i by x_i . Then, for every $A \in [m]$, note that we can resolve the corresponding clause of type (SAT-4) with the unit clauses $\neg \text{sat}_\ell^A$ of type (SAT-1) to obtain $\bigvee_{\ell \in C_A} \text{sat}_\ell^A$. Now, for each sat_ℓ^A in this clause, cut with the corresponding clause of type (SAT-2) or (SAT-3) to obtain $\bigvee_{\ell \in C_A} \ell$, which is just C_A . In this way we have derived every axiom C_A of φ , and we can now proceed with the refutation of φ in the natural way. $\square$

Finally, we define the reflection principle for any proof system.

Definition 2.10 (The REFL formulas). Let Q be a Cook–Reckhow propositional proof system and let $n, m, s \in \mathbb{N}$. We define $\text{REFL}_{n,m,s}^Q := \neg \text{SAT}(\varphi, \alpha) \vee \neg \text{REF}^Q(\varphi, \pi)$ where the SAT instance is for CNF formulas with n variables and m clauses and the REF^Q instance is for Q -refutations of such formulas of size s , and refer to the sequence of tautologies $\text{REFL}^Q := \{\text{REFL}_{n,m,s}^Q\}_{n,m,s \in \mathbb{N}}$ as the *reflection principle for Q* .

A useful property of EF is the fact that any propositional system S , however strong, can always be seen as a Frege-like system due to the fact that $\text{EF} + \text{REFL}^S \geq_p S$ [Kra19, Theorem 8.4.3].

2.4.3 Reflection principles for first-order theories

The reflection principles above are the propositional analogues of the well-studied first-order reflection principles for theories of arithmetic. For a strong enough recursively axiomatizable theory of arithmetic T and a class Φ of sentences, the schema $\Phi\text{-Ref}_T$ stands for the collection of all formulas of the form

$$\exists \pi \text{Prf}_T(\ulcorner \varphi \urcorner, \pi) \rightarrow \varphi \quad (\text{Ref}_{T,\varphi})$$

for every $\varphi \in \Phi$. Here, the notation $\ulcorner \varphi \urcorner$ stands for the encoding of φ in a fixed suitable arithmetization of syntax, and Prf_T stands for the provability statement in this arithmetization.

3 The Proof Analysis Problem: definitions and basic facts

For a CNF formula $\varphi(x_1, \dots, x_n)$, we denote by $\text{REF}_s(\varphi)$ the propositional formula claiming that there exists a Resolution refutation of φ in size s . Different encodings of this formula have been considered in the

literature. For our purposes, REF consists of s of *blocks* of variables, each of them describing a clauses in a purported Resolution refutation of φ of size s . (See Section 2.4 for a full rendering of the variables and clauses involved in $\text{REF}_s(\varphi)$.)

We are interested in the following decision problem.

Definition 3.1 (The Proof Analysis Problem, PAP_Q). Let Q be a propositional proof system. We define the *Proof Analysis Problem for Q* to be the language

$$\text{PAP}_Q := \{(\varphi, \pi, 1^s) \mid \varphi \in \text{SAT} \text{ and } \pi : Q \vdash \neg \text{REF}_s(\varphi)\}.$$

We denote by $\text{PAP}_Q[s(n)]$ the problem where the size parameter s is restricted to be at least $s(n)$ and n denotes the number of variables of φ .

The problem asks, given the proof of a Resolution lower bound in a fixed proof system Q , to decide whether the underlying formula is satisfiable or not. Note that whenever φ is satisfiable there is no Resolution refutation and thus any lower bound holds, so the problem is well-defined.

Analogous to the notion of whether a proof system is automatable, PAP naturally induces a notion of whether, for a given proof system, its Resolution lower bounds are “analyzable”.

Definition 3.2 (Analyzability). We say that a propositional proof system Q is *analyzable* if there exists some constant $c > 0$ such that $\text{PAP}_Q[n^c] \in \mathbf{P}$.

Remark 3.3. It might seem more intuitive to define a proof system Q to be analyzable if $\text{PAP}_Q \in \mathbf{P}$, without restrictions on the size parameter. Note, however, that for most reasonable proof systems, the language PAP_Q taken as a whole contains some degenerate instances that make the problem trivially $\mathbf{NP}$ -hard. For example, if the size parameter is set to $s = 1$, then certainly proving a Resolution lower bound against φ is easy already for Resolution itself, and we can map a CNF formula φ to the PAP_Q -instance $(\varphi, \pi, 1)$ for some easy to construct Q -proof π that checks there is no Resolution refutation of φ in one clause. ◀

It is easy to see that for every Cook–Reckhow system Q , the problem PAP_Q is in $\mathbf{NP}$. Similarly, we note that unlike automatability, analyzability is naturally downwards-closed under p -simulations. Namely, if S is p -simulated by Q and Q is analyzable, so is S ; this is not the case with automatability, where a search algorithm for Q may not be used to search for proofs in a weaker S .

The following is a corollary of the results of Atserias and Müller [AM20]. Here, by $\mathbf{P}$ -uniform we mean the standard notion of uniformity by which there is a polynomial-time descriptor Turing machine that on input 1^ℓ outputs the circuit solving the problem for inputs of size ℓ (see, e.g., [AB09, Definition 6.12; All25]).

Proposition 3.4. *It holds that $\text{PAP}_{\text{Res}}[n^2]$ is in $\mathbf{P}$ -uniform $\mathbf{AC}^0$. That is, Resolution is analyzable.*

Proof. Let us first describe the general polynomial-time algorithm that puts $\text{PAP}_{\text{Res}}[n^2]$ in $\mathbf{P}$, and we later elaborate on how this can be computed in $\mathbf{P}$ -uniform $\mathbf{AC}^0$. Indeed, by the Resolution lower bound on REF formulas (Theorem 1.6), there exists $\varepsilon > 0$ such that for every $s \in \mathbb{N}$, if a formula φ over n variables is unsatisfiable, then a correct Resolution refutation π of $\text{REF}_s(\varphi)$ must have size $|\pi| > 2^{\varepsilon \cdot s/n}$. Given an input $(\varphi, \pi, 1^s)$ to $\text{PAP}_{\text{Res}}[n^2]$, to decide if the instance belongs in the language, it suffices to check (i) that π is a correct Resolution refutation of $\text{REF}_s(\varphi)$ and (ii) that $|\pi|$ is smaller than the lower bound $2^{\varepsilon \cdot s/n}$. If (i) fails, we immediately reject, and otherwise, if (ii) fails, the input size is large enough to brute-force SAT in polynomial time. Here we use the fact that $s \geq n^2$, hence $|\pi| \geq 2^{\varepsilon \cdot s/n} \geq 2^{\varepsilon n}$, and thus the input size is large enough.

Let us now argue that this entire computation is possible within $\mathbf{P}$ -uniform $\mathbf{AC}^0$. For the sake of precision, let us fix the following natural binary encoding for PAP_{Res} . An input $(\varphi, \pi, 1^s)$ will be of the form $(1^n, 1^m, C_1, \dots, C_m, 1^t, \pi, 1^s)$. Here, the first part of the tuple corresponds to the encoding of φ , a

CNF formula over n variables and m clauses $C_1, \dots, C_m$, and we assume that these clauses are initially represented as strings of length $2n$ with indicators for every possible literal. The Resolution refutation of $\text{REF}_s(\varphi)$ is encoded by 1^t and π , where π is an assignment to the $N := N(n, m, t, s) = \text{poly}(n, m, t, s)$ variables of $\text{REF}_t(\text{REF}_s(\varphi))$, as per Section 2.4, and the number of variables N can be easily computed in polynomial time. For the purpose of unique decoding, we assume that the tuple $(1^n, 1^m, C_1, \dots, C_m, 1^t, \pi, 1^s)$ is encoded by bit-doubling: each bit is duplicated and 01 is used as separators. We assume that there are no separators between the clauses $C_1, \dots, C_m$, so a correct input contains only five separators.

Now, when dealing with binary strings of even length ℓ , there are at most $O(\ell^4)$ possible ways of interpreting such strings as a tuple of the form $(1^n, 1^m, C_1, \dots, C_m, 1^t, \pi, 1^s)$. This is because we can choose values for n, m, t and s in the interval $[\ell/2 - 5]$, where $\ell/2 - 5$ comes from the fact that we duplicated every bit and introduced 10 bits for the five separators between $1^n, 1^m, C_1, \dots, C_m, 1^t, \pi$, and 1^s , not counting separators between C_1 and C_m . One can then check that this choice of n, m, t and s conforms to the desired pattern: the segment for the clauses C_1 to C_m has length exactly $2nm$, and the segment for π has length exactly $N = N(n, m, t, s)$, as per Remark 2.6. That is, it must hold that $\ell = 2(n + m + 2nm + t + N(n, m, t, s) + s) + 10$ and $s \geq n^2$. There are at most $O(\ell^4)$ such choices for $(n, m, t, s) \in [\ell/2 - 5]^4$, hence the upper bound. Furthermore, it is easy to see that, due to the bit-doubling, every string can only encode correctly one input of the form $(1^n, 1^m, C_1, \dots, C_m, 1^t, \pi, 1^s)$, so the decoding is unique.

The P-uniform descriptor machine for inputs of even length ℓ now works as follows. On input 1^ℓ , for ℓ even, it tries all possible $O(\ell^4)$ ways of separating the lengths, and for each interpretation (n, m, t, s) of the lengths it constructs a different constant-depth Boolean circuit $D_{n,m,t,s}$, as follows.

- (a) If the interpretations of the lengths is inconsistent, in the sense that the string cannot correspond to something of the form $(1^n, 1^m, C_1, \dots, C_m, 1^t, \pi, 1^s)$, then it outputs the constant circuit 0.
- (b) If the interpretation of the lengths is valid and $|\pi| < 2^{\varepsilon \cdot s/n}$, then it simply constructs the circuit that checks that π is a correct Resolution refutation of $\text{REF}_s(\varphi)$ in at most t clauses; that is, it outputs the formula $\text{REF}_t(\text{REF}_s(\varphi))$, which itself depends on the variables encoding φ . Since REF formulas are in CNF, nesting these together with φ will result in a total depth of 5 (see Section 7.2 for a more detailed treatment of how the depth increases when nesting the REF formulas).
- (c) If the interpretation of the lengths is valid and $|\pi| \geq 2^{\varepsilon \cdot s/n}$, then the descriptor outputs the conjunction of two circuits: one is the same as before, checking the correctness of π as a refutation of $\text{REF}_s(\varphi)$ in at most t clauses, and the other is the trivial circuit of size $\text{poly}(n, m) \cdot 2^n$ that brute-forces the satisfiability of φ . More formally, this is a big disjunction of fan-in 2^n , where each wire goes to the formula $\text{SAT}(\varphi, \alpha)$ from Definition 2.8 for different hard-wired values of $\alpha \in \{0, 1\}^n$. Since $\text{SAT}(\varphi, \alpha)$ is a CNF formula, this brute-forcing circuit has depth 3, and combined with the circuit checking the correctness of π , the entire circuit has depth 5 in this case.

For each interpretation of the lengths there is also a circuit $\text{Correct}_{n,m,t,s}$ that verifies that the input correctly encodes a PAP_{Res} instance of the right size. This amounts to checking that the separators are in the right place and the double-bit encoding is correctly implemented, which can all be verified in depth 3.

Finally, the descriptor machine outputs the circuit

$$R_\ell := \bigvee_{(n,m,t,s) \in [\ell/2-5]^4} D_{n,m,t,s} \wedge \text{Correct}_{n,m,t,s} \quad (3.1)$$

consisting of the disjunction of all the circuits above for every interpretation of the lengths.

The final circuit R_ℓ correctly computes $\text{PAP}_{\text{Res}}[n^2]$ on inputs of even length ℓ , has depth 6 and polynomial size. Indeed, the constructions (a) and (b) above both have polynomial size, and whenever we construct the exponential-size circuit in (c), it is with respect to a segment of the string that has itself size exponential

in n . Finally, the descriptor machine runs in polynomial time given only the length ℓ of the input string x , so we can conclude that $\text{PAP}_{\text{Res}}[n^2]$ is in $\mathbf{P}$ -uniform $\mathbf{AC}^0$. $\square$

The fact that PAP_{Res} is so easy makes it natural to ask whether the same is true for the *search version* of the problem.

Definition 3.5 (Search version of PAP). For a propositional proof system Q , we denote by FPAP_Q the *search version of the Proof Analysis Problem for Q* , defined as follows.

FPAP_Q (search version of PAP_Q)	
Input	A CNF formula φ , a size parameter s in unary and a proof π such that $\pi : Q \vdash \neg \text{REF}_s(\varphi)$.
Output	Either a satisfying assignment for φ , if $\varphi \in \text{SAT}$, or 0 otherwise.

Similarly to $\text{PAP}_Q[s(n)]$, we define $\text{FPAP}_Q[s(n)]$ to be the search problem where the size parameter is at least $s(n)$.

If we impose a polynomial upper bound on the size of π , then by the lower bound on REF formulas, there is always a satisfying assignment for φ , and the problem $\text{FPAP}_{\text{Res}}[n^c]$ for any $c > 0$ is in $\mathbf{TFNP}$. The fact that $\text{PAP}_{\text{Res}}[n^2] \in \mathbf{P}$, does not, however, directly imply that $\text{FPAP}_{\text{Res}}[n^c] \in \mathbf{FP}$ for any $c > 0$. Namely, it is not clear that given a polynomial-size proof π of $\text{REF}_s(\varphi)$ one can extract a satisfying assignment of φ , even if one can conclude that φ is satisfiable. We show in Section 4 that FPAP_{Res} is in $\mathbf{FP}$ —although the algorithm is not quite as straightforward as the one for the decision problem.

We see PAP and the analyzability of a proof system as closely related to automatability. The following proposition captures this idea and underlines the relevance on PAP in showing hardness of automatability.

Proposition 3.6. *Let $Q \geq \text{Res}$. If Q is both analyzable and automatable, then $\mathbf{P} = \mathbf{NP}$.*

Proof. If Q is analyzable and automatable, this means that $\text{PAP}_Q[n^c] \in \mathbf{P}$ for some constant $c > 0$, and that there is an automating algorithm A for Q . We call the polynomial-time algorithm for $\text{PAP}_Q[n^c]$ an *analyzer* and observe that these two combined can solve 3SAT in polynomial time as follows. Given a 3-CNF formula φ , construct the formula $\text{REF}_{n^c}(\varphi)$, stating that φ does not have Resolution refutations of size n^c . Since $Q \geq \text{Res}$, by the upper bound construction [Pud03, Theorem 4.1; AM20, Lemma 11], whenever φ is satisfiable, there will be size- $n^{O(1)}$ refutations in Resolution, and hence also in Q , and the automating algorithm A will succeed in finding some refutation in polynomial time. Feed this refutation to the Q -analyzer to decide whether $\varphi \in \text{SAT}$. If the automating algorithm failed to output a polynomial-size proof, then we would already know that $\varphi \notin \text{SAT}$. $\square$

The previous proposition can be seen as an abstract way of stating the $\mathbf{NP}$ -hardness of automating Resolution too. Since $\text{PAP}_{\text{Res}}[n^2] \in \mathbf{P}$, that means that Resolution cannot be automatable unless $\mathbf{P} = \mathbf{NP}$. In Section 5 we study the possibility of analyzing algorithms for strong proof systems actually leading to the hardness of their automatability—and establish that this is highly unlikely.

4 The extraction algorithm

This section proves that the search version of the Proof Analysis Problem for Resolution is in $\mathbf{FP}$. Our algorithm (in fact, two algorithms) arise from closely observing the lower bound on the REF formulas and attempting to make it fully constructive, in a style amenable to formalization in weak theories of arithmetic like PV_1 (in the style of Cook and Pitassi [CP90]).

Recall that the lower bound can be presented in two steps: first, a random restriction argument takes a small refutation and produces a low block-width refutation of a restricted formula, followed by a block-width lower bound for this restricted formula, which overall bounds the size of the original refutation.

Our algorithm works analogously. On input a refutation π of $\text{REF}_s(\varphi)$, it first finds a restriction ρ such that $\pi_{\upharpoonright \rho}$ is a refutation of a restricted version of $\text{REF}_s(\varphi)$ and has low block-width. Then, we have a second algorithm that, inspired by the proof of the block-width lower bound, analyses this low block-width refutation and extracts a satisfying assignment.

We present the algorithm in two steps. First, two alternatives to perform block-width reduction are described in Section 4.1. These correspond, respectively, to a random restriction and a deterministic restriction argument. In Section 4.2 we explain how to design the algorithm that analyzes low block-width refutations, which is essentially the Prover-Delayer strategy behind the block-width lower bound for the REF formulas. Putting them together yields the desired procedure.

4.1 The block-width reduction algorithm

Recall that we assume that in our definition of the REF formula there is a variable enable^i for every block $i \in [s]$ that allows us to *disable* that block (see Definition 2.4). For succinctness, we often denote enable^i simply by e_i and refer to it as an *enabling variable*.

Let us first define a kind of restriction that will come up a lot in our arguments.

Definition 4.1 (Disabling restrictions). We say that a restriction $\rho \in \{0, 1, *\}^N$ to the N variables of $\text{REF}_s(\varphi)$ is *d-disabling* if it satisfies that (i) exactly d blocks are disabled, and the rest are all enabled, (ii) every variable belonging to a disabled block is assigned a value, and (iii) no other variable is assigned.

A key property of disabling assignments is that they can never falsify any axioms of $\text{REF}_s(\varphi)$, all the enabling variables disappear after the restriction and $\text{REF}_s(\varphi)_{\upharpoonright \rho}$ is essentially an instance of $\text{REF}_{s-d}(\varphi)$, except there are some pointer variables pointing to the d disabled blocks that are still hanging.

A first approach to perform block-width reduction is inspired by random restriction arguments, and requires randomness.

Lemma 4.2 (Randomized block-width reduction). *Let $p \in [0, 1)$ and let N be the number of variables of the $\text{REF}_s(\varphi)$ formula for some CNF formula φ over n variables. There exists a randomized algorithm R taking as input 1^N , and outputting an $\lfloor s/2 \rfloor$ -disabling restriction $\rho \in \{0, 1, *\}^N$, such that for every Resolution refutation π of the formula $\text{REF}_s(\varphi)$, the following properties hold:*

- (i) *the restriction ρ does not falsify $\text{REF}_s(\varphi)$;*
- (ii) *with probability at least p , the block-width of $\pi_{\upharpoonright \rho}$ is at most $O(\log |\pi| - \log(1 - p))$;*
- (iii) *the running time of $R(1^N)$ is $O(N)$.*

Proof. The proof follows closely the random restriction argument of de Rezende et al. [dRGN+21, Section 6.3]. For simplicity, let us assume s is even. (Note that, without loss of generality s can be even, since if π is a correct refutation of $\text{REF}_s(\varphi)$ and s is odd, then π can be turned into a refutation of essentially $\text{REF}_{s-1}(\varphi)$ by hitting π with the restriction that disables and fully restricts one block).

Assume the root block corresponds to block $B_{\perp}$, which is not counted towards block-width, and consider the following random restriction: pair all s blocks into $s/2$ pairs, and for each pair, with probability $1/2$, decide which block in the pair is going to be disabled. Now, if a block is disabled, all of its remaining variables are assigned uniformly at random. We denote by ρ the restriction obtained in this way, which the algorithm outputs.

We claim that with probability at least p , the restriction succeeds in lowering the block-width of any Resolution derivation π to $O(\log |\pi| - \log(1 - p))$. Indeed, if ℓ is a literal corresponding to the variable

e_i determining whether a certain block is disabled, then $\Pr_\rho[\ell_{\uparrow\rho} = 1] = 1/2$. For every other literal ℓ in a block B_i ,

$$\Pr_\rho[B_i \text{ is disabled and } \ell_{\uparrow\rho} = 1] = \Pr_\rho[B_i \text{ is disabled}] \cdot \Pr_\rho[\ell_{\uparrow\rho} = 1 \mid B_i \text{ is disabled}] \quad (4.1)$$

$$= \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}. \quad (4.2)$$

Hence, for every literal ℓ not from $B_\perp$, $\Pr_\rho[\ell_{\uparrow\rho} = 1] \geq 1/4$.

Now, if C is a clause of block-width at least w , we have that

$$\Pr_\rho[C_{\uparrow\rho} \neq 1] \leq (3/4)^{w/2}, \quad (4.3)$$

where the $1/2$ in the exponent comes from the fact that if two consecutive blocks are present, meaning that they were paired together and only one of them was enabled, their values depend on each other.

Then, if π was indeed a Resolution derivation of $\text{REF}_s(\varphi)$, by a union bound,

$$\Pr_\rho[\pi_{\uparrow\rho} \text{ has a clause of block-width at most } w] \leq \text{length}(\pi) \cdot (3/4)^{w/2} \leq |\pi| \cdot (3/4)^{w/2}, \quad (4.4)$$

which is the failure probability for property (ii) in the statement. For success probability at least p , we want to choose w such that $|\pi| \cdot (3/4)^{w/2} \leq 1 - p$. This bound is met by choosing $w \geq 2(\log(|\pi|/(1-p))/(\log 4/3))$, meaning that with probability p , the restriction ρ will satisfy all clauses of at least this width.

It suffices to argue that properties (i) and (iii) are also satisfied. Indeed, by the way we designed the restriction, after applying ρ there are no disabling variables left and all variables in the disabled blocks have been restricted, to this is exactly $s/2$ -disabling.

As for the running time, the algorithm is simply sampling the restriction, which takes time $O(N)$. $\square$

We now move on to a fully deterministic algorithm that takes as input an actual refutation π and outputs a restriction that *always* manages to reduce the block-width.

Lemma 4.3 (Deterministic block-width reduction). *There exists a constant $c > 0$ and a deterministic algorithm taking as input a Resolution refutation π of the formula $\text{REF}_s(\varphi)$ over N variables and outputting a d -disabling restriction $\rho \in \{0, 1, *\}^N$ with $d \leq c/2 \cdot (\sqrt{s \log |\pi|})$ such that*

- (i) *the restriction ρ does not falsify $\text{REF}_s(\varphi)$;*
- (ii) *the block-width of $\pi_{\uparrow\rho}$ is at most $c \cdot (\sqrt{s \log |\pi|})$;*
- (iii) *the algorithm runs in time $\text{poly}(|\pi|, s)$.*

Proof. We employ a greedy strategy to construct the restriction, meaning that we look at all the clauses of high block-width and we iteratively choose to restrict a literal that kills a significant fraction of these clauses.

More formally, let w be a parameter to be optimized later, and given π , let W denote the set of all clauses in π with block-width at least w . Through the following iterative process we will enable and disable some blocks. Whenever we enable a block, we will also mark it as not active by keeping track of a set $\text{ActiveBlocks} \subseteq [s]$, meaning that when choosing greedily the next literal to restrict, inactive blocks are not a valid choice; and whenever we disable a block, we add it to a set $D \subseteq [s]$ to keep track of it.

Algorithm 4.1: Deterministic block-width reduction

Repeat the following procedure iteratively, starting with $\rho := \emptyset$, $\text{ActiveBlocks} := [s]$, and $D := \emptyset$, and stop whenever W is empty:

1. Find the most frequent block $i \in \text{ActiveBlocks}$ among the ones mentioned in the clauses in W .
2. Look at the literal e_i of the variable used to disable block i .
 - (a) If e_i appears positively in at least $1/3$ of all the clauses in W that mention block i , then set $\rho := \rho \cup \{e_i \mapsto 1\}$, $\text{ActiveBlocks} := \text{ActiveBlocks} \setminus \{i\}$, $W := W_{\upharpoonright \rho}$, and go back to step (1).
 - (b) If e_i does not appear positively in at least $1/3$ of the clauses in W mentioning block i , then set $\rho := \rho \cup \{e_i \mapsto 0\}$, $W := W_{\upharpoonright \rho}$, $D := D \cup \{i\}$, and for every other variable x of block i ,
 - i. if x appears in W positively more often than negatively, then set $\rho := \rho \cup \{x \mapsto 1\}$, and $W := W_{\upharpoonright \rho}$;
 - ii. if x appears in W negatively more often than positively, then set $\rho := \rho \cup \{x \mapsto 0\}$, and $W := W_{\upharpoonright \rho}$;
 - iii. repeat for every variable of block i .
 - (c) Once all the variables of block i have been taken care of, go back to step (1).

The procedure terminates once W is either empty or all clauses in W mention only blocks that are no longer in ActiveBlocks . At this point, $|D|$ blocks have been disabled. For the remaining blocks that were not mentioned by any clause in W , enable all of them by setting the corresponding variables $e_i \mapsto 1$. This completes the construction of the restriction ρ , and the algorithm outputs ρ .

The procedure runs for at most s iterations, since each iteration takes care of one of the blocks mentioned by the clauses in the initial W and we never deal with a block twice. Therefore, the algorithm runs in time $\text{poly}(|\pi|, s)$.

As for the correctness of the algorithm, the restriction ρ is d -disabling by construction for $d = |D|$. It is left to argue that for a suitable choice of w , there exists a constant $c > 0$ such that $d \leq c/2(\sqrt{s \log |\pi|})$ and the block-width of $\pi_{\upharpoonright \rho}$ is at most $c \cdot (\sqrt{s \log |\pi|})$.

We want to choose w so that after $\ell \leq s$ iterations, the set W becomes empty. At the first iteration, by an averaging argument, we know that the most frequent block is mentioned in at least a w/s fraction of $|W|$. More generally, at iteration ℓ , block-width might have decreased up to $w - (\ell - 1)$ and up to $\ell - 1$ blocks may have become inactive, so the same averaging argument tells us that the most frequent active block is mentioned in at least a $(w - (\ell - 1))/(s - (\ell - 1))$ fraction of the clauses. Furthermore, observe that if at a given iteration block i is the most frequent active block, we are not promised to kill all the clauses mentioning i , but we are guaranteed to kill at least $1/3$ of them. Indeed, if we enable block i that is because it appeared in at least $1/3$ of all the clauses in W mentioning i ; and otherwise we are guaranteed to restrict at least $1/2$ of the remaining at least $2/3$ fraction of the clauses in W mentioning i , which amounts to at least $1/3$ fraction.

Therefore, if ρ_ℓ is the restriction built after ℓ iterations,

$$|W_{\upharpoonright \rho_\ell}| \leq |W| \cdot \left(1 - \frac{w}{3s}\right) \cdot \left(1 - \frac{w-1}{3(s-1)}\right) \cdots \left(1 - \frac{w-(\ell-1)}{3(s-(\ell-1))}\right) \quad (4.5)$$

$$\leq |W| \cdot \left(1 - \frac{w-\ell}{3s}\right)^\ell \quad (4.6)$$

$$\leq |W| \cdot e^{-\ell \cdot \frac{w-\ell}{3s}}. \quad (4.7)$$

We want to ensure that for some $\ell \leq s$ we achieve $|W_{\upharpoonright \rho_\ell}| < 1$. It suffices to have $|W| \cdot e^{-\ell \cdot \frac{w-\ell}{3s}} < 1$. Taking logarithms on both sides we have

$$\ln |W| < \ell \cdot \frac{w-\ell}{3s}, \quad (4.8)$$

which holds already for $\ell = \lfloor w/2 \rfloor$, assuming $w > \sqrt{12s \ln |\pi|} \geq \sqrt{12s \ln |W|}$.

Now it suffices to choose a constant c such that $w := c \cdot \sqrt{s \log |\pi|} > \sqrt{12s \ln |\pi|}$. In this way we get that after at most $\ell := \lfloor w/2 \rfloor$ iterations, $W_{\upharpoonright \rho_\ell} = \emptyset$ and thus the block-width of $\pi_{\upharpoonright \rho_\ell}$ is also at most $c \cdot \sqrt{s \log |\pi|}$. Furthermore, note that $|D| \leq \ell$, since the algorithm only runs for at most ℓ iterations, meaning that ρ_ℓ is d -disabling for $d \leq \ell = \lfloor w/2 \rfloor \leq (c/2)\sqrt{s \log |\pi|}$, as desired. $\square$

4.2 The block-width analysis algorithm

Using one of the two algorithms above, we can take a refutation π of $\text{REF}_s(\varphi)$ and obtain a new refutation π' of the restricted formula $\text{REF}_s(\varphi)_{\upharpoonright \rho}$ in low block-width. We can now show how to analyze this refutation, inspired by the block-width lower bound, and succeed in finding a satisfying assignment whenever one exists.

We first state the following simple but crucial technical fact used in the proof.

Fact 4.4. *Let φ be CNF formula over n variables. If C is a non-tautological width- n clause over the variables of φ that is not the weakening of any clause of φ , then the unique assignment that falsifies C is a satisfying assignment for φ .*

Now we can present the algorithm and prove its correctness.

Lemma 4.5 (Assignment extraction). *There exists a deterministic algorithm E such that for every $s \in \mathbb{N}$, every π a purported Resolution refutation of $\text{REF}_s(\varphi)$ for a CNF formula $\varphi(x_1, \dots, x_n)$ with m clauses, and every $\rho \in \{0, 1, *\}^N$ a d -disabling restriction to the N variables in $\text{REF}_s(\varphi)$, it holds that $E(\varphi, \rho, s, \pi)$ terminates in time $\text{poly}(|\pi|, s, n, m)$ and provides exactly one of the following outputs:*

- (a) *an incorrect derivation step in π ;*
- (b) *a clause $C \in \pi_{\upharpoonright \rho}$ of block-width at least $1/3 \lfloor (s - d - n)/n \rfloor$;*
- (c) *a satisfying assignment for φ .*

Proof. We traverse the refutation π inspired by the Delayer's strategy in the Prover-Delayer game that yields a block-width lower bound for the restricted REF formulas—and the correctness of the algorithm is essentially the proof of this block-width lower bound.

Before the traversal of π starts, we arrange the $s - d$ blocks enabled by ρ in a layered manner, so that there are n layers, each containing $\lfloor (s - d)/n \rfloor$ blocks (with the remainder blocks left from the flooring operations collected all in the last layer), plus one additional layer on top with a single block corresponding

to the root. We see the root as laying at layer 0, and intuitively blocks in layer i will be obtained by resolving over variable x_{i+1} . Throughout the traversal, we keep a record $\alpha \in \{0, 1, *\}^N$ which we call the *reservation*, in which we “reserve” information needed to continue the traversal. (Intuitively, this record keeps the information that the Delayer has in mind when playing against the Prover.) A reservation is a partial assignment to the variables of $\text{REF}_s(\varphi)$, and hence encodes patches of a potential refutation. In particular, it will encode information about how blocks are connected: say, some block B will be derived from block B' , meaning that the reservation encodes that the left pointer of B' is B , etc. Hence, when talking about a reservation, we will often say that a certain block has *information about their parents* or *information about their children*, to mean that such a connection is registered in the reservation. Figure 4.1 represents the layout used by the Delayer for his strategy together with such a reservation.

The algorithm proceeds as follows. First, let $\pi := \pi_{\upharpoonright \rho}$, initialize $\alpha := \rho$, and collect in a set $D \subseteq [s]$ the d blocks that are disabled by the restriction ρ . Note that ρ is a d -disabling restriction, meaning that it only enables and disables blocks, and sets the value of all the variables in disabled blocks, meaning that the restriction cannot possibly falsify any axiom of $\text{REF}_s(\varphi)$. Note as well that in π , after hitting it with ρ , there are no longer resolution steps over enabling variables nor over variables belonging to a disabled block.

In what follows, capitals letters in roman font, like A , refer to clauses in the refutation π , while capital letters in calligraphic font, like $\mathcal{A}$, refer to clauses encoded in the blocks of $\text{REF}_s(\varphi)$ which are determined by assignments to the variables of $\text{REF}_s(\varphi)$.

Algorithm 4.2: Block-width analysis and assignment extraction

Let C be the root of π and traverse the proof dag following these instructions.

1. If C is obtained by an illegal derivation step, halt and output C ; if C is a leaf of π , halt and output failure. Otherwise, continue to Step 2.
2. If C was obtained from weakening a clause $C' \subseteq C$, then set $C := C'$ and move to Step 4.
3. If C was derived from clauses $A \vee v$ and $B \vee \neg v$ by resolving over v , attempt the following reservations according to these rules (with the condition that blocks mentioned in the set D can never be reserved).

(a) If v belongs to a block on layer $0 \leq i < n$, where layer 0 stands for the root block, we have two cases:

- i. if α has no information about this block, update α by reserving two unreserved blocks on layer $i + 1$ so that the block of v encodes the clause $\bigvee_{j=1}^i x_j$ and it was obtained by resolving the clauses $x_{i+1} \vee \bigvee_{j=1}^i x_j$ and $\neg x_{i+1} \vee \bigvee_{j=1}^i x_j$, to be encoded in the two reserved blocks on layer $i + 1$;
- ii. if the block was reserved in α but it had no children attached, then α already determined the clause C that is to be encoded in this block. Then, update α by reserving two unreserved blocks on layer $i + 1$ and so that that the block of v encodes the clause C and it was obtained by resolving the clauses $x_{i+1} \vee C$ and $\neg x_{i+1} \vee C$, to be encoded in the two reserved blocks on layer $i + 1$.
- iii. Otherwise, do nothing.

If this reservation fails because there are not enough free blocks available, halt and output the clause C .

(b) If v belongs to a block on layer n , we distinguish two cases:

- i. if the block is not mentioned in α , then try to find the first axiom in φ such that $\bigvee_{j=1}^n x_j$ is a weakening of it, and reserve it so it encodes $\mathcal{A} := \bigvee_{j=1}^n x_j$ and its pointers point to this axiom;

- ii. if the block was reserved in α but it was not pointing to any axiom, then there is a clause $\mathcal{A}$ associated to it by the reservation; try to find the first clause in φ such that $\mathcal{A}$ is a weakening of it and point to it in α .
- iii. Otherwise, do nothing.

If this reservation fails because no axiom could be found, then halt and output the assignment to the variables $x_1, \dots, x_n$ given by $\neg \mathcal{A}$, the negation of the clause encoded by $\mathcal{A}$.

If the reservations succeeded, look at $\alpha(v)$, which is now guaranteed to be defined. If $\alpha(v) = 1$, then move to $C := B \vee \neg v$, and otherwise move to $C := A \vee v$.

4. Clean-up the reservations in α as follows: α should only contain information about (i) blocks disabled by ρ and (ii) blocks mentioned in C or possibly the children of these according to α . Erase all other information from α .
5. Go to Step 1.

Since the algorithm is only traversing a path inside the proof dag of π , the running time of this procedure is never longer than a polynomial in the size of π . As for the correctness of the algorithm, we now show that this behaves exactly as claimed. The central claim is that, at the beginning of each iteration, when looking at clause C , the following invariant is satisfied. Here, $\text{bw}(\alpha)$ stands for the number of blocks mentioned by the variables assigned by α , and $\text{bw}(C)$ is the block-width of a clause C .

Claim (Invariant). The following hold at the beginning of each iteration of the algorithm, when dealing with the clause C in the traversal of π :

- (i) the reservation α falsifies C ;
- (ii) a block $B \notin D$ is only reserved in α if it is either mentioned in C or its parent according to α is mentioned in C , and, in particular, $\text{bw}(\alpha) - d \leq 3 \text{bw}(C)$;
- (iii) if α encodes any information about a block B from layer i , and $B \notin D$, then α also determines that B contains exactly i literals over the variables $x_1, \dots, x_i$ and no two literals for the same variable;
- (iv) the reservation α does not falsify any axiom of $\text{REF}_s(\varphi)$.

Proof sketch. The invariant is readily verified at the initial iteration, when $C = \perp$ and $\alpha = \rho$, the d -disabling restriction given as input.

Now, by straightforward structural induction, it is easy to see that assuming that the invariant holds at the beginning of an iteration and the algorithm correctly proceeds to the next iteration without halting, the invariant holds again at the beginning of the new iteration. $\square$

Note that, if the algorithm reached a clause C that happened to be a leaf of π , then by point (i) of the invariant α would be falsifying C , which would mean falsifying an axiom of $\text{REF}_s(\varphi)$, contradicting point (iv) of the very same invariant. Thus, if π is a correct Resolution refutation, that means that the algorithm always halts before reaching a leaf, and always for one of the following two reasons.

- (a) The algorithm attempted the reservation of a block at level $1 \leq i < n$, but there were no free blocks left. This means that α already reserved at least $\lfloor (s-d)/n \rfloor - 1$ blocks on that layer, and so $\text{bw}(\alpha) \geq \lfloor (s-d)/n \rfloor - 1 + d$, since each layer $i < n$ contains exactly $\lfloor (s-d)/n \rfloor$ blocks. By point (ii) of the invariant we have that $\text{bw}(\alpha) - d \leq 3 \text{bw}(C)$, so putting this together we have that when outputting C we are outputting a clause of block-width at least $1/3 \lfloor (s-d-n)/n \rfloor$, as desired.

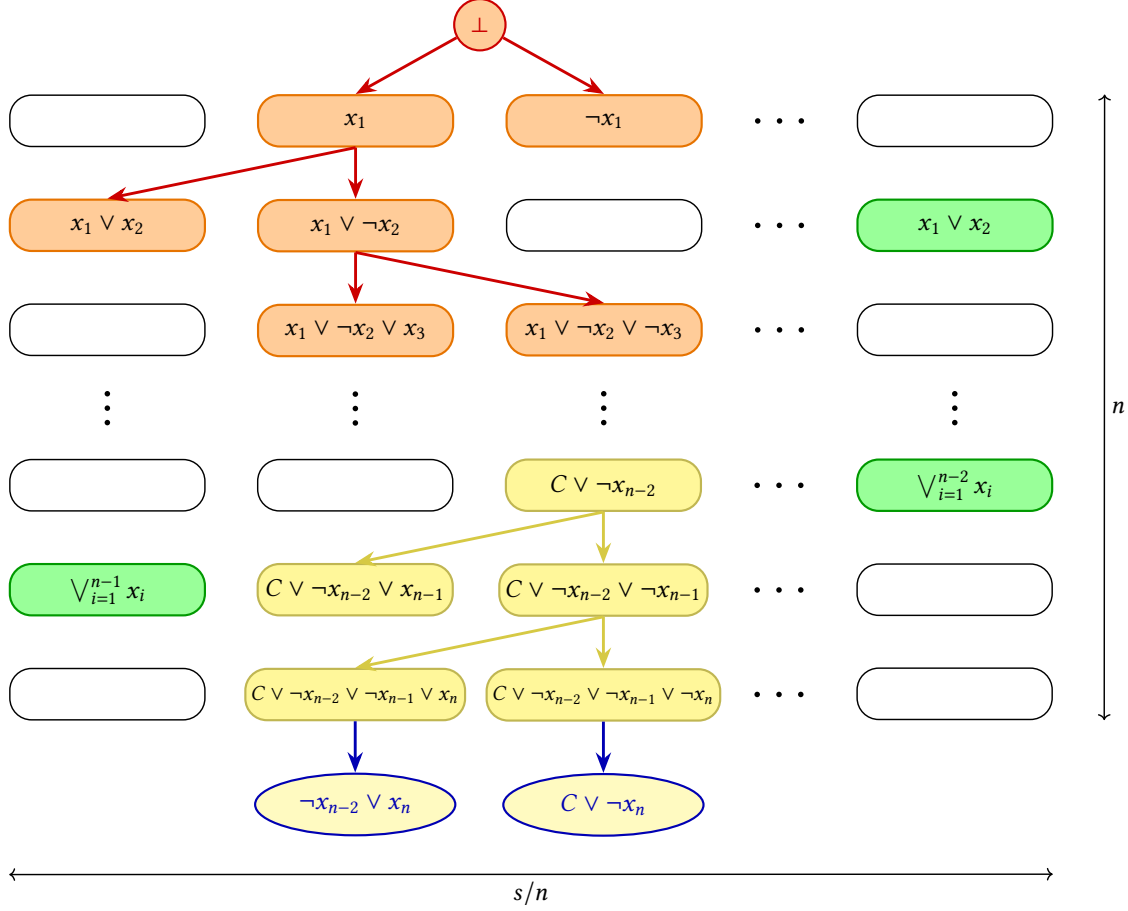


Figure 4.1: Snapshot of the execution of Algorithm 4.2. The Prover–Delayer game is played over the formula $\text{REF}_s(\varphi)$ for s blocks, arranged in n layers with s/n blocks per layer. In **orange** and in **yellow** are two partial trees in which the Delayer answers consistently by locally following the canonical exponential-size tree-like refutation. The **orange** tree starts from the root, while the **yellow** one starts from some clause C of width $n - 3$ that emerged from previous queries. In **green**, blocks that were queried “out of context”: at layer i , the Delayer answers with the clause $x_1 \vee x_2 \vee x_3 \vee \dots \vee x_i$ where all i literals are positive. At layer n there are width- n clauses pointing to some axiom of φ (in **blue**) in a valid weakening step.

- (b) The reservation α had a clause $\mathcal{A}$ encoded in a block at layer n , but it failed to find an axiom of φ that $\mathcal{A}$ was a weakening of. By point (iii) of the invariant, since the block is at layer n , it encodes a width- n clause, and by Fact 4.4, $\neg\mathcal{A}$ encodes a satisfying assignment of φ . In this case the algorithm outputs this assignment, which satisfies the desired behavior.

This completes the proof of correctness of the algorithm. □

4.3 Putting it together

The following is a formal restatement of Theorem 1.1.

Theorem 4.6. *It holds that $\text{FPAP}_{\text{Res}}[n^3] \in \text{FP}$. That is, there exists a deterministic polynomial-time algorithm solving the search version of the Proof Analysis problem for Resolution whenever the lower bound parameter satisfies $s \geq n^3$.*

Proof. Let $(\varphi(x_1, \dots, x_n), \pi, 1^s)$ be an instance of the Proof Analysis Problem with $s \geq n^3$. First, check whether π is indeed a correct Resolution refutation of $\text{REF}_s(\varphi)$. If not, reject. Otherwise, run the algorithm from Lemma 4.3 on π , which outputs a d -disabling restriction ρ , with $d \leq (c/2)\sqrt{s \log |\pi|}$, such that $\pi|_\rho$ is a Resolution refutation of $\text{REF}_s(\varphi)|_\rho$ of block-width at most $c \cdot \sqrt{s \log |\pi|}$, for some fixed constant c . Then run the extraction algorithm from Lemma 4.5 on π and ρ . Since π is a correct refutation, it must be the case that the extraction algorithm from Lemma 4.5 outputs either a clause of $\pi|_\rho$ of block-width at least $(s - d - n)/3n$ or a satisfying assignment of φ . In the latter case, we are done. In the former case, it holds that $1/3 \lfloor (s - d - n)/n \rfloor \leq c \cdot \sqrt{s \log |\pi|}$, which implies $|\pi| > 2^{\varepsilon s/n^2} \geq 2^{\varepsilon \cdot n}$ for some small enough ε and sufficiently large n . Since π is so large we can, in time polynomial in the size of the input, go over all 2^n assignments to the variables of φ and output a satisfying assignment of φ if one exists, and otherwise reject. $\square$

If we are interested in inputs where the lower bound is quadratic instead of cubic, then we can still achieve polynomial time at the cost of randomness.

Theorem 4.7. *There exists a zero-error randomized polynomial-time algorithm solving the search version of the Proof Analysis problem for Resolution whenever the lower bound parameter satisfies $s \geq n^2$. That is, $\text{FPAP}_{\text{Res}}[n^2] \in \text{FZPP}$.*

Proof. We carry out the proof for a fixed constant success probability p , but the argument works for any $p < 1$. The algorithm is essentially the same as before, except we now use the randomized width-reduction procedure in Lemma 4.2 at the beginning, instead of the greedy deterministic one.

Observe that the randomized algorithm in Lemma 4.2 can be used with zero-error, because once a restriction ρ is sampled, we can check if it successfully reduces the block-width to the desired bound, and run it again as many times as needed, which puts us in **FZPP**.

As for why s can now be allowed to be n^2 , observe that the randomized procedure achieves better width reduction, of $O(\log |\pi|)$ whenever p is a constant. Combining this with the $(s - d - n)/3n$ bound of block-width given by Lemma 4.5, which in this case becomes $s/6n - 1/3$ because $d = s/2$, we now have $|\pi| > 2^{\Omega(s/n)}$, meaning that $s \geq n^2$ suffices to obtain an exponential lower bound. $\square$

As discussed in introduction, the deterministic algorithm in Theorem 4.6 gives us a Levin reduction between 3SAT and the Proof Size Problem for Resolution (PSP_{Res} , see Section 2.2.3). Here the search version of 3SAT is the one that finds satisfying assignments of satisfiable formulas, while the search version of PSP_{Res} consists in finding a Resolution refutation of the right size.

Corollary 4.8. *There is a polynomial-time Levin reduction from the search problem for 3SAT to the Proof Size Problem for Resolution.*

Proof. In [AM20], a 3-CNF formula φ is mapped to the formula $\text{REF}_{n^2}(\varphi)$. If, instead, we map it to $\text{REF}_{n^3}(\varphi)$, then this is still a many-one reduction from 3SAT and hardness of automatability still follows, but this is now a Levin reduction: given a satisfying assignment for φ , we can always come up with a short Resolution refutation of $\text{REF}_{n^3}(\varphi)$ using the standard upper bound (see Section 7); and given a Resolution refutation π of $\text{REF}_{n^3}(\varphi)$ of polynomial-size, we can extract a satisfying assignment for φ using Theorem 4.6. $\square$

4.4 Assignment extraction as information efficiency

The statement that Resolution refutations of $\text{REF}_s(\varphi)$ must “leak” satisfying assignments has an eminently information-theoretic flavor. This is no coincidence, as Theorem 4.6 can be interpreted in terms of the *information efficiency* of Resolution refutations, a variant of Kolmogorov complexity in the context of proof complexity.

Krajíček [Kra22] introduced the notion of information efficiency to measure the complexity of describing propositional proofs based on the well-established concept of time-bounded Kolmogorov complexity. Fix a universal Turing machine U such that $U(e, x, 1^t)$ simulates the machine with code e on input x for t steps. For every string $x \in \{0, 1\}^*$, the *conditional time-bounded Kolmogorov complexity* (or *conditional Levin complexity*) of x given a string $y \in \{0, 1\}^*$ is defined as

$$\text{Kt}(x \mid y) := \min\{|e| + \lceil \log t \rceil \mid U(e, y, 1^t) = x\}. \quad (4.9)$$

Then, for a propositional proof system S , the *information efficiency* of a formula $\varphi \in \text{TAUT}$ is defined as

$$\text{info}_S(\varphi) := \min\{\text{Kt}(\pi \mid \varphi) \mid \pi : S \vdash \varphi\}. \quad (4.10)$$

That is, $\text{info}_S(\varphi)$ is the smallest Levin-complexity of a proof π of φ , given the formula φ . It is easy to see that for every $\varphi \in \text{TAUT}$, the bounds $\log \text{size}_S(\varphi) \leq \text{info}_S(\varphi) \leq O(\text{size}_S(\varphi))$ hold. The lower the information efficiency of a formula, the easier its proofs are to describe.

Because of the assignment extraction algorithm, if π is a correct Resolution refutation of $\text{REF}_s(\varphi)$, then the description of π must at least include the description of a satisfying assignment for φ . This means that the information efficiency of $\text{REF}_s(\varphi)$ should be (up to constant factors) precisely the same as the Levin complexity of the easiest satisfying assignment for φ . The following statement makes this precise and can be seen as an information-theoretic analogue of Theorem 4.6.

Theorem 4.9 (Assignment extraction as information efficiency). *For every CNF formula φ over n variables and $\text{poly}(n)$ clauses and every $s \geq n^3$, if φ is satisfiable, then*

$$\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi)) = \Theta(\min\{\text{Kt}(\alpha \mid \varphi) \mid \varphi(\alpha) = 1\}).$$

On the other hand, if φ is unsatisfiable, then $\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi)) = \Omega(n)$.

Proof. We prove first that $\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi)) = O(\min\{\text{Kt}(\alpha \mid \varphi) \mid \varphi(\alpha) = 1\})$. Let α^* be a satisfying assignment that minimizes $\text{Kt}(\alpha \mid \varphi)$ with a description e_{α^*} and time t_{α^*} , and let $P(\varphi, \alpha, s)$ denote the Turing machine that constructs Pudlák's upper bound. Then,

$$\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi)) \leq \text{Kt}(P(\varphi, \alpha^*, s) \mid \neg \text{REF}_s(\varphi)) \quad (4.11)$$

$$\leq |e_{\alpha^*}| + \lceil \log t_{\alpha^*} \rceil + O(1) + O(\log n) \quad (4.12)$$

$$= O(\text{Kt}(\alpha^* \mid \varphi)) \quad (4.13)$$

$$= O(\min\{\text{Kt}(\alpha \mid \varphi) \mid \varphi(\alpha) = 1\}), \quad (4.14)$$

where in eq. (4.12) the constant term $O(1)$ corresponds to the code of Pudlák's algorithm P , and since φ is a CNF with $\text{poly}(n)$ clauses and P runs in polynomial time, the log-term accounting for running P is $\lceil \log n^{O(1)} \rceil = O(\log n)$.

For the other direction, let π^* be a Resolution refutation that minimizes $\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi))$, with description e_{π^*} and time t_{π^*} . Let A denote the assignment extraction algorithm from Theorem 4.6. Then,

$$\min\{\text{Kt}(\alpha \mid \varphi) \mid \varphi(\alpha) = 1\} \leq \text{Kt}(A(\pi^*, \varphi) \mid \varphi) \quad (4.15)$$

$$\leq |e_{\pi^*}| + \lceil \log t_{\pi^*} \rceil + O(1) + O(\log n) \quad (4.16)$$

$$= O(\text{Kt}(\pi^* \mid \varphi)) \quad (4.17)$$

$$= O(\text{Kt}(\pi^* \mid \neg \text{REF}_s(\varphi))) \quad (4.18)$$

$$= O(\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi))). \quad (4.19)$$

We have that eq. (4.15) follows because $s \geq n^3$ so Theorem 4.6 guarantees the extraction algorithm A succeeds in finding a satisfying assignment, while the $O(\log n)$ term in eq. (4.16) follows again because the algorithm A runs in time $n^{O(1)}$.

The case when φ is unsatisfiable is an immediate corollary of the lower bound on REF formulas (Theorem 1.6): since $\text{REF}_s(\varphi)$ requires Resolution size $2^{\Omega(n)}$, then writing such a proof also requires exponential time and hence $\text{Kt}(\pi \mid \neg \text{REF}_s(\varphi)) \geq \log 2^{\Omega(n)} = \Omega(n)$ for every refutation π , giving $\text{info}_{\text{Res}}(\neg \text{REF}_s(\varphi)) = \Omega(n)$. (Note that if φ does have a refutation of size s , then the information efficiency of $\neg \text{REF}_s(\varphi)$ is not defined, so we cannot claim a matching upper bound of $O(n)$.) $\square$

5 The Proof Analysis Problem for Extended Frege is NP-complete

In light of the extraction algorithm in Theorem 4.6, it is natural to ask whether stronger proof systems are also analyzable. As shown in Proposition 3.6, the existence of a polynomial-time proof analysis algorithm for a proof system S implies that automating S is NP-hard. Could this be the route towards proving that automating systems like Extended Frege is NP-hard?

This turns out to be unlikely. While for Resolution $\text{PAP}_{\text{Res}}[n^2] \in \text{AC}^0$ and $\text{FPAP}_{\text{Res}}[n^3] \in \text{FP}$, already the decision problem for Extended Frege PAP_{EF} turns out to be NP-complete. This extends to every proof system S that p -simulates Extended Frege: it is NP-complete to decide whether a formula φ is satisfiable given an S -proof of a Resolution lower bound on φ . We dedicate Section 5.1 to proving this. Section 5.2 builds on this NP-hardness result to investigate whether finding polynomial-time analysis algorithms (for weaker systems where they exist) requires proving proof complexity lower bounds first.

5.1 Hardness proof

The idea of the hardness proof is to show that Extended Frege can prove certain Resolution lower bounds “agnostically”, in the sense that the proof employed will not depend on whether the underlying formula is satisfiable or not. This means that an algorithm that distinguishes between “true Resolution lower bounds” (those where the underlying formula is unsatisfiable) from “trivial ones” (those proven for satisfiable formulas) will be able to decide SAT and hence all of NP.

Our proof works by embedding every propositional formula into a suitable disjunction with the pigeonhole principle formulas. Conveniently, Haken’s Resolution lower bound [Hak85] for the pigeonhole principle was already formalized by Cook and Pitassi in PV_1 .

Theorem 5.1 (Cook and Pitassi, 1990 [CP90]). *There exist a positive $\varepsilon_0 \in \mathbb{Q}$ and $n_0 \in \mathbb{N}$ such that*

$$\text{PV}_1 \vdash \forall n \forall \pi (\text{Ref}_{\text{Res}}(\text{PHP}_{n-1}^n, \pi) \wedge n \geq n_0 \rightarrow \|\pi\| > \varepsilon_0 n).$$

Our reduction will use the following natural encoding of disjunctions as CNF formulas, with the aid of one additional variable.

Definition 5.2. Let $\varphi = C_1 \wedge \dots \wedge C_k$ and $\psi = D_1 \wedge \dots \wedge D_m$ be two CNF formulas over the variables $x_1, \dots, x_n$. We denote by $\text{Disj}(\varphi, \psi)$ the formula

$$\text{Disj}(\varphi, \psi) := \bigwedge_{i=1}^k (\neg y \vee C_i) \wedge \bigwedge_{i=1}^m (y \vee D_i),$$

where y is a fresh variable not present in φ or ψ .

Intuitively, the variable y in the definition above acts as a “switch”: if true, then φ must hold; else, ψ must hold. We can now prove that as long as one of the “disjuncts” is hard for Resolution, the entire formula must be hard as well.

Theorem 5.3. *For every positive constant $c \in \mathbb{N}$, there exist $n_0 \in \mathbb{N}$ such that*

$$S_2^1 \vdash \forall \varphi \forall \pi \forall n \leq \varphi \forall m \leq \varphi (n \geq n_0 \wedge 3\text{-CNF}(\varphi, n, m) \wedge |\pi| \leq n^c \rightarrow \neg \text{Ref}_{\text{Res}}(\text{Disj}(\varphi, \text{PHP}_n), \pi)).$$

Proof. We work in S_2^1 . Suppose for contradiction that π is indeed a Resolution refutation of $\text{Disj}(\varphi, \text{PHP}_n)$. Consider the restriction ρ mapping $y \mapsto 0$. We have that $\text{Disj}(\varphi, \text{PHP}_n) \equiv \text{PHP}_n$. Since S_2^1 can prove that Resolution is closed under restrictions, it follows that $\pi_{\upharpoonright \rho}$ is a Resolution refutation of PHP_n in size n^c . However, by Theorem 5.1, necessarily $\|\pi_{\upharpoonright \rho}\| > \varepsilon_0 n$, implying $\|\pi\| > \varepsilon_0 n$. This contradicts the assumption $|\pi| \leq n^c$, when $n \geq n_0$ for n_0 chosen large enough. $\square$

Corollary 5.4. *For every positive constant $c \in \mathbb{N}$, there exists a polynomial-time computable function t such that for every 3-CNF formula φ over a large enough number n of variables, $t(\varphi)$ outputs an Extended Frege proof π such that*

$$\pi : \text{EF} \vdash \neg \text{REF}_{n^c}(\text{Disj}(\varphi, \text{PHP}_n)).$$

Proof. The formula in Theorem 5.3 is $\forall \Pi_1^b$, so we can apply Cook’s translation (Theorem 2.3) to get polynomial-size EF-proofs of $\neg \text{REF}_{n^c}(\text{Disj}(\varphi, \text{PHP}_n))$. Extended Frege can then polynomially and uniformly prove that the REF-like formula obtained from the translation is equisatisfiable to the REF formulas as we have defined them in Definition 2.4. We then have that EF proves $\neg \text{REF}_{n^c}(\text{Disj}(\varphi, \text{PHP}_n))$ in polynomial size and the proofs can be produced uniformly in polynomial time. $\square$

The previous upper bound in Extended Frege works for every φ , regardless of its satisfiability. This is the key idea behind the final reduction.

Theorem 5.5. *For every positive constant $c \in \mathbb{N}$, the language 3SAT reduces to $\text{PAP}_{\text{EF}}[n^c]$ under polynomial-time many-one Levin reductions.*

Proof. The reduction maps a 3-CNF formula φ over n variables to the instance $(\psi, \pi, 1^s)$, where ψ is the formula $\psi := \neg \text{REF}_{n^c}(\text{Disj}(\varphi, \text{PHP}_n))$, together with the Extended Frege proof π given by the map $t(\varphi)$ in Corollary 5.4, and the size parameter 1^s is 1^{n^c} .

By Corollary 5.4, the proof π is always a correct EF-proof, regardless of the satisfiability of φ . Now, if $\varphi \in 3\text{SAT}$, then there exists a satisfying assignment for φ , which can be extended into a satisfying assignment for $\text{Disj}(\varphi, \text{PHP}_n)$ setting the switching variable $y \mapsto 1$, and hence ψ is satisfiable and $(\psi, \pi, 1^{n^c}) \in \text{PAP}_{\text{EF}}$. On the other hand, if $\varphi \notin 3\text{SAT}$, then π is still a valid EF-proof but we have that $\text{Disj}(\varphi, \text{PHP}_n)$ is unsatisfiable, since any satisfying assignment would need to satisfy either φ or PHP_n , which is impossible. Thus, we get $(\psi, \pi, 1^{n^c}) \notin \text{PAP}_{\text{EF}}$.

Note that this is also a Levin reduction: a satisfying assignment of φ can be immediately extended into a satisfying assignment for ψ , and a satisfying assignment for ψ must necessarily include an assignment to φ . $\square$

This yields the following formal restatement of Theorem 1.3.

Corollary 5.6. *For every propositional proof system Q that p -simulates Extended Frege and every polynomial $s(n)$, the problem $\text{PAP}_Q[s(n)]$ is NP-complete under polynomial-time many-one Levin reductions.*

Proof. Membership in NP is trivial, since $\text{PAP}_Q \in \text{NP}$ for every Cook–Reckhow system Q . Hardness for $\text{PAP}_{\text{EF}}[s(n)]$ is given by Theorem 5.5, and since Q p -simulates EF, this means that an instance $(\psi, \pi, 1^t)$ of PAP_{EF} with $t \geq s(n)$ can be turned into an instance $(\psi, \pi', 1^t)$, where π' is the Q -proof obtained by simulating π . $\square$

It should be clear that there is nothing essential about Extended Frege in the corollary above, except for the fact that it is the weakest proof system where we know how to formalize a Resolution lower bound. The proof generalizes as follows. Below, by saying that a proof system S is *constructively closed under modus ponens*, we mean there exists a constant $p > 0$ such that given an S -proof π of φ and an S -proof τ of $\varphi \rightarrow \psi$, we can obtain a proof of ψ in time $(|\pi| + |\tau|)^p$. Similarly, a proof system S *proves that Resolution is closed under restrictions* if there exists a constant $r > 0$ such that given a formula φ and a restriction ρ , we can obtain an S -proof of $\text{REF}_S(\varphi) \rightarrow \text{REF}_S(\varphi|_\rho)$ in time $(|\varphi| + s)^r$.

Corollary 5.7. *Let S be a propositional proof system constructively closed under modus ponens and proving that Resolution is closed under restrictions and suppose there exists a constant $c > 0$ and a $\mathbf{P}$ -uniform sequence of unsatisfiable formulas $\{\psi_n\}_{n \in \mathbb{N}}$ such that ψ_n has n variables and $\text{poly}(n)$ clauses and there exists a $\mathbf{P}$ -uniform sequence of S -proofs $\{\pi_n\}_{n \in \mathbb{N}}$ such that $\pi_n : S \vdash \neg \text{REF}_{n^c}(\psi_n)$. Then, the problem $\text{PAP}_S[n^c]$ is $\mathbf{NP}$ -complete.*

Proof sketch. The proof follows the argument in Theorem 5.5: the $\mathbf{NP}$ -hardness reduction starts from a formula φ and constructs an S -proof of the formula $\neg \text{REF}_{n^c}(\text{Disj}(\varphi, \psi_n))$, which becomes an instance of $\text{PAP}_S[n^c]$. $\square$

5.2 Does analyzability require lower bounds?

Despite the $\mathbf{NP}$ -completeness results above, it remains open whether proof systems below Extended Frege are analyzable. For those proof systems that are in fact analyzable, it is natural to ask whether their analysis algorithms do inevitably rely on proof complexity lower bounds for the systems in question. After all, the only example of an analysis algorithm we have is the one for Resolution, and it heavily relies on the lower bound proof of [AM20]. It then seems reasonable to conjecture the following.

Conjecture 5.8 (Analyzability requires lower bounds). *For every propositional proof system Q , if Q is analyzable, then Q is not optimal.*

The requirement that Q is not optimal is a natural formalization of lower bounds in this context: if Q is not optimal, then there exists an explicit family of tautologies that are hard for Q , but easy for some other system. Since the lower bounds for REF -like formulas arising in the context of PAP_{Res} are precisely Resolution lower bounds not provable in Resolution but easy for other systems, non-optimality captures the kind of lower bounds we expect are necessary.

We could also relax the conclusion of Q not being optimal by simply Q not being *p-optimal*, which means that there exists an explicit family of tautologies whose Q -proofs are either long, or short but hard to find. Note, in particular, that Q could fail to be *p-optimal* and still be polynomially bounded.

Conjecture 5.9 (Analyzability requires lower bounds or proofs that are hard to find). *For every propositional proof system Q , if Q is analyzable, then Q is not *p-optimal*.*

Clearly, Conjecture 5.8 implies Conjecture 5.9. Building on the proof of $\mathbf{NP}$ -hardness of PAP_{EF} we can prove that both conjectures are likely true—but very hard to prove.

Proposition 5.10. *The following hold:*

- (i) *Conjecture 5.8 is true if, and only if, $\mathbf{NP} \neq \mathbf{coNP}$;*
- (ii) *Conjecture 5.9 is true if, and only if, $\mathbf{P} \neq \mathbf{NP}$.*

Proof. We start by proving (i). For the forward direction, we assume $\mathbf{NP} = \mathbf{coNP}$ and show that the conjecture fails. In this case there exists a polynomially bounded and hence optimal proof system Q . We define the following proof system Q^* based on Q . A proof π in this system is of one of the following forms:

- (a) if $\pi = \langle \varphi, \tau_1, \neg\text{REF}_s(\varphi), \tau_2 \rangle$, where φ is a CNF formula, τ_1 is a correct Q -proof of $\neg\varphi$, and τ_2 is a correct Q -proof of $\neg\text{REF}_s(\varphi)$ for some $s \in \mathbb{N}$, then $Q^\star$ outputs $\neg\text{REF}_s(\varphi)$;
- (b) if $\pi = \langle \varphi, \alpha, \neg\text{REF}_s(\varphi), \tau \rangle$, where φ is a CNF formula, $\varphi(\alpha) = 1$, and τ is a correct Q -proof of $\neg\text{REF}_s(\varphi)$ for some $s \in \mathbb{N}$, then $Q^\star$ outputs $\neg\text{REF}_s(\varphi)$;
- (c) if $\pi = \langle \varphi, \tau \rangle$ and φ is not of the form $\neg\text{REF}_s(\psi)$ for any s and ψ , and τ is a correct Q -proof of φ , then $Q^\star$ outputs φ ;
- (d) in any other case, $Q^\star$ outputs a trivial tautology $p \vee \neg p$.

We claim that $Q^\star$ is a Cook–Reckhow system that is both analyzable and optimal. First, it is sound, because Q is and every $Q^\star$ -proof relies on correct Q -proofs; second, it is complete, because if φ is not a $\neg\text{REF}$ formula, then one can always prove φ via case (c) above using the completeness of Q , and if $\varphi = \neg\text{REF}_s(\psi)$ for some ψ , then since Q is complete, there is always a proof of $\neg\text{REF}_s(\psi)$ together with either a Q -proof of $\neg\psi$ or a satisfying assignment for ψ . Finally, $Q^\star$ is clearly polynomial-time computable because Q is.

It remains to argue that $Q^\star$ is both analyzable and optimal. Indeed, $\text{PAP}_{Q^\star} \in \mathbf{P}$: by definition, if π is a $Q^\star$ -proof of $\neg\text{REF}_s(\varphi)$ for some s , then π is of the form (a) or (b) above. If π is of the form (a), then π includes a proof of the unsatisfiability of φ , so the analyzer immediately rejects after checking the correctness of this proof. If π is of the form (b), then the analyzer can simply check that α is a correct satisfying assignment for φ and conclude that φ is satisfiable. At the same time, it is easy to see that $Q^\star$ is polynomially bounded, and hence optimal. Indeed, all non- $\neg\text{REF}$ formulas have polynomial-size proofs by case (c) as Q is polynomially bounded. Similarly, in cases (a) and (b), there are always polynomial-size τ_1 and τ_2 to choose as Q is polynomially bounded, so the entire $Q^\star$ is polynomially bounded. This means that the conjecture fails for $Q^\star$.

Now, for the backwards direction, if the conjecture fails, we show that $\mathbf{NP} = \mathbf{coNP}$. For the conjecture to fail there must exist a proof system $S^\star$ that is analyzable yet optimal. Suppose $S^\star$ is analyzable by virtue of $\text{PAP}_{S^\star}[n^c] \in \mathbf{P}$ for some $c > 0$. We then construct a non-deterministic polynomial-time procedure for the $\mathbf{coNP}$ -complete set 3UNSAT . On an input 3-CNF formula φ with n variables and m clauses, non-deterministically guess an $S^\star$ -proof of the formula $\neg\text{REF}_{n^c}(\text{Disj}(\varphi, \text{PHP}_n))$. By Corollary 5.4, polynomial-size such proofs exist in Extended Frege. Since $S^\star$ is optimal we have $S^\star \geq \text{EF}$ and some polynomial-size proof is available in $S^\star$ too. Now, since $S^\star$ is analyzable, we can decide deterministically in polynomial-time whether φ is satisfiable or not by analyzing this proof.

We now prove (ii). If $\mathbf{P} = \mathbf{NP}$, then there exists a polynomially bounded and p-optimal proof system $Q^\star$. Since $\mathbf{P} = \mathbf{NP}$ the system $Q^\star$ is trivially analyzable, but it is p-optimal, so the conjecture fails for $Q^\star$.

For the other direction, suppose $\mathbf{P} \neq \mathbf{NP}$ but Conjecture 5.9 fails. That means there exists a system that is analyzable yet p-optimal. Since S is p-optimal, it follows that S p-simulates EF, and by Corollary 5.6 we have that $\text{PAP}_S[n^c]$ is $\mathbf{NP}$ -complete for every $c \in \mathbb{N}$. Since $\mathbf{P} \neq \mathbf{NP}$ we get that $\text{PAP}_S[n^c] \notin \mathbf{P}$ for any $c > 0$, a contradiction with the fact that S is analyzable. $\square$

We finish by proving that under a standard assumption in circuit complexity, the analyzability of a propositional proof system is in fact equivalent to lower bounds on *all* the non-trivial REF formulas. The following statement makes this precise, and is the formal restatement of Theorem 1.4. (The statement assumes some mild conditions on the behavior of propositional proof systems; see Proposition 9.2 for formal definitions of these, and the paragraph before Corollary 5.7 for the definition of a system *proving that Resolution is closed under restrictions*.)

Theorem 5.11. *Let S be a propositional proof system that simulates Resolution, is constructively closed under modus ponens and contrapositions, and proves that Resolution is closed under restrictions. Assuming there exists a constant $\delta > 0$ such that $3\text{SAT} \notin \mathbf{ioSIZE}[2^{\delta n}]$, the following are equivalent:*

- (i) *the proof system S is analyzable;*

- (ii) there are constants $c > 0$, $n_0 \geq 1$ and $\varepsilon > 0$ such that for every unsatisfiable 3-CNF formula φ over $n \geq n_0$ variables, every S -proof of the formula $\neg \text{REF}_{n^c}(\varphi)$ requires size at least $2^{\varepsilon n}$.

Proof. It is easy to see that if item (ii) holds for constant c , then $\text{PAP}_S[n^c] \in \mathbf{P}$ (this is precisely the argument behind Proposition 3.4, and does not require the circuit lower bound assumption), so we focus on the implication from item (i) to item (ii). By contradiction: suppose S is analyzable and hence $\text{PAP}_S[n^c] \in \mathbf{P}$ for some constant $c > 0$, but item (ii) fails: then, in particular, for every $\varepsilon > 0$ there must exist a (possibly non-uniform) sequence of unsatisfiable 3-CNF formulas $\{\psi_n\}_{n \in \mathbb{N}}$ such that for infinitely many n , the system S can prove $\neg \text{REF}_{n^c}(\psi_n)$ in size at most $2^{\varepsilon n}$.

The goal is now to construct a (possibly non-uniform) circuit family that solves 3SAT in time $O(2^{\delta n})$ infinitely often by using the NP-hardness reduction in Corollary 5.6 as follows. Suppose that d is the constant such that $\text{PAP}_S[n^c] \in \mathbf{P}$ by virtue of a family of Boolean circuits of size n^d , and let r and p be, respectively, the constants in the blow-up for proving that Resolution is closed under restrictions and the closure under under contrapositions. Let $\varepsilon := \delta/dpr$ and let $\{\psi_n\}_{n \in \mathbb{N}}$ be the sequence of formulas as in the paragraph above for this particular choice of ε . Now, given a 3-CNF formula φ over n variables, suppose n is one of the infinitely many values for which there is an S -proof π_0 of $\neg \text{REF}_{n^c}(\psi_n)$ in size $2^{\varepsilon n}$. From the fact that such a proof exists and S proves that Resolution is closed under restrictions, it follows that there is also an S -proof π of $\neg \text{REF}_{n^c}(\text{Disj}(\varphi, \psi_n))$ in size $2^{\varepsilon prn}$: apply first the fact that Resolution is closed under restrictions to get a proof of $\text{REF}_{n^c}(\text{Disj}(\varphi, \psi_n)) \rightarrow \text{REF}_{n^c}(\psi_n)$; then, contraposition with $\neg \text{REF}_{n^c}(\psi_n)$ yields the desired formula and the size of the proof is at most $|\pi_0|^{pr}$. Now, since $\text{PAP}_S[n^c] \in \mathbf{P}$ and there exists a Boolean circuit of size n^d solving this problem, we can use this circuit on the input $(\text{Disj}(\varphi, \psi_n), \pi, 1^{n^c})$, which we construct using π_0 as advice. The result of this computation tells us whether φ is satisfiable, and the overall size of the circuit is $2^{\varepsilon dprn} = 2^{\delta n}$. This means $3\text{SAT} \in \text{ioSIZE}[2^{\delta n}]$, contradicting our assumption. $\square$

6 The Atserias–Müller lower bound in PV_1

In this section we undertake the task of formalizing the Atserias–Müller lower bound in bounded arithmetic to prove Theorem 1.5. There are at least two possible approaches here, at least at a conceptual level. The first one is to formalize in PV_1 the correctness of the extraction algorithm from Section 4, and use this to derive the lower bound along the lines of the argument in Theorem 4.6, carefully verifying that we never exceed the reasoning power of the theory.

The alternative approach, which we choose to take, is to forget about the extraction algorithm at first glance and attempt a direct formalization of the theorem seeing it purely as a proof complexity lower bound. Then, from the form of the statement, witnessing theorems will recover the algorithm together with its proof of correctness. We opt for this second route, since it is in fact the original hint for why the algorithm should exist in the first place, and because restating the proof in this format can be insightful in itself.

Of course, the proof we choose to formalize deviates from the original one in a few crucial points, although the overall structure is preserved. The main difference is the deterministic restriction replacing the random restriction argument in [AM20]. This deterministic procedure corresponds exactly to the greedy algorithm behind the width-reduction technique in Lemma 4.3.

We start by stating the lower bound in the language of PV_1 , which posits that a correct Resolution refutation π of $\text{REF}_s(\varphi)$ must have size at least $2^{\varepsilon s/n^2}$ whenever $\varphi \notin \text{SAT}$, for some fixed positive ε . Since we do not have exponentiation in PV_1 , we scale down the bound and state it as $||\pi|| > \varepsilon s/n^2$, and the

first-order formula becomes

$$\begin{aligned} \text{AM}_{\varepsilon, n_0} := \forall \varphi \forall \pi \forall n \forall s \Big(& \left(n \geq n_0 \wedge \text{CNF}(\varphi, n) \wedge \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \right. \\ & \left. \wedge \forall \alpha \leq \varphi(\neg \text{Sat}(\varphi, \alpha)) \right) \rightarrow \|\pi\| > \varepsilon s / n^2 \Big). \end{aligned} \quad (\text{AM}_{\varepsilon, n_0})$$

Here the different predicates and function symbols all have the obvious intended meaning and it is readily verified that they are all computable in polynomial time and there exist function symbols for them in the language of PV.

The formula itself is $\forall \Sigma_1^b$. This becomes clear when rewriting it in prenex form, which gives us the statement

$$\forall \varphi \forall n \forall s \forall \pi \exists \alpha \leq \varphi \left(n < n_0 \vee \neg \text{CNF}(\varphi, n) \vee \neg \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \vee \text{Sat}(\varphi, \alpha) \vee \|\pi\| > \varepsilon s / n^2 \right). \quad (6.1)$$

If we were to apply Buss's witnessing theorem here, we would get a polynomial-time function witnessing the existential quantifier on α , which corresponds precisely to the extraction algorithm.

We carry out the formalization in three subsections. Section 6.1 proves the restriction argument, Section 6.2 carries out the block-width lower bound and Section 6.3 puts these together. We also recall that $S_2^1(\text{PV})$, which we denote simply as S_2^1 , is $\forall \Sigma_1^b$ -conservative over PV_1 , so we will often carry out arguments in S_2^1 instead of PV_1 and use the induction principles available there without further comment.

6.1 The restriction argument

We want to prove the following $\forall \Sigma_1^b$ formula, stating the restriction argument:

$$\begin{aligned} \text{AM-Restriction}_c := \forall \varphi \forall n \forall s \forall \pi \Big(& \text{CNF}(\varphi, n) \wedge \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \\ & \rightarrow \exists \rho \leq \pi \Big(\text{bw}(\pi_{\upharpoonright \rho}) \leq c \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil \\ & \quad \wedge \text{REF}_s(\varphi)_{\upharpoonright \rho} \neq \perp \\ & \quad \wedge \text{Disabling}(\rho, \varphi, n, s) \leq c/2 \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil \Big). \end{aligned} \quad (\text{AM-Restriction}_c)$$

The formula AM-Restriction_c states that for every CNF formula φ over n variables and every correct Resolution refutation π of the $\text{REF}_s(\varphi)$ formula, there exists a d -disabling restriction $\rho \in \{0, 1, *\}^N$ to the N variables of $\text{REF}_s(\varphi)$ such that $\text{bw}(\pi_{\upharpoonright \rho}) \leq c \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil$ and $d \leq c/2 \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil$.

Throughout this section we freely use rationals and reals and notation like $\sqrt{\cdot}$, always assuming that some suitable rational approximation is used under the hood. This is standard for formalizations in these theories (see, for example, the standard style of the formalizations by Jeřábek [Jeř05]).

It is easy to see that there exists a PV function W that given π outputs the set of clauses in π of block-width strictly larger than $c \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil$. Similarly, there is a PV function $W_{\upharpoonright \rho}$ that given π and ρ defines the set of clauses in $\pi_{\upharpoonright \rho}$ of block-width strictly larger than $c \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil$, and the basic properties of these functions can be proven in S_2^1 .

For a given restriction ρ that sets values of variables in ℓ different blocks, we will denote by $\tau := ((b_1, X_1), \dots, (b_\ell, X_\ell))$ an ordering $b_1, \dots, b_\ell \in [s]$ of the blocks that are mentioned by ρ , and, for each of them, an ordering X_i of all the variables in block b_i . We call τ a *trace* for ρ and for every $k \in [\ell]$ and S a set

of variables of $\text{REF}_s(\varphi)$, we denote by $\rho_{k,S}$ the subrestriction of ρ defined as

$$\rho_{k,S}(x) := \begin{cases} \rho(x) & \text{if } x \text{ belongs in one of the blocks } b_1, \dots, b_k \in \tau \text{ or if } x \in S \\ * & \text{otherwise} \end{cases} \quad (6.2)$$

and $\rho_{0,S} := *^N$. We will denote by ρ_k the subrestriction $\rho_{k,\emptyset}$.

Definition 6.1 (Most-killing property). We will say that a restriction ρ enjoys the *most-killing property* with respect to a trace $\tau = ((b_1, X_1), \dots, (b_\ell, X_\ell))$ if it holds that for every $k \in [\ell]$, the block $i \in [s] \setminus \{b_1, \dots, b_{k-1}\}$ that is the most frequent block mentioned in $W_{\upharpoonright \rho_{k-1}}$ happens to be precisely b_k and ρ_k satisfies the following conditions.

1. If e_i appears positively in at least $1/3$ of all the clauses in $W_{\upharpoonright \rho_{k-1}}$ that mention block i , then $\rho_k(e_i) = 1$ and $\rho_k(x) = *$ for every other variable $x \neq e_i$ in block i .
2. If e_i does *not* appear positively in at least $1/3$ of all the clauses in $W_{\upharpoonright \rho_{k-1}}$ that mention block i , then $\rho_k(e_i) = 0$ and for every other variable in block i , ρ_k assigns values to them respecting the ordering in $X_i = (e_i, x_{i,1}, x_{i,2}, \dots)$ and with the following priority. For $j = 1, \dots, |X_i| - 1$,
 - (a) if $x_{i,j}$ appears positively more often than negatively in $W_{\upharpoonright \rho_{k-1}, \{e_i, x_{i,1}, \dots, x_{i,j-1}\}}$, then $\rho_k(x_{i,j}) = 1$;
 - (b) if $x_{i,j}$ appears negatively more often than positively in $W_{\upharpoonright \rho_{k-1}, \{e_i, x_{i,1}, \dots, x_{i,j-1}\}}$, then $\rho_k(x_{i,j}) = 0$.

Note that verifying that ρ enjoys the most-killing property with respect to τ is possible in polynomial-time and hence there is a PV function that performs the check and S_2^1 can prove this.

We now prove the following claim by induction.

Lemma 6.2. *Let $c \in \mathbb{N}$. The following is provable in S_2^1 . For every CNF formula φ over n variables and π a correct Resolution refutation of $\text{REF}_s(\varphi)$, let $w := c \cdot \left\lceil \sqrt{s \log |\pi|} \right\rceil$ and let W denote the set of clauses in π of block-width larger than w . Then, for every ℓ , the formula $\Psi(\varphi, \pi, s, \ell)$ defined as follows holds: if $\ell \leq w/2$, then there exists a restriction $\rho \in \{0, 1, *\}^N$, where N is the number of variables of the $\text{REF}_s(\varphi)$ formula, a trace $\tau = ((b_1, X_1), \dots, (b_\ell, X_\ell))$ and $d \leq \ell$ such that:*

- (i) ρ is d -disabling;
- (ii) ρ has the most-killing property according to τ ;
- (iii) $(3s)^\ell \cdot |W_{\upharpoonright \rho}| \leq |W| \cdot (3s - w + \ell)^\ell$.

Proof. Observe that the formula Ψ is Σ_1^b . First, the existential quantifiers on ρ , τ and d are bounded by π and φ . The properties (i) and (ii) are checkable in polynomial time and hence there are PV relations for them and S_2^1 can prove their basic properties. To show that item (iii) can be properly expressed by a PV formula, note that because π is a correct Resolution refutation of $\text{REF}_s(\varphi)$, it must hold that $s \leq |\pi|$ and

$$w = c \left\lceil \sqrt{s \log |\pi|} \right\rceil \leq c \left\lceil \sqrt{s |\pi|} \right\rceil \leq c \left\lceil \sqrt{|\pi| |\pi|} \right\rceil \leq c |\pi|, \quad (6.3)$$

where the last equality holds because $|\pi| \leq |\pi|$. As a consequence, $w \in \text{Log}$, which means that for every $\ell \leq w/2$, $\ell \in \text{Log}$ and thus s^ℓ exists because $s^\ell = 2^{\lceil s \rceil \ell} = s \# 2^\ell$.

We show that $S_2^1 \vdash \forall \ell \Psi(\varphi, \pi, s, \ell)$. We proceed by induction on the parameter ℓ , the length of the trace τ . Note that this corresponds to Length Induction over Σ_1^b formulas and is hence available in S_2^1 .

If $\ell = 0$, then we can take the empty restriction $\rho := *^N$, which disables $d = 0$ variables and the trace τ to be the empty sequence, which trivially satisfy the conditions.

We assume now that $\Psi(\varphi, \pi, s, \ell)$ holds and we prove $\Psi(\varphi, \pi, s, \ell + 1)$. If $\ell + 1 > w/2$, then we are done, so assume $\ell + 1 \leq w/2$. By induction hypothesis, there exist ρ , d and τ satisfying the desired properties for ℓ . We extend ρ into ρ' by following the conditions of the most-killing property. This amounts to applying one iteration of Algorithm 4.1. There is a PV function for this, as well as for extracting the most frequent block in the refutation and S_2^1 can prove the basic properties of these functions. The trace is then extended by the most frequent block $b_{\ell+1}$ mentioned in $W_{\upharpoonright\rho}$ and the ordering of the variables can be any fixed ordering of the variables used in the construction of ρ' by Algorithm 4.1.

If ρ was d -disabling for $d \leq \ell$, then we have that ρ' is at most $(d + 1)$ -disabling, since one iteration of Algorithm 4.1 disables at most one additional block, and we have $d + 1 \leq \ell + 1$. By construction, it is easy to see that ρ' has the most-killing property with respect to τ' . It is only left to verify that

$$(3s)^{\ell+1} \cdot |W_{\upharpoonright\rho'}| \leq |W| \cdot (3s - w + \ell + 1)^{\ell+1}. \quad (6.4)$$

The set W contains all the clauses in π of block-width at least w . The restriction ρ' sets the variables of $\ell + 1$ blocks, meaning that every clause in W not trivialized by ρ' has block-width at least $w - (\ell + 1)$, and these clauses are precisely the ones in $W_{\upharpoonright\rho'}$ by definition.

Furthermore, the restriction ρ' disables at most $\ell + 1$ blocks, so an averaging argument implies that there is a non-restricted block mentioned in at least $|W_{\upharpoonright\rho}|(w - (\ell + 1))/(s - (\ell + 1))$ clauses. In particular, the most frequent non-restricted block must be mentioned at least that often. Since $|W| \leq |\pi|$ and $s \leq |\pi|$, we have that $|W|$ and s are in Log and therefore we can use exact counting in S_2^1 to carry out this averaging argument (see Section 2.3.2 for details).

By the way we constructed ρ' following Algorithm 4.1 we are guaranteed to kill at least $1/3$ of all the clauses mentioning the most frequent unrestricted block $b_{\ell+1}$. Indeed, by inspecting the conditions of the most-killing property (Definition 6.1) if we enable block $b_{\ell+1}$ that is because it appeared in at least $1/3$ of all the clauses mentioning $b_{\ell+1}$; and otherwise we are guaranteed to restrict at least $1/2$ of the remaining $2/3$ fraction of the clauses mentioning $b_{\ell+1}$, which amounts to a $1/3$ fraction.

Together, this means that

$$|W_{\upharpoonright\rho'}| \leq |W_{\upharpoonright\rho}| \cdot \left(1 - \frac{w - (\ell + 1)}{3s}\right), \quad (6.5)$$

which is the same as

$$3s \cdot |W_{\upharpoonright\rho'}| \leq |W_{\upharpoonright\rho}| \cdot (3s - w + \ell + 1). \quad (6.6)$$

As ρ' extends the restriction ρ , it is clear that $|W_{\upharpoonright\rho'}| \leq |W_{\upharpoonright\rho}|$, and by induction hypothesis we know that

$$(3s)^\ell \cdot |W_{\upharpoonright\rho}| \leq |W| \cdot (3s - w + \ell)^\ell, \quad (6.7)$$

meaning that

$$3s \cdot |W_{\upharpoonright\rho'}| \leq |W_{\upharpoonright\rho}| \cdot (3s - w + \ell + 1) \quad (6.8)$$

$$\leq \frac{1}{(3s)^\ell} |W| \cdot (3s - w + \ell)^\ell \cdot (3s - w + \ell + 1) \quad (6.9)$$

$$\leq \frac{1}{(3s)^\ell} |W| \cdot (3s - w + \ell + 1)^{\ell+1}, \quad (6.10)$$

from which the desired $(3s)^{\ell+1} \cdot |W_{\upharpoonright\rho'}| \leq |W| \cdot (3s - w + \ell + 1)^{\ell+1}$ follows. $\square$

Before we complete the proof, we have the following technical lemma that will help us deal with our bounds inside the theory.

Lemma 6.3. *In PV_1 , for all $n, m, p \in \text{Log}$, if $n < m$ and $p \geq 1$, it holds that*

$$\left(1 - \frac{n}{m}\right)^p \leq 2^{-\frac{n}{m}p}.$$

Proof. Since $p \in \text{Log}$, we take n and m to be fixed and proceed by Length Induction on p . The base case $p = 1$ is covered by a proposition of Müller and Pich [MP20, Proposition 2.5], who formalized in PV_1 that for all $n, m \in \text{Log}$ such that $n < m$, it holds that $1 - n/m \leq 2^{-n/m}$. If the bound holds for p , then for $p + 1$ we have

$$\left(1 - \frac{n}{m}\right)^{p+1} = \left(1 - \frac{n}{m}\right)^p \cdot \left(1 - \frac{n}{m}\right) \leq 2^{-\frac{n}{m}p} \cdot 2^{-\frac{n}{m}} = 2^{-\frac{n}{m}(p+1)}. \quad (6.11)$$

The first equality holds because $(1 - n/m) \in \text{Log}$ and PV_1 can prove the basic properties of powers for Log-sized objects; the next inequality holds by induction hypothesis and the same proposition by Müller and Pich as in the base case. This completes the proof. $\square$

We are ready to show that PV_1 proves the desired restriction argument.

Lemma 6.4 (Formalized deterministic width-reduction in PV_1). *It holds that $PV_1 \vdash \text{AM-Restriction}_4$.*

Proof. Let φ be a CNF formula over n variables and π a correct Resolution refutation of $\text{REF}_s(\varphi)$ for some s . Apply Lemma 6.2 in S_2^1 for $\ell := w/2$, where $w := 4 \cdot \lceil \sqrt{s \log |\pi|} \rceil$. This gives us a restriction ρ that is d -disabling for $d \leq \ell$ and $(3s)^\ell \cdot |W_{\upharpoonright \rho}| \leq |W| \cdot (3s - w + \ell)^\ell$. We want to show that $\text{bw}(\pi_{\upharpoonright \rho}) \leq w$, that $\text{REF}_s(\varphi)_{\upharpoonright \rho} \neq \perp$ and that ρ is d -disabling, for $d \leq w/2$.

That ρ is d -disabling is guaranteed by the lemma, and because it is d -disabling, the restriction only sets the values of enabling variables and, whenever a block is enabled, it does not restrict any other variables in the block. As a consequence, no axiom of $\text{REF}_s(\varphi)$ is ever falsified by ρ .

It is only left to verify that the block-width of $\pi_{\upharpoonright \rho}$ is at most w , as desired. Lemma 6.2 guarantees that $|W_{\upharpoonright \rho}| \leq |W| \cdot (1 - (w - \ell)/3s)^\ell$, and substituting our choice of ℓ we have that

$$|W_{\upharpoonright \rho}| \leq |W| \cdot \left(1 - \frac{w/2}{3s}\right)^{w/2} \leq |W| \cdot 2^{-\frac{w^2}{12s}}, \quad (6.12)$$

where the last inequality is provable in S_2^1 , as shown in Lemma 6.3.

We want $|W| \cdot 2^{-w^2/12s} < 1$, so

$$|W| \cdot 2^{-\frac{w^2}{12s}} < 1 \Leftrightarrow \log |W| < \frac{w^2}{12s} \quad (6.13)$$

$$\Leftrightarrow \sqrt{12s \log |W|} < w, \quad (6.14)$$

and this last inequality is verified for our choice of w .

It follows that $W_{\upharpoonright \rho} = \emptyset$ and hence $\text{bw}(\pi_{\upharpoonright \rho}) \leq w$, as desired. Since S_2^1 is $\forall \Sigma_1^b$ -conservative over PV_1 , we have the sentence in PV_1 . $\square$

6.2 The block-width lower bound

The crucial part of the formalization, and the place where we most clearly see how satisfying assignments can be extracted from Resolution refutations, is in the width lower bound. The restriction argument we just covered reduces the block-width of a refutation regardless of whether the underlying formula is satisfiable or not; it is the width lower bound that only holds when the formula is unsatisfiable.

Let us first state the formula we want to prove, which we encode in the following $\forall\Sigma_1^b$ statement:

$$\begin{aligned} \text{AM-WLB} := \forall\varphi\forall n\forall s\forall\pi \Big(& \text{CNF}(\varphi, n) \wedge \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \wedge \forall\alpha \leq \varphi. \neg\text{SAT}(\varphi, \alpha) \\ & \rightarrow \exists C \leq \pi \Big(C \in \pi \wedge \text{bw}(C) \geq 1/3(\lfloor s/n \rfloor - 1) \Big) \Big). \end{aligned} \quad (\text{AM-WLB})$$

When writing the formula in prenex form, pulling out the universal quantifier on α and turning it into an existential one, it becomes clear how the statement is $\forall\Sigma_1^b$ and the extraction of a satisfying assignment can be performed by witnessing the first existential quantifier:

$$\begin{aligned} \text{AM-WLB} \equiv \forall\varphi\forall n\forall s\forall\pi \exists\alpha \leq \varphi \exists C \leq \pi \Big(& \text{CNF}(\varphi, n) \wedge \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \\ & \rightarrow \text{Sat}(\varphi, \alpha) \vee \Big(C \in \pi \wedge \text{bw}(C) \geq 1/3(\lfloor s/n \rfloor - 1) \Big) \Big). \end{aligned} \quad (6.15)$$

The crucial fact that makes the block-width lower bound go through, and which fundamentally distinguishes the SAT and UNSAT cases is that, if $\varphi \in \text{UNSAT}$, then every width- n clause over the variables of φ can be weakened from some clause of φ , while a width- n clause that is *not* the weakening of any axiom will necessarily encode a satisfying assignment. This was earlier stated as Fact 4.4. We now reprove the statement in S_2^1 .

Lemma 6.5 (Fact 4.4 in S_2^1). *Let φ be a Boolean formula in CNF over n variables. If C is a width- n clause over the variables of φ that is not the weakening of any clause of φ , then $\neg C$ encodes a satisfying assignment for φ .*

Proof. By Δ_1^b -induction on the number n of variables $x_1, \dots, x_n$ of φ , which is available in S_2^1 .

If $n = 1$, then the only two width-1 clauses are x_1 and $\neg x_1$. The three possible CNF formulas are x_1 , $\neg x_1$ and $x_1 \wedge \neg x_1$, and it is easy to check that the statement holds.

Suppose the statement holds for formulas with up to n variables, and let φ be a formula over $n + 1$ variables and let C be a clause of width $n + 1$. Assume $x_{n+1} \in C$, and restrict $x_{n+1} \mapsto 0$. We obtain a new formula φ_0 and a clause C_0 . For every clause D in φ we have the following cases:

- (a) the variable x_{n+1} appeared in D negatively, and hence D is satisfied by setting $x_{n+1} \mapsto 0$;
- (b) the variable x_{n+1} appeared in D positively, or the variable did not appear at all; in both cases we have that this is a clause over variables $x_1, \dots, x_n$ and, by assumption, C and hence also C_0 is not a weakening of it. By induction hypothesis, $\neg C_0$ and then also $\neg C$ encode a satisfying assignment to D .

Hence, every clause in φ is satisfied by $\neg C$. If $\neg x_{n+1} \in D$ the the same argument goes through by restricting $x_{n+1} \mapsto 1$. This completes the proof. $\square$

To prove the rest of the width lower bound, we show the following lemma first, which essentially corresponds to the invariant proved within Lemma 4.5 to derive the correctness of the assignment extraction algorithm. We restate the invariant here as a property of a path in the Resolution refutation.

Definition 6.6 (Reservation invariant). Let π be a Resolution refutation of $\text{REF}_s(\varphi)$ for some CNF formula $\varphi(x_1, \dots, x_n)$, let N be the number of variables of $\text{REF}_s(\varphi)$, $d \leq \text{depth}(\pi)$, and let $C = (C_1, \dots, C_d)$ be a length- d path in π starting from $C_1 = \perp$. Let the s blocks of $\text{REF}_s(\varphi)$ be arranged in a layered manner, so that there are n layers, each containing $\lfloor s/n \rfloor$ blocks, with the remainder blocks left from the flooring operations collected all in the last layer, plus one additional layer on top with a single block corresponding to the root. We say that C satisfies the *reservation invariant* with respect to a sequence $A = (\alpha_1, \dots, \alpha_d)$ of restrictions $\alpha_i \in \{0, 1, *\}^N$ if, for every $i \in [d]$,

- (i) the restriction α_i falsifies C_i ;
- (ii) a block is only mentioned in α_i if it is either mentioned in C_i or its parent according to α_i is mentioned in C_i , and, in particular, $\text{bw}(\alpha_i) \leq 3 \text{bw}(C_i)$;
- (iii) if α_i restricts some variable in block B from layer ℓ , then it does so in the following way: B must contain exactly ℓ literals over the variables $x_1, \dots, x_\ell$ and no two literals for the same variable;
- (iv) the restriction α_i does not falsify any clause of $\text{REF}_s(\varphi)$;
- (v) the path determined by C corresponds to A in the following way: if C_i was derived from C_{i+1} by resolving over variable v , then $\alpha_{i+1}(v)$ is defined and v appears in C_{i+1} with polarity $1 - \alpha_{i+1}(v)$.

The lemma now states that a path exists that satisfies the invariant or, alternatively, correctly leads to a wide clause or a satisfying assignment.

Lemma 6.7. *The following is provable in S_2^1 . For every CNF formula φ over n variables and π a correct Resolution refutation of $\text{REF}_s(\varphi)$, for every d , the formula $\Psi(\varphi, \pi, s, d)$ defined as follows holds: if $1 \leq d \leq \text{depth}(\pi)$, then there exists a sequence $C = (C_1, \dots, C_d)$ of clauses of π and a sequence $A = (\alpha_1, \dots, \alpha_d)$ of restrictions such that at least one of the following conditions holds:*

- (a) *there is $C_i \in C$ that satisfies $\text{bw}(C_i) \geq 1/3(\lfloor s/n \rfloor - 1)$;*
- (b) *there is C_i in and its corresponding restriction α_i that contain a satisfying assignment to φ ;*
- (c) *the sequence C is a path in π starting at $C_1 = \perp$ and the reservation invariant holds for C with respect to A .*

Proof. The formula Ψ is Σ_1^b because the existential quantifiers on C and A are bounded by π and items (a)-(c) are all checkable by PV functions and S_2^1 proves their basic properties. We can then proceed by induction on the parameter d , which corresponds to Length Induction over a Σ_1^b formula, available in S_2^1 .

For the base case, $d = 1$ and we can pick $C_1 = \perp$ and $\alpha_1 = *^N$. It is immediate to verify that the reservation invariant is satisfied, so item (c) holds.

Now, suppose the statement holds for d , and we prove it for $d + 1$. If $d + 1 > \text{depth}(\pi)$, then we are done. Otherwise, by induction hypothesis there exists a sequence $C = (C_1, \dots, C_d)$ and a series of restrictions $A = (\alpha_1, \dots, \alpha_d)$. If (a) or (b) hold for them, then we simply extend C with $C_{d+1} := \perp$ and $\alpha_{d+1} := *^N$ and either (a) or (b) will still hold.

If we are in case (c) but (a) and (b) failed, then α_d does not falsify any clause of $\text{REF}_s(\varphi)$ (as per item (iv) of the reservation invariant), and hence the clause was derived from some previous clause in π , either by weakening or by a resolution step. Here we simply move to one of the children in π and extend α_d into α_{d+1} following the reservation strategy in Algorithm 4.2. More precisely, we run one iteration of Algorithm 4.2 starting the traversal from C_d and taking α_d as the restriction kept in memory by the algorithm (this corresponds to Steps 2-4 of Algorithm 4.2). Crucially, there is a PV function that carries out this computation.

Let us analyze the outcome of this procedure. If the iteration of Algorithm 4.2 succeeds in extending the reservation from α_d to α_{d+1} , then by the way α_{d+1} is constructed from α_d and by the fact that the induction hypothesis guaranteed that α_d satisfies the invariant, we have that α_{d+1} will immediately satisfy the reservation invariant too. In this case, item (c) holds and we are done.

We argue that the reservation process cannot fail. If it did, it must be that the process failed at Step 3b or 3c of Algorithm 4.2.

- If the process failed in Step 3b, the algorithm attempted the reservation of a block at layer $1 \leq i < n$, but there were no free blocks left. This means that α_d already reserved at least $\lfloor s/n \rfloor - 1$ blocks on that layer, and so $\text{bw}(\alpha_d) \geq \lfloor s/n \rfloor - 1$, since each layer $i < n$ contains exactly $\lfloor s/n \rfloor$ blocks. By point

(ii) of the reservation invariant we have that $\text{bw}(\alpha_d) \leq 3 \text{bw}(C_d)$, so putting this together we have that C_d has block-width at least $1/3(\lfloor s/n \rfloor - 1)$, contradicting that we were not in case (a).

- If the process failed in Step 3c, this implies the reservation α_d had a clause $\mathcal{A}$ encoded in a block at layer n , but it failed to find a clause of φ that $\mathcal{A}$ was a weakening of. By point (iii) of the invariant, since the block is at layer n , $\mathcal{A}$ is a width- n clause, and by Lemma 6.5 in S_2^1 , $\neg \mathcal{A}$ encodes a satisfying assignment of φ , contradicting that we were not in case (b).

This completes the induction. $\square$

We are now ready to prove AM-WLB.

Lemma 6.8 (Formalized block-width lower bound in PV_1). *It holds that $PV_1 \vdash \text{AM-WLB}$.*

Proof. Working in S_2^1 , let π be a correct Resolution refutation of $\text{REF}_s(\varphi)$ for a CNF formula φ over n variables. We can apply Lemma 6.7 for $d = \text{depth}(\pi)$ to obtain a sequence of clauses $C = (C_1, \dots, C_d)$ of π and a sequence of restrictions $A = (\alpha_1, \dots, \alpha_d)$, satisfying one of conditions (a)-(c) in the statement. We claim that condition (c) cannot occur. Suppose it did. If C was really a path in π starting at the root $C_1 = \perp$, since π is a correct Resolution refutation this means the underlying graph of π is a DAG and therefore the clause C_d at depth d must be a leaf. It is not hard to see that S_2^1 can prove this. This implies that C_d is a clause of $\text{REF}_s(\varphi)$. Now, items (i) and (iv) of the reservation invariant in Definition 6.6 contradict each other: the restriction α_d must falsify C_d , which is a clause of $\text{REF}_s(\varphi)$; but item (iv) promised that α_d would not falsify any clause of the $\text{REF}_s(\varphi)$.

The viable option then is that we are in case (a) or (b). In case (a), some $C_i \in \pi$ that appears in the sequence $(C_1, \dots, C_d)$ has block-width at least $1/3(\lfloor s/n \rfloor - 1)$. In case (b), we immediately get a satisfying assignment for φ .

Since AM-WLB is a $\forall \Sigma_1^b$ sentence and S_2^1 is $\forall \Sigma_1^b$ -conservative over PV_1 , we get that $PV_1 \vdash \text{AM-WLB}$. $\square$

6.3 Formalization of the final lower bound statement

We are ready to combine the restriction argument and the block-width lower to get a size lower bound.

Theorem 6.9. *There exist a positive $\varepsilon \in \mathbb{Q}$ and $n_0 \in \mathbb{N}$ such that $PV_1 \vdash \text{AM}_{\varepsilon, n_0}$.*

Proof. Let $\varepsilon \in \mathbb{Q}$ and $n_0 \in \mathbb{N}$ be universal constants that can be computed from the rest of the argument. Working in PV_1 , let π be a Resolution refutation of $\text{REF}_s(\varphi)$ for a CNF formula φ over n variables, with $n \geq n_0$. We apply Lemma 6.4 to conclude that $PV_1 \vdash \text{AM-Restriction}_4$, from which we get in PV_1 that there is some restriction ρ such that $\pi_{\upharpoonright \rho}$ is a Resolution refutation of $\text{REF}_s(\varphi)_{\upharpoonright \rho}$ and the block-width of $\pi_{\upharpoonright \rho}$ is at most $4 \cdot \sqrt{s \log |\pi|}$. Furthermore, ρ is d -disabling for some $d \leq 2 \lceil \sqrt{s \log |\pi|} \rceil$.

Before proceeding, extend ρ into a restriction ρ' that further enables all blocks not touched by ρ and sets the values of pointers that are pointing at disabled blocks. In this way, $\text{REF}_s(\varphi)_{\upharpoonright \rho'}$ becomes exactly $\text{REF}_{s-d}(\varphi)$, as ρ was d -disabling, and $\pi_{\upharpoonright \rho'}$ is a refutation of $\text{REF}_{s-d}(\varphi)$.

Assume now that φ is unsatisfiable, or else we are already done. By Lemma 6.8 we have that $PV_1 \vdash \text{AM-WLB}$. Since we assume φ is unsatisfiable, applying this block-width lower bound to $\pi_{\upharpoonright \rho'}$ we conclude that there exists a clause in $\pi_{\upharpoonright \rho'}$ of block-width at least $1/3(\lfloor (s-d)/n \rfloor - 1)$. It then holds that

$$1/3(\lfloor (s-d)/n \rfloor - 1) \leq 4 \cdot \lceil \sqrt{s \log |\pi|} \rceil, \quad (6.16)$$

and it is not hard to see that PV_1 can derive from this inequality that $\|\pi\| > \varepsilon s/n^2$ for some small enough ε and large enough n . $\square$

7 Pudlák's upper bound in Resolution

The fact that the $\text{REF}_s(\varphi)$ formulas admit short Resolution refutations whenever φ is satisfiable is originally due to Pudlák [Pud03, Theorem 4.1]. However, technically speaking, our definition of REF is in the *relativized* form as used by Atserias and Müller, who crucially showed that the upper bound still works even in the presence of the disabling variables [AM20, Lemma 11], and the proof goes through for the encoding with pointer variables in binary too [dRGN+21, Lemma 2.1.i].

Our goal is to show that the upper bound construction can be proven correct in Resolution itself. The main technicality to overcome is that we want to show that from a satisfying assignment α to a CNF formula φ , one can construct not only a Resolution refutation of $\text{REF}(\varphi)$, but that this refutation can itself be encoded as a satisfying assignment to the formula $\text{REF}(\text{REF}(\varphi))$, and that the correctness of this can be certified in Resolution.

Let $P(\varphi, \alpha, s)$ denote the circuit that constructs a refutation of $\text{REF}_s(\varphi)$ given the satisfying assignment α . Intuitively, we want to derive a propositional formula that states

$$(\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s)) \rightarrow \text{REF}(\text{REF}_s(\varphi), \pi). \quad (7.1)$$

Our goal is to refute the negation of this formula in Resolution, which will not be a CNF formula. However, since Resolution is implicationally complete, the implication can be obtained by showing instead that there is a derivation of the form

$$\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s) \vdash_{\text{Res}} \text{REF}(\text{REF}_s(\varphi), \pi). \quad (7.2)$$

The main obstacle then will be to show that the circuit P can be presented in a simple way that can also be handled by Resolution.

In Section 7.1 we describe the construction in natural language, in a style that streamlines the previous existing proofs. In Section 7.2 we describe the circuit P , which turns out to be a very low depth circuit that Resolution will be able to reason about. Finally, Section 7.3 puts it together to derive the correctness of the construction in Resolution.

7.1 Description of the construction

The first step in the construction of the upper bound is to describe the conjunct $\pi = P(\varphi, \alpha, s)$ in the formula (7.2). The circuit P relates φ and α , which are variables of $\text{SAT}(\varphi, \alpha)$, to the variables π of $\text{REF}(\text{REF}_s(\varphi))$. Before we describe the circuit P , we give a natural language explanation of what the upper bound construction is doing.

We remark that in the following description, the formula φ and a satisfying assignment α have been fixed. This means that there are no longer any a-lit_ℓ^A variables encoding the formula (see Definition 2.8). To simplify notation, we sometimes view α as a function over literals so that $\alpha(x_i) = \alpha_i$ and $\alpha(\neg x_i) = \neg \alpha_i$.

General structure of the construction. The goal of the refutation is to derive, for every $B \in [s]$, the clause

$$\text{True}(B, \alpha) := \neg \text{enable}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B, \quad (\text{True}(B, \alpha))$$

encoding that if block B is enabled then it contains a clause that is satisfied by α . Each $\text{True}(B, \alpha)$ is derived from $\text{True}(C, \alpha)$ for all $C \in [B-1]$, and from $\text{True}(s, \alpha)$ one can easily derive the empty clause (by resolving with the unit clauses enable^s and $\neg \text{lit}_\ell^s$ for all ℓ such that $\alpha(\ell) = 1$).

We will derive the clause $\text{True}(B, \alpha)$ by first deriving the clauses

$$\text{derived}^B \vee \text{True}(B, \alpha) \quad \text{and} \quad \neg \text{derived}^B \vee \text{True}(B, \alpha) \quad (7.3)$$

and then applying one Resolution step.

The derivation of $\text{derived}^B \vee \text{True}(B, \alpha)$. For every $A \in [m]$ pick an arbitrary literal ℓ in clause A of φ which is made true by α . By weakening the axiom (REF-3, B, A, ℓ), which is $\neg \text{enable}^B \vee \neg \text{weak}_A^B \vee \text{lit}_\ell^B$ since the variables $a\text{-lit}_\ell^A$ are no longer present once a formula has been fixed, we obtain the clause

$$\neg \text{enable}^B \vee \neg \text{weak}_A^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B. \quad (L_1(B, A))$$

Now cut successively (REF-7, B) with $(L_1(B, A))$ for $A \in [m]$ to get $\text{derived}^B \vee \text{True}(B, \alpha)$. That is, there will be m lines $L_2(B, 1)$ to $L_2(B, m)$, each of the form

$$\neg \text{enable}^B \vee \text{derived}^B \vee \bigvee_{i=A+1}^m \text{weak}_i^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B, \quad (L_2(B, A))$$

such that $L_2(B, A+1)$ is obtained by resolving $L_2(B, A)$ with $L_1(B, A+1)$ over variable weak_{A+1}^B . Note that $L_2(B, m)$ is precisely $\text{derived}^B \vee \text{True}(B, \alpha)$.

It is not hard to see that, for each $B \in [s]$, this derivation of $\text{derived}^B \vee \text{True}(B, \alpha)$ consists of $\Theta(m)$ resolution steps.

The derivation of $\neg \text{derived}^B \vee \text{True}(B, \alpha)$. We assume that for every $C \in [B-1]$ we have derived $\text{True}(C, \alpha)$. We first carry out the following derivation for every $C \in [B-1]$ and $i \in [n]$, leading to auxiliary clauses $(R_2(B, C, i))$ defined below.

Let us suppose that $\alpha(x_i) = 0$ (the dual case is analogous but following the right-hand side pointers). For $j \in [n]$, let $\ell_j = x_j$ if $\alpha(x_j) = 1$ and $\ell_j = \neg x_j$ if $\alpha(x_j) = 0$. Note that for each $j \in [n]$ we have the axiom (REF-1, B, C, i, ℓ_j), which is

$$\neg \text{enable}^B \vee \text{res}_{x_i}^B \vee \neg \text{lpoint}_C^B \vee \neg \text{lit}_{\ell_j}^C \vee \text{lit}_{\ell_j}^B. \quad (R_1^A(B, C, i, j))$$

Successively resolving $\text{True}(C, \alpha)$ with $(R_1^A(B, C, i, j))$ over variable $\text{lit}_{\ell_j}^C$ for $j \in [n]$ we get

$$\neg \text{enable}^B \vee \neg \text{enable}^C \vee \neg \text{res}_{x_i}^B \vee \neg \text{lpoint}_C^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B. \quad (R_1(B, C, i))$$

This consists of n lines, which we refer to as $R_1(B, C, i, j)$ for $j \in [n]$. Note that $R_1(B, C, i, n)$ refers to the same line $(R_1(B, C, i))$. We cut $(R_1(B, C, i))$ over variable enable^C with the clause $R_2^A(B, C, i) := \neg \text{enable}^B \vee \neg \text{lpoint}_C^B \vee \text{enable}^C$, which is (REF-8, B, C), to obtain

$$\neg \text{enable}^B \vee \neg \text{res}_{x_i}^B \vee \neg \text{lpoint}_C^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B. \quad (R_2(B, C, i))$$

We now cut (REF-4, B) with $(R_2(B, C, i))$ over variable $\text{res}_{x_i}^B$ for every $i \in [n]$ to get

$$\neg \text{enable}^B \vee \neg \text{derived}^B \vee \neg \text{lpoint}_C^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B. \quad (R_3(B, C, i))$$

Finally cutting over the variable lpoint_C^B the clause $R_3^A(B, i) := \neg \text{enable}^B \vee \neg \text{derived}^B \vee \bigvee_{C \in [B-1]} \text{lpoint}_C^B$, which is (REF-5, B), with $(R_3(B, C, i))$ for each $C \in [B-1]$, we get

$$\neg \text{enable}^B \vee \neg \text{derived}^B \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B, \quad (R_4(B))$$

which is exactly $\neg \text{derived}^B \vee \text{True}(B, \alpha)$. This step consists of $B-1$ lines, which we denote by $R_4(B, C, i)$ for $C \in [B-1]$, where the last line $R_4(B, B-1, i)$ is $(R_4(B))$.

We note that, for each $B \in [s]$, we derive $\neg \text{derived}^B \vee \text{True}(B, \alpha)$ in $\Theta(sn^2)$ resolution steps.

Contradiction from True(s). Once we have derived $\text{True}(s)$, we resolve it with (REF-11) to get

$$\bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B, \quad (7.4)$$

which we then resolve with the axioms (REF-10, ℓ) for the n literals ℓ such that $\alpha(\ell) = 1$ to get the empty clause.

We observe that the total number of clauses in this resolution refutation of $\text{REF}_s(\varphi)$, for a satisfiable formula φ , is $\Theta(s(m + sn^2))$.

7.2 The construction as a low-depth circuit

The expression $\pi = P(\varphi, \alpha, s)$ in the formula (7.2) is a short-hand for a conjunction of clauses describing the upper bound construction for refuting $\text{REF}_s(\varphi)$ as presented in Section 7.1. As noted above, the number of clauses in this refutation is $\tau = \Theta(s(m + sn^2))$, where n is the number of variables and m is the number of clauses of φ . We only define $\pi = P(\varphi, \alpha, s)$ for π being a proof of length $t \geq \tau$, as we will only consider this short-hand in this parameter regime. This subsection is dedicated to the description of the conjunction of clauses encoding $\pi = P(\varphi, \alpha, s)$. Each clause C that appears in the construction, including the axioms of REF that are used in cuts, is mapped to some $\mathcal{B} \in [\tau]$ which corresponds to its position in the proof. Since this is a one-to-one mapping, we sometimes abuse notation and identify C with $\mathcal{B}$.

The key observation is that each variable of π is a function of at most two variables of φ and α , and thus the construction can be expressed as a depth-2 circuit. This is because if step $\mathcal{B}$ of the construction is, say, to derive a clause C (over variables of $\text{REF}_s(\varphi)$), then we will add the unit clauses

$$\text{enable}^{\mathcal{B}} \wedge \bigwedge_{z \in C} \text{lit}_z^{\mathcal{B}} \wedge \bigwedge_{z \notin C} \neg \text{lit}_z^{\mathcal{B}}, \quad (7.5)$$

encoding that clause C is the clause at step $\mathcal{B}$ in the proof. If the construction obtains this clause by a Resolution step, we add the conjunct $\text{derived}^{\mathcal{B}}$ and if it is by a weakening of an axiom, we add the conjunct $\neg \text{derived}^{\mathcal{B}}$. Similarly, we include appropriate unit clauses of the variables of type lpoint , rpoint and weak . For the most part, the construction is a ready-made template that only depends on α and φ in a few crucial points. In this case, we will include clauses that encode this dependence. For example, suppose that at

step $\mathcal{B}$ we derive the clause

$$\bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_\ell^B \quad (7.6)$$

that does depend on α . Here B is a block of the inner REF formula, while this clause itself will be instantiated as a block $\mathcal{B}$ of the outer REF. We then add, for $i \in [n]$, the clauses encoding

$$\text{lit}_{\text{lit}_B^{\mathcal{B}}}^{\mathcal{B}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{lit}_{\text{lit}_{\neg x_i}^B}^{\mathcal{B}} \leftrightarrow \neg \alpha_i. \quad (7.7)$$

We first describe in more detail how the variables related to the lines $(L_1(B, A))$ and $(L_2(B, A))$ are defined. Line $(L_1(B, A))$ is the only one that depends on both α and φ ; the rest of the construction depends solely on α . For subsequent lines in the construction, we simply describe the parts that depend on α as the others are completely determined by the ready-made template, as explained above.

Fix $A \in [m]$, and let $\mathcal{L} \in [t]$ denote the block corresponding to $(L_1(B, A))$. We include the unit clauses

$$\text{enable}^{\mathcal{L}} \wedge \neg \text{derived}^{\mathcal{L}} \quad (7.8)$$

determining that this line is enabled and is not derived. We also include unit clauses $\neg \text{res}_z^{\mathcal{L}}$, $\neg \text{lpoint}_{\mathcal{B}}^{\mathcal{L}}$ and $\neg \text{rpoint}_{\mathcal{B}}^{\mathcal{L}}$ for all $\mathcal{B} \in [t]$ (since $\mathcal{L}$ is not obtained by a Resolution step). We set the lit variables as explained above, that is, we have clauses encoding, for $i \in [n]$,

$$\text{lit}_{\text{lit}_B^{\mathcal{L}}}^{\mathcal{L}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{lit}_{\text{lit}_{\neg x_i}^B}^{\mathcal{L}} \leftrightarrow \neg \alpha_i, \quad (7.9)$$

the unit clauses

$$\text{lit}_{\neg \text{enable}^B}^{\mathcal{L}} \wedge \text{lit}_{\neg \text{weak}_A^B}^{\mathcal{L}}, \quad (7.10)$$

and the unit clauses $\neg \text{lit}_{\ell'}^{\mathcal{L}}$ encoding that no other literal ℓ' (not appearing above) is present in the clause. It remains to define the $\text{weak}^{\mathcal{L}}$ variables. Note that $(L_1(B, A))$ can be obtained by weakening any of the axioms (REF-3, B, A, ℓ), for any ℓ that appears in clause A of φ and that is made true by α . We therefore set, for every $i \in [n]$,

$$\text{weak}_{(\text{REF-3}, B, A, x_i)}^{\mathcal{L}} \leftrightarrow \text{a-lit}_{x_i}^A \wedge \alpha_i \quad \text{and} \quad \text{weak}_{(\text{REF-3}, B, A, \neg x_i)}^{\mathcal{L}} \leftrightarrow \text{a-lit}_{\neg x_i}^A \wedge \neg \alpha_i. \quad (7.11)$$

Here the variables $\text{a-lit}_{x_i}^A$ and $\text{a-lit}_{\neg x_i}^A$ are part of the $\text{SAT}(\varphi, \alpha)$ formula (as in Definition 2.8), and determine which literals appear in what clauses of φ . Observe that we are technically establishing that the clause $(L_1(B, A))$ will have many weakening pointers. While this is somewhat unusual in real Resolution refutations, the refutation is still sound and none of the axioms in Definition 2.4 are violated. The advantage is that we do not have to establish which is the, say, first satisfied literal of every axiom. Even though (potentially) having multiple weakening pointers is not strictly necessary, it simplifies the clauses needed to express $\pi = P(\varphi, \alpha, s)$.

Now fix $A \in [m]$ and let $\mathcal{L}$ be the block corresponding to $(L_2(B, A))$. We add the unit clauses

$$\text{enable}^{\mathcal{L}} \wedge \text{derived}^{\mathcal{L}} \wedge \text{res}_{\text{weak}_A^B}^{\mathcal{L}} \wedge \bigwedge_{z \neq \text{weak}_A^B} \neg \text{res}_z^{\mathcal{L}} \quad (7.12)$$

that determine that this line is enabled, it is derived and obtained by resolving over variable weak_A^B . We also include unit clauses enforcing all $\text{weak}^{\mathcal{L}}$ variables to be 0 (since $\mathcal{L}$ is not a weakening of any axiom).

The $\text{lit}^{\mathcal{L}}$ variables are encoded as explained above. As for the pointers, we include the unit clauses

$$\text{rpoint}_{L_1(B,A)}^{\mathcal{L}} \wedge \text{lpoint}_{L_2(B,A-1)}^{\mathcal{L}} \wedge \bigwedge_{\mathcal{B} \neq L_1(B,A)} \neg \text{rpoint}_{\mathcal{B}}^{\mathcal{L}} \wedge \bigwedge_{\mathcal{B} \neq L_2(B,A-1)} \neg \text{lpoint}_{\mathcal{B}}^{\mathcal{L}}. \quad (7.13)$$

For the second set of clauses, used in the derivation of $\neg \text{derived}^B \vee \text{True}(B, \alpha)$, we describe the parts that depend on α . These can be split into two groups, the parts that depend only on whether we consider lpoint or rpoint , that is, only on α_i , and those that depend also on ℓ_j . We start with the former. If in the construction we described above a clause on a line corresponding to, say, block $\mathcal{R}$ contains the literal $\neg \text{lpoint}_C^B$ for the case when $\alpha_i = 0$, then we include the clauses encoding

$$\text{lit}_{\neg \text{rpoint}_C^B}^{\mathcal{R}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{lit}_{\neg \text{lpoint}_C^B}^{\mathcal{R}} \leftrightarrow \neg \alpha_i, \quad (7.14)$$

so that the literal $\neg \text{rpoint}_C^B$ is present iff $\alpha(x_i) = 1$ and $\neg \text{lpoint}_C^B$ is present if and only if $\alpha(x_i) = 0$. Similarly if it contains the literal lpoint_C^B , we include $\text{lit}_{\text{rpoint}_C^B}^{\mathcal{R}} \leftrightarrow \alpha_i$ and $\text{lit}_{\text{lpoint}_C^B}^{\mathcal{R}} \leftrightarrow \neg \alpha_i$. For $\mathcal{R} = R_4(B, C, i)$ (one of the intermediate steps to derive $R_4(B)$), we include clauses

$$\text{res}_{\text{rpoint}_C^B}^{\mathcal{R}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{res}_{\text{lpoint}_C^B}^{\mathcal{R}} \leftrightarrow \neg \alpha_i, \quad (7.15)$$

encoding that the resolved variable is either rpoint_C^B or lpoint_C^B , depending on α_i . For $\mathcal{R} = R_2^A(B, C, i)$ we include

$$\text{weak}_{(\text{REF-9}, B, C)}^{\mathcal{R}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{weak}_{(\text{REF-8}, B, C)}^{\mathcal{R}} \leftrightarrow \neg \alpha_i, \quad (7.16)$$

and for $\mathcal{R} = R_3^A(B, i)$ we include

$$\text{weak}_{(\text{REF-6}, B)}^{\mathcal{R}} \leftrightarrow \alpha_i \quad \text{and} \quad \text{weak}_{(\text{REF-5}, B)}^{\mathcal{R}} \leftrightarrow \neg \alpha_i, \quad (7.17)$$

encoding, in each case, which axiom $\mathcal{R}$ is a weakening of.

We now move to the parts that (also) depend on ℓ_j . If the line corresponding to block $\mathcal{R}$ contains $\text{lit}_{\ell_j}^{B'}$ for some block B' we include

$$\text{lit}_{\text{lit}_{x_j}^{B'}}^{\mathcal{R}} \leftrightarrow \alpha_j \quad \text{and} \quad \text{lit}_{\text{lit}_{\neg x_j}^{B'}}^{\mathcal{R}} \leftrightarrow \neg \alpha_j. \quad (7.18)$$

Similarly, if it contains $\neg \text{lit}_{\ell_j}^{B'}$ for some block B' we include $\text{lit}_{\neg \text{lit}_{x_j}^{B'}}^{\mathcal{R}} \leftrightarrow \alpha_j$ and $\text{lit}_{\neg \text{lit}_{\neg x_j}^{B'}}^{\mathcal{R}} \leftrightarrow \neg \alpha_j$. If $\mathcal{R}$ is a line $R_1(B, C, i, j)$ (one of the intermediate steps to derive $R_1(B, C, i)$) that is obtained by resolving over variable $\text{lit}_{\ell_j}^C$, we include the clauses

$$\text{res}_{\text{lit}_{x_j}^C}^{\mathcal{R}} \leftrightarrow \alpha_j \quad \text{and} \quad \text{res}_{\text{lit}_{\neg x_j}^C}^{\mathcal{R}} \leftrightarrow \neg \alpha_j. \quad (7.19)$$

Finally, for $\mathcal{R}$ being $R_1^A(B, C, i, j)$, we include

$$\text{weak}_{(\text{REF-2}, B, C, i, x_j)}^{\mathcal{R}} \leftrightarrow \alpha_i \wedge \alpha_j, \quad \text{weak}_{(\text{REF-1}, B, C, i, x_j)}^{\mathcal{R}} \leftrightarrow \neg \alpha_i \wedge \alpha_j, \quad (7.20)$$

$$\text{weak}_{(\text{REF-2}, B, C, i, \neg x_j)}^{\mathcal{R}} \leftrightarrow \alpha_i \wedge \neg \alpha_j, \quad \text{and} \quad \text{weak}_{(\text{REF-1}, B, C, i, \neg x_j)}^{\mathcal{R}} \leftrightarrow \neg \alpha_i \wedge \neg \alpha_j. \quad (7.21)$$

We have thus far described how we assign variable referring to blocks $\mathcal{B}$ of π for $\mathcal{B} \leq \tau$ (where we recall $\tau = \Theta(s(m + sn^2))$ is the number of clauses in the refutation of $\text{REF}_s(\varphi)$ described in Section 7.1). If $t > \tau$, we can extend the construction easily by copying all clauses on block τ to the last block t (to make

sure we satisfy that the last block is enabled and contains the empty clause, and is derived from the correct previous clauses) and for all other $\mathcal{B} \in [t - 1]$ with $\mathcal{B} > \tau$ we include unit clauses setting all variables to 0, in particular, we include clauses $\neg \text{enable}^{\mathcal{B}}$ encoding that these blocks are not used in the refutations described by $P(\varphi, \alpha, s)$.

We denote by $\pi = P(\varphi, \alpha, s)$ all the conjuncts describing the refutation of $\text{REF}_s(\varphi)$ as an assignment to the outer REF formula.

7.3 Correctness of the construction in Resolution

Before we formally show that the construction presented above is provably correct in Resolution, we need to explicitly describe the formula $\text{REF}_t(\text{REF}_s(\varphi), \pi)$. This formula states that π is a valid Resolution refutation of the formula $\text{REF}_s(\varphi)$, stating that φ has a size- s Resolution refutation. Recall that the variables of a formula $\text{REF}_t(\psi, \pi)$ are the variables describing π and those describing ψ . When $\psi = \text{REF}_s(\varphi)$, most of the variables of ψ are already set, and the only remaining ones are the variables describing φ . These variables determine which clauses of $\text{REF}_s(\varphi)$ exist, that is, they indicate which of the (REF-3) axioms are present in $\text{REF}_s(\varphi)$. The other axioms describing $\text{REF}_s(\varphi)$ (see Definition 2.4) do not depend on φ .

Recall that the formula φ has n variables and m clauses, and that the literals that appear in each clause are determined by the variables a-lit, that is, a-lit_ℓ^A is set to 1 if the literal ℓ appears in clause A of φ . For every literal $\ell \in \text{Lit}_n$ and for every $A \in [m]$, if a-lit_ℓ^A is set to 1 then the axiom $(\text{enable}^B \wedge \text{weak}_A^B) \rightarrow \text{lit}_\ell^B$, which is (REF-3) with $\text{a-lit}_\ell^A = 1$, appears in $\text{REF}_s(\varphi)$, and if a-lit_ℓ^A is set to 0 then it does not appear in $\text{REF}_s(\varphi)$.

In order to write the propositional formula $\text{REF}_t(\text{REF}_s(\varphi), \pi)$, we need to know how many clauses appear in $\text{REF}_s(\varphi)$. This, in turn, depends on φ . To deal with this, we include all potential axioms $(\text{enable}^B \wedge \text{weak}_A^B) \rightarrow \text{lit}_\ell^B$, and we let the variables a-lit deactivate the axiom that should not exist, which can be done by enforcing that no block in π can be a weakening of that axiom. Formally, let $(\text{REF-3}, B, A, \ell)$ indicate the position in $\text{REF}_s(\varphi)$ of the potential (REF-3) axiom defined over B, A and ℓ . For every block $\mathcal{B}$ of π , for every $B \in [s]$, $A \in [m]$ and $\ell \in \text{Lit}_n$, we include in $\text{REF}_t(\text{REF}_s(\varphi), \pi)$ the clause $\text{a-lit}_\ell^A \vee \neg \text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{B}}$.

We are now ready to show that the construction is provably correct in Resolution.

Theorem 7.1 (Pudlák's upper bound in Resolution). *For every $n, m, s \in \mathbb{N}$, there is some $\tau = \Theta(s(m + sn^2))$ such that for $t \geq \tau$ there exist uniform polynomial-size Resolution derivations of the form*

$$\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s) \vdash_{\text{Res}} \text{REF}_t(\text{REF}_s(\varphi), \pi),$$

where the SAT formula is for CNF formulas with n variables and m clauses, and $\pi = P(\varphi, \alpha, s)$ stands for the conjunction of clauses describing the upper bound construction, as per Section 7.2. Moreover, the derivations can be described uniformly in polynomial time.

Proof. Let $\tau = \Theta(s(m + sn^2))$ be the number of clauses in the refutation of $\text{REF}_s(\varphi)$ described in Section 7.1, and let $\mathfrak{C}$ denote the set of clauses in the formula $\text{REF}_t(\text{REF}_s(\varphi), \pi)$. The clauses $\pi = P(\varphi, \alpha, s)$ encode the correct computation of the construction, meaning that if $\text{SAT}(\varphi, \alpha)$ holds, then indeed π encodes a correct Resolution refutation, so all clauses $c \in \mathfrak{C}$ are sound inferences: for every $c \in \mathfrak{C}$, we have $\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s) \models c$. We only need to verify that there are efficient derivations in Resolution for each $c \in \mathfrak{C}$.

Observe that the clauses of the form $\text{a-lit}_\ell^A \vee \neg \text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{B}}$ are either present in $\pi = P(\varphi, \alpha, s)$ (by 7.11), if $\mathcal{B}$ denotes the block corresponding to $(L_1(B, A))$, or otherwise can be obtained by weakening the unit clause $\neg \text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{B}}$, which in this case is part of $\pi = P(\varphi, \alpha, s)$. For all other clauses $c \in \mathfrak{C}$, we argue that it is efficiently derivable in Resolution by considering different cases, according to which type of axiom (REF-1 - REF-11) it is, as well as to the block $\mathcal{B} \in [t]$ it refers to, where we say that a clause

$c \in \mathfrak{C}$ refers to block $\mathcal{B} \in [t]$ if the enable variable at the beginning of the clause is $\text{enable}^{\mathcal{B}}$ (and clauses (REF-10) and (REF-11) refer to the root t). For most cases, we show that Resolution can obtain c from only $\pi = P(\varphi, \alpha, s)$ (namely, the clauses in $\text{SAT}(\varphi, \alpha)$ are not necessary for the derivation), either by weakening a clause or by a constant-size derivation. There are only some clauses c of type (REF-7) which are not semantically implied by $\pi = P(\varphi, \alpha, s)$ —these, we argue, can be derived from $\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s)$ in Resolution in size $O(n)$.

Recall that the clauses in $\pi = P(\varphi, \alpha, s)$ are such that each variable of π is a function of at most two variables of φ and α . By inspecting the description of the refutation as a circuit in Section 7.2, it is clear that the only variables of π that depend on both a-lit and α variables are precisely the weak $^{\mathcal{L}}_{(\text{REF-3}, \mathcal{B}, A, \ell)}$ variables in line (7.11). All other variables are either enforced to be true or false by unit clauses or depend only on α variables. We first argue that for all clauses $c \in \mathfrak{C}$ that do not contain the weak $^{\mathcal{L}}_{(\text{REF-3}, \mathcal{B}, A, \ell)}$ variables in line (7.11), it holds that $\pi = P(\varphi, \alpha, s) \vdash_{\text{Res}} c$. In these cases, the argument is very straightforward, and c will follow by weakening or by one application of the Resolution rule followed by weakening.

Before we split into cases according to the axiom type of the clause c , we make one simple observation regarding the blocks $\mathcal{B} \in [t]$ which are larger than τ . Recall that the output of P is a refutation of size τ , but we may set t to be a larger value. In this case, all non-root blocks larger than τ will simply be disabled. This implies that any clause c that refers to a block $\mathcal{B}$ such that $\tau < \mathcal{B} < t$, it holds that c contains the literal $\neg \text{enable}^{\mathcal{B}}$ and therefore can be obtained by weakening the unit clause $\neg \text{enable}^{\mathcal{B}}$, which will appear in $\pi = P(\varphi, \alpha, s)$. We therefore assume $\mathcal{B} \leq \tau$ or $\mathcal{B} = t$ (in which case $\pi = P(\varphi, \alpha, s)$ contains the unit clause $\text{enable}^{\mathcal{B}}$).

If c is a clause (REF-10) or (REF-11): that is, either

$$\neg \text{lit}_{\ell}^t \quad \text{or} \quad \text{enable}^t,$$

then these clauses are present in $\pi = P(\varphi, \alpha, s)$. Indeed, since the τ -th clause of the refutation of $\text{REF}_s(\varphi)$ as presented in Section 7.1 is the empty clause, by (7.5) we have that the clauses $\neg \text{lit}_{\ell}^{\tau}$ and enable^{τ} are present in $\pi = P(\varphi, \alpha, s)$, and therefore so are $\neg \text{lit}_{\ell}^t$ and enable^t , since we copied such clauses for t .

If c is a clause (REF-5) or (REF-6): that is, either

$$\left(\text{enable}^{\mathcal{B}} \wedge \text{derived}^{\mathcal{B}} \right) \rightarrow \bigvee_{\substack{\mathcal{B}' \in [s] \\ \mathcal{B}' < \mathcal{B}}} \text{lpoint}_{\mathcal{B}'}^{\mathcal{B}} \quad \text{or} \quad \left(\text{enable}^{\mathcal{B}} \wedge \text{derived}^{\mathcal{B}} \right) \rightarrow \bigvee_{\substack{\mathcal{B}' \in [s] \\ \mathcal{B}' < \mathcal{B}}} \text{rpoint}_{\mathcal{B}'}^{\mathcal{B}},$$

then we have two cases. If $\mathcal{B}$ corresponds to a clause obtained from a weakening of an axiom, then we have that the unit clause $\neg \text{derived}^{\mathcal{B}}$ is in $\pi = P(\varphi, \alpha, s)$ (by (7.8)), and therefore the clause c can be derived by weakening. If $\mathcal{B}$ corresponds to a clause obtained from a derivation step, then $\pi = P(\varphi, \alpha, s)$ contains unit clauses $\text{lpoint}_{\mathcal{B}'}^{\mathcal{B}}$ and $\text{rpoint}_{\mathcal{B}'}^{\mathcal{B}}$ for some $\mathcal{B}'$ (by (7.13)) and we again conclude that c can be derived by weakening.

If c is a clause (REF-4): that is,

$$\left(\text{enable}^{\mathcal{B}} \wedge \text{derived}^{\mathcal{B}} \right) \rightarrow \bigvee_x \text{res}_x^{\mathcal{B}},$$

we have a similar case. Either $\mathcal{B}$ was not obtained by a derivation (and the clause $\neg \text{derived}^{\mathcal{B}}$ is in $\pi = P(\varphi, \alpha, s)$), or there is an x for which the unit clause $\text{res}_x^{\mathcal{B}}$ is present in $\pi = P(\varphi, \alpha, s)$ or can be derived from two clauses $\text{res}_x^{\mathcal{B}} \vee \alpha_i$ and $\text{res}_x^{\mathcal{B}} \vee \neg \alpha_i$ which are in $\pi = P(\varphi, \alpha, s)$ (by (7.15)). The clause c can then be obtained by weakening either $\neg \text{derived}^{\mathcal{B}}$ or $\text{res}_x^{\mathcal{B}}$.

If c is a clause **(REF-8)** or **(REF-9)**: that is, either

$$\left(\text{enable}^{\mathcal{B}} \wedge \text{lpoint}_{\mathcal{B}'}^{\mathcal{B}} \right) \rightarrow \text{enable}^{\mathcal{B}'} \quad \text{or} \quad \left(\text{enable}^{\mathcal{B}} \wedge \text{rpoint}_{\mathcal{B}'}^{\mathcal{B}} \right) \rightarrow \text{enable}^{\mathcal{B}'},$$

then either $\mathcal{B}$ was obtained by a derivation step involving the clause $\mathcal{B}'$, in which case $\pi = P(\varphi, \alpha, s)$ contains the unit clause $\text{enable}^{\mathcal{B}'}$, or it was not, in which case $\pi = P(\varphi, \alpha, s)$ contains the unit clauses $\neg \text{lpoint}_{\mathcal{B}'}^{\mathcal{B}}$ and $\neg \text{rpoint}_{\mathcal{B}'}^{\mathcal{B}}$. Either way, c can be derived by a weakening step.

If c is a clause **(REF-1)** or **(REF-2)**: that is, either

$$\left(\text{enable}^{\mathcal{B}} \wedge \text{res}_{x_i}^{\mathcal{B}} \wedge \text{lpoint}_{\mathcal{B}'}^{\mathcal{B}} \wedge \text{lit}_{\ell}^{\mathcal{B}'} \right) \rightarrow \text{lit}_{\ell}^{\mathcal{B}} \quad \text{or} \quad \left(\text{enable}^{\mathcal{B}} \wedge \text{res}_{x_i}^{\mathcal{B}} \wedge \text{rpoint}_{\mathcal{B}'}^{\mathcal{B}} \wedge \text{lit}_{\ell}^{\mathcal{B}'} \right) \rightarrow \text{lit}_{\ell}^{\mathcal{B}},$$

then, in the only case when c cannot be obtained by weakening, it can be derived by first resolving over α_i the clauses $\neg \text{lit}_{\ell}^{\mathcal{B}'} \vee \alpha_i$ and $\text{lit}_{\ell}^{\mathcal{B}} \vee \neg \alpha_i$ (or $\neg \text{lit}_{\ell}^{\mathcal{B}'} \vee \neg \alpha_i$ and $\text{lit}_{\ell}^{\mathcal{B}} \vee \alpha_i$), to obtain $\neg \text{lit}_{\ell}^{\mathcal{B}'} \vee \text{lit}_{\ell}^{\mathcal{B}}$, and then weakening.

If c is a clause **(REF-3)** that does not contain $\text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{L}}$: that is,

$$\left(\text{enable}^{\mathcal{B}} \wedge \text{weak}_A^{\mathcal{B}} \wedge \text{a-lit}_{\ell}^A \right) \rightarrow \text{lit}_{\ell}^{\mathcal{B}},$$

then either it can be obtained by weakening, or it can be derived by resolving a clause encoding $\text{lit}_{\ell}^{\mathcal{B}} \leftrightarrow \alpha$ or $\text{lit}_{\ell}^{\mathcal{B}} \leftrightarrow \neg \alpha$ of the type (7.14) with a clause encoding $\text{weak}_A^{\mathcal{B}} \leftrightarrow \alpha_i$ or $\text{weak}_A^{\mathcal{B}} \leftrightarrow \neg \alpha_i$ of the type (7.16) or (7.17), and then weakening.

If c is a clause **(REF-7)** that does not contain $\text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{L}}$: that is,

$$\left(\text{enable}^{\mathcal{B}} \wedge \neg \text{derived}^{\mathcal{B}} \right) \rightarrow \bigvee_{A \in [m]} \text{weak}_A^{\mathcal{B}},$$

then either it can be obtained by weakening, or it can be obtained by resolving a clause encoding $\text{weak}_A^{\mathcal{B}} \leftrightarrow \alpha_i$ with a clause encoding $\text{weak}_A^{\mathcal{B}} \leftrightarrow \neg \alpha_i$, for two clauses of the type (7.16) or of the type (7.17), and then weakening.

It remains to argue that if c is one of the axiom **(REF-3)** or **(REF-7)** that contains the variable $\text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{L}}$ from (7.11), Resolution can derive c efficiently from $\text{SAT}(\varphi, \alpha) \wedge \pi = P(\varphi, \alpha, s)$. We first argue that Resolution can efficiently derive axioms c of type **(REF-3)**. Observe that the only case when c cannot be obtained by weakening is when c is

$$\left(\text{enable}^{\mathcal{L}} \wedge \text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{L}} \right) \rightarrow \text{lit}_{\ell'}^{\mathcal{L}}, \quad (7.22)$$

for $\ell' = \text{lit}_{\ell}^{\mathcal{B}}$. Indeed, $\text{a-lit}_{\ell'}^{(\text{REF-3}, B, A, \ell)} = 0$ for all ℓ' that are not $\neg \text{enable}^{\mathcal{B}}$, $\neg \text{weak}_A^{\mathcal{B}}$ or $\text{lit}_{\ell}^{\mathcal{B}}$ (since the axiom **(REF-3), B, A, ℓ** without the a-lit_{ℓ}^A variable is $(\text{enable}^{\mathcal{B}} \wedge \text{weak}_A^{\mathcal{B}}) \rightarrow \text{lit}_{\ell}^{\mathcal{B}}$) and $\pi = P(\varphi, \alpha, s)$ contains the clauses $\text{lit}_{\neg \text{enable}^{\mathcal{B}}}^{\mathcal{L}}$ and $\text{lit}_{\neg \text{weak}_A^{\mathcal{B}}}^{\mathcal{L}}$, since $\mathcal{L}$ corresponds to clause $(L_1(B, A))$, that is,

$$\neg \text{enable}^{\mathcal{B}} \vee \neg \text{weak}_A^{\mathcal{B}} \vee \bigvee_{\substack{\ell \in \text{Lit}_n \\ \alpha(\ell)=1}} \text{lit}_{\ell}^{\mathcal{B}}. \quad (7.23)$$

Now when $\ell' = \text{lit}_{\ell}^{\mathcal{B}}$, we can derive the clause $\neg \text{weak}_{(\text{REF-3}, B, A, \ell)}^{\mathcal{L}} \vee \text{lit}_{\ell'}^{\mathcal{L}}$ by resolving a clause of (7.7) with a clause of (7.11), and then (7.23) can be obtained by weakening.

Finally, we argue that if c is one of the axiom (REF-7) that depend on (7.11), Resolution can derive it in $O(n)$ steps. Recall that, by (7.11), for all $i \in [n]$ and $A \in [m]$, the formula $\pi = P(\varphi, \alpha, s)$ contains the clause

$$\text{weak}_{(\text{REF-3}, B, A, x_j)}^{\mathcal{L}} \vee \neg \text{a-lit}_{x_j}^A \vee \neg \alpha_j \quad (7.24)$$

and the clause

$$\text{weak}_{(\text{REF-3}, B, A, \neg x_j)}^{\mathcal{L}} \vee \neg \text{a-lit}_{\neg x_j}^A \vee \alpha_j. \quad (7.25)$$

By resolving the clause (7.24) with (SAT-1, A, x_j) and then with (SAT-2, A, j), we obtain the clause

$$\text{weak}_{(\text{REF-3}, B, A, x_i)}^{\mathcal{L}} \vee \text{sat}_{x_i}^A. \quad (7.26)$$

Similarly, we can derive

$$\text{weak}_{(\text{REF-3}, B, A, x_i)}^{\mathcal{L}} \vee \text{sat}_{\neg x_i}^A \quad (7.27)$$

by resolving (7.25) with (SAT-1, $A, \neg x_i$) and then with (SAT-3, A, i). Resolving (SAT-4) with each (7.26) and (7.27) for all $i \in [n]$, we obtain

$$\bigvee_{i \in [n]} \left(\text{weak}_{(\text{REF-3}, B, A, x_i)}^{\mathcal{L}} \vee \text{weak}_{(\text{REF-3}, B, A, \neg x_i)}^{\mathcal{L}} \right), \quad (7.28)$$

from which c can be derived by weakening. This completes the proof. $\square$

8 Extended Frege proves that automating Resolution is NP-hard

We now have the lower bound (Theorem 6.9) needed to show the NP-hardness of automatability formalized in PV_1 . This is a $\forall \Sigma_1^b$ sentence, meaning that the existential quantifiers can be witnessed by polynomial-time functions and via Cook's translation turned into propositional formulas with short Extended Frege proofs.

Theorem 8.1 (Extraction algorithm in EF). *There exists a uniform family of polynomial-size circuits $\{E_{n,m,s,t}\}_{n,m,s,t \in \mathbb{N}}$ such that for every CNF formula φ over n variables and m clauses, if φ is satisfiable and π is a Resolution refutation of $\text{REF}_s(\varphi)$ for $s \geq n^3$, then the circuit $E_{n,m,s,t}(\varphi, \pi)$ outputs a satisfying assignment for φ . Furthermore, this is provable in Extended Frege, that is, there exists a polynomial $\ell(n, m, s, t)$ such that*

$$\text{EF} \vdash_{\ell(n,m,s,t)} \text{REF}_t(\text{REF}_s(\varphi), \pi) \rightarrow \text{SAT}(\varphi, E_{n,m,s,t}(\varphi, \pi)),$$

and these proofs can be generated uniformly in polynomial time.

Proof. By Theorem 6.9, we know there exists a positive $\varepsilon \in \mathbb{Q}$ and $n_0 \in \mathbb{N}$ such that $\text{PV}_1 \vdash \text{AM}_{\varepsilon, n_0}$, the first-order statement encoding the lower bound on REF formulas ($\text{AM}_{\varepsilon, n_0}$). Since $\text{AM}_{\varepsilon, n_0}$ is a $\forall \Sigma_1^b$ formula, by Buss's witnessing theorem (Theorem 2.2), there exists a PV function E_0 such that

$$\begin{aligned} \text{PV}_1 \vdash \forall \varphi \forall n \forall s \forall \pi \Big(n < n_0 \vee \neg \text{CNF}(\varphi, n) \vee \neg \text{Ref}_{\text{Res}}(\text{REF}_s(\varphi), \pi) \\ \vee \text{Sat}(\varphi, E_0(\varphi, n, s, \pi)) \vee \|\pi\| > \varepsilon s / n^2 \Big). \end{aligned} \quad (8.1)$$

This means that, when plugging a formula φ that is indeed a CNF formula over $n > n_0$ variables and the length of π is $|\pi| \leq 2^{\varepsilon n}$ and $s \geq n^3$, then the witnessing function E_0 correctly succeeds in finding a satisfying assignment of φ , and this is all provable in PV_1 . Building on this E_0 , we can further define a new polynomial-time function E that first checks whether $|\pi| \leq 2^{\varepsilon n}$, and then runs E_0 , and otherwise performs a brute-force check for a satisfying assignment for φ . It is not hard to see, building on eq. (8.1), that PV_1 can

now prove that this function E is a polynomial-time algorithm that always succeeds in finding a satisfying assignment if one exists, regardless of the size of π .

By applying Cook's translation (Theorem 2.3) to eq. (8.1) and restricting $s \geq n^3$, we obtain a uniform family of EF proofs showing the correctness of E as an extraction algorithm. Note that $\text{Ref}_{\text{Res}}(\varphi, \pi)$ is the first-order analogue of the propositional formula $\text{REF}_s(\varphi, \pi)$ that we have been studying until now. Cook's translation on $\text{Ref}_{\text{Res}}(\varphi, \pi)$ gives a propositional formula with the same behavior as $\text{REF}_s(\varphi)$. While $\llbracket \text{Ref}_{\text{Res}} \rrbracket_{n,s}$ may not be syntactically the same as $\text{REF}_s(\varphi, \pi)$, it is not hard to see that $\text{PV}_1 \vdash \forall n \forall s (\text{Taut}(\llbracket \text{Ref}_{\text{Res}} \rrbracket_{n,s}) \leftrightarrow \text{Taut}(\ulcorner \text{REF}_s(\varphi) \urcorner))$, making them effectively equivalent.

Since the proofs generated by Cook's translation are all polynomial-size and uniform, we conclude that there exists a polynomial $\ell(n, m, s, t)$ such that for every $s \geq n^3$,

$$\text{EF} \vdash_{\ell(n,m,s,t)} \text{REF}_t(\text{REF}_s(\varphi, \tau), \pi) \rightarrow \text{SAT}(\varphi, E_{n,m,s,t}(\varphi, \pi)), \quad (8.2)$$

where $E_{n,m,s,t}$ is the polynomial-size circuit encoding the computation of the extraction algorithm E on CNF formulas with n variables and m clauses, size parameter s and the refutation being analyzed consist of t clauses. $\square$

Similarly, the upper bound construction of Pudlák, which we formalized in Resolution, clearly goes through in stronger systems too (Theorem 7.1).

Suppose now that $A = \{A_{n,m,s}\}_{n \in \mathbb{N}}$ is a family of circuits that automate Resolution, meaning that there is a constant $c \in \mathbb{N}$ such that on input a CNF formula φ over n variables and m clauses, the circuit $A_{n,m,s}(\varphi)$ outputs a Resolution refutation of φ of size s^c if a refutation of size s exists. Consider the propositional formulas stating the correctness of this algorithm,

$$\text{AUT}_{n,m,s}^A := \text{REF}_s(\varphi, \pi) \rightarrow \text{REF}_{s^c}(\varphi, A_{n,m,s}(\varphi)). \quad (\text{AUT}^A)$$

We want to prove that if Extended Frege can efficiently derive the (AUT^A) formulas above for some sequence of circuits $\{A_{n,m,s}\}_{n,m,s \in \mathbb{N}}$, then it can also show that there is a family of small circuits $\{C_n\}_{n \in \mathbb{N}}$ solving SAT on 3-CNF formulas. We encode this as

$$\text{SAT}_n^C := \text{SAT}(\varphi, \alpha) \rightarrow \text{SAT}(\varphi, C_n(\varphi)). \quad (\text{SAT}^C)$$

We show that EF can derive (SAT^C) from (AUT^A) .

Theorem 8.2 (Resolution is NP-hard to automate, in EF). *Suppose Resolution is automatable by a sequence of circuits $A = \{A_{n,m,s}\}_{n,m,s \in \mathbb{N}}$, and suppose $\text{EF} \vdash_{\text{poly}} \text{AUT}_{n,m,s}^A$. Then, there exists a family of circuits $C = \{C_n\}_{n \in \mathbb{N}}$ of size $n^{O(1)}$ such that $\text{EF} \vdash_{n^{O(1)}} \text{SAT}_n^C$.*

Proof. Let $E = \{E_{n,m,s,t}\}_{n,m,s,t \in \mathbb{N}}$ be the sequence of polynomial-size circuits carrying out the computation of the extraction algorithm, as in Theorem 8.1, and let $\{P_{n,m,s}\}_{n,m,s \in \mathbb{N}}$ be the sequence of circuits constructing the canonical refutations of Pudlák. By Theorem 8.1, Theorem 7.1 and the assumptions in the statement, we have that Extended Frege can efficiently prove

- (i) the correctness of the upper bound on REF formulas, $\text{EF} \vdash_{\text{poly}} \text{SAT}_n(\varphi, \alpha) \rightarrow \text{REF}_t(\text{REF}_s(\varphi), P(\varphi, \alpha, s))$, for any $t \geq \tau$, where $\tau = \tau(n, m, s) = \text{poly}(n, m, s)$ is the parameter in Theorem 7.1;
- (ii) the correctness of the automating circuits A , $\text{EF} \vdash_{\text{poly}} \text{REF}_s(\varphi, \pi) \rightarrow \text{REF}_{s^c}(\varphi, A_{n,m,s}(\varphi))$;
- (iii) the correctness of the extraction algorithm E , $\text{EF} \vdash_{\text{poly}} \text{REF}_t(\text{REF}_s(\varphi), \pi) \rightarrow \text{SAT}_n(\varphi, E_{n,m,s,t}(\varphi, \pi))$, as long as $s \geq n^3$.

The circuit C_n solving SAT will be $C_n(\varphi) := E_{n,8n^3,n^3,t^c}(\varphi, A_{n,m,t}(\text{REF}_{n^3}(\varphi)))$ for a large enough $t \geq \tau = \text{poly}(n)$, where τ is again the parameter in Theorem 7.1. Namely, we use the automating circuit A to

produce a Resolution refutation from which we can extract a satisfying assignment using the extraction algorithm E . In between, the upper bound statement guarantees that a small refutation exists, and thus A will succeed in finding a not much larger one. The parameters are set up so that the three statements above correctly fit with each other: given a 3-CNF formula φ with n variables (and at most $8n^3$ clauses), we know that since φ is satisfiable, there is a size- τ refutation of $\text{REF}_s(\varphi)$, where $\tau = \text{poly}(n, s)$ and $s = n^3$. The extraction algorithm is guaranteed to work when $s \geq n^3$, so we choose to run the automating algorithm on $\text{REF}_{n^3}(\varphi)$. The automating algorithm is guaranteed to find a refutation of size t^c , and the extraction algorithm obtains a satisfying assignment from it. The fact that C is provably correct in EF then follows immediately by a chain of *modus ponens* on items (i)-(iii) above. $\square$

9 Universality of REF formulas

The formalizations in Theorem 6.9 and Theorem 7.1 have the interesting consequence of tightly relating the provability of arbitrary formulas to the provability of associated Resolution lower bounds. In Section 9.1 we make this precise for Extended Frege and stronger systems. In Section 9.2 we exploit this characterization to show that looking for proofs of tautologies in strong proof systems is equivalent to efficiently looking for proofs of Resolution lower bounds. Finally, Section 9.3 exploits similar ideas to provide examples of true exponential Resolution lower bounds that are unprovable in different propositional systems and first-order theories.

Our results apply to a wide range of proof systems, assuming some mild conditions on their behaviour.

Definition 9.1. We say that a proof system S is *reasonably strong* if it is polynomially equivalent to $\text{EF} + A$ for some set A of tautologies recognizable in polynomial time.

Recall that for every Cook–Reckhow system S , we have $\text{EF} + \text{REFL}^S \geq_p S$ (see also Section 2.2.2 for the definition of systems of the form $\text{EF} + A$). Reasonably strong proof systems have some features that make them nice to work with. We state them here for convenience.

Proposition 9.2 ([Kra25, Theorem 2.4.4]). *The following hold for every reasonably strong proof system S :*

- (i) *the system S is constructively closed under formula substitutions, i.e., given a proof of $\varphi(x_1, \dots, x_n)$ in size s and formulas $\psi_1, \dots, \psi_n$, we can obtain a proof of $\varphi(\psi_1, \dots, \psi_n)$ in time $\text{poly}(s, n, |\psi_1|, \dots, |\psi_n|)$;*
- (ii) *the system S is constructively closed under modus ponens, i.e., given an S -proof of φ in size s and an S -proof of $\varphi \rightarrow \psi$ in size t , we can obtain an S -proof of ψ in time $\text{poly}(s, t)$;*
- (iii) *the system S is constructively closed under contrapositions, i.e., given an S -proof of $\neg\psi$ in size s and an S -proof of $\varphi \rightarrow \psi$ in size t , we can obtain an S -proof of $\neg\varphi$ in time $\text{poly}(s, t)$;*
- (iv) *the system S implicationally simulates Resolution: for every two CNF formulas φ and ψ such that Resolution performs the derivation $\varphi \vdash_{\text{Res}} \psi$ in size s , there is an S -proof of $\varphi \rightarrow \psi$ in size $\text{poly}(s)$.*

We need one more definition before we proceed.

Definition 9.3. Let $\varphi(x_1, \dots, x_n)$ be a CNF formula and let $1 \leq k \leq n$. We define $\varphi[k]$ to be the CNF formula $\varphi \wedge \text{Ext}_n^k$, where Ext_n^k consists of all clauses encoding the extensions

$$y_{\{\ell_1, \dots, \ell_k\}} \leftrightarrow \ell_1 \wedge \dots \wedge \ell_k$$

for every set of literals $\{\ell_1, \dots, \ell_k\}$ over the variables $x_1, \dots, x_n$, where the y -variables are fresh variables.

It is well-known that if $\text{Res}(k)$ refutes φ in size s , then Resolution refutes $\varphi[k]$ in size $O(ks)$, and if Resolution refutes $\varphi[k]$ in size s , then $\text{Res}(k)$ refutes φ in size $O(ks)$ [AB04, Lemmas 1-2].

The following lemma is an important consequence of the formalization of Pudlák’s upper bound.

Lemma 9.4. *Let $S \geq \text{Res}$ be a propositional proof system closed under literal substitutions, modus ponens, contrapositions, and implicationally simulating Resolution. Then, there exists $p(n, m, s) = \text{poly}(n, m, s)$ such that for every n -variate CNF formula φ with m clauses, and every $t \geq p(n, m, s)$ if $S \vdash \neg \text{REF}_t(\text{REF}_s(\varphi))$ in size ℓ , then $S \vdash \neg \varphi[2]$ in size $\text{poly}(n, m, s, t, \ell)$.*

Proof. Assume that $S \vdash \neg \text{REF}_t(\text{REF}_s(\varphi), \pi)$ in size ℓ . By Theorem 7.1, we have that Resolution has polynomial-size derivations of the form

$$\text{SAT}_{\upharpoonright \varphi}(\alpha) \wedge \pi = P_{\upharpoonright \varphi}(\alpha) \vdash_{\text{Res}} \text{REF}_t(\text{REF}_s(\varphi), \pi), \quad (9.1)$$

where φ is fixed but α is encoded by free variables, and we define $p(n, m, s) = \tau(n, m, s) = \text{poly}(n, m, s)$ to be the parameter in Theorem 7.1. Since S implicationally simulates Resolution, it follows that S has polynomial-size refutations of

$$\text{SAT}_{\upharpoonright \varphi}(\alpha) \wedge \pi = P_{\upharpoonright \varphi}(\alpha) \rightarrow \text{REF}_t(\text{REF}_s(\varphi), \pi). \quad (9.2)$$

Since S is also closed under contrapositions, we have that there are polynomial-size S -proofs of

$$\neg (\text{SAT}_{\upharpoonright \varphi}(\alpha) \wedge \pi = P_{\upharpoonright \varphi}(\alpha)). \quad (9.3)$$

Now, by Proposition 2.9.(i), we can use a substitution on the variables of SAT to get a proof of $\neg(\varphi(\alpha) \wedge \pi = P_{\upharpoonright \varphi}(\alpha))$. By the way we defined the clauses in $\pi = P_{\upharpoonright \varphi}(\alpha)$ in Section 7.2, once φ has been fixed, there are no variables of type a-lit left. By inspecting all the extension axiom from (7.8) to (7.21) we see that all extension axioms relate variables of π to literals over the α variables. Crucially, the width of these extensions is at most two, given by the extensions (7.20) and (7.21). Furthermore, all possible extensions over two α literals are available, meaning that $(\varphi(\alpha) \wedge \pi = P_{\upharpoonright \varphi}(\alpha)) \equiv \varphi \wedge \text{Ext}_n^2 = \varphi[2]$, so this is in fact a proof of $\neg \varphi[2]$. $\square$

9.1 Propositional fragments of the Atserias–Müller lower bound

As a consequence of Theorem 8.1 and Theorem 7.1, we can characterize the exact fragment of the Atserias–Müller lower bound efficiently provable by every strong enough propositional proof system. Namely, if EF can argue that φ is unsatisfiable, then it can also argue that $\text{REF}_{n^3}(\varphi)$ is hard for Resolution; and, conversely, if $\text{REF}_{n^3}(\varphi)$ is provably hard for Resolution, then EF can argue that φ itself is unsatisfiable. The key insight here is that the statement “ $\text{REF}_{n^3}(\varphi)$ is hard for Resolution” is exactly a REF formula itself, of the form $\text{REF}(\text{REF}(\varphi))$.

The following is a formal restatement of Theorem 1.8.

Theorem 9.5. *Let S be a reasonably strong proof system. There is $p(n, m, s) = \text{poly}(n, m, s)$ such that for every CNF formula $\varphi(x_1, \dots, x_n)$ with m clauses, every $t \in \mathbb{N}$, and every $s \geq n^3$,*

- (i) *if $S \vdash \neg \varphi$ in size ℓ , then $S \vdash \neg \text{REF}_t(\text{REF}_s(\varphi_n))$ in size $\text{poly}(n, m, s, t, \ell)$;*
- (ii) *if $S \vdash \neg \text{REF}_{p(n, m, s)}(\text{REF}_s(\varphi_n))$ in size ℓ , then $S \vdash \neg \varphi_n$ in size $\text{poly}(n, m, s, \ell)$.*

Furthermore, in both cases, given an S -proof of the left-hand side statement, one can obtain a proof of the right-hand side statement in polynomial time.

Proof. In the following REF formulas φ is a fixed formula, while the only free variables are the π and α variables.

- (i) If $S \vdash \neg \varphi$ via some proof π of size ℓ , then by Proposition 2.9, S also proves $\neg \text{SAT}_{\upharpoonright \varphi}(\alpha)$ and we can easily substitute the extraction circuit $E_{\upharpoonright \varphi}(\pi)$ into α , getting a proof π' of $\neg \text{SAT}_{\upharpoonright \varphi}(E_{\upharpoonright \varphi}(\pi))$. By Theorem 8.1,

EF proves $\text{REF}_t(\text{REF}_s(\varphi, \tau), \pi) \rightarrow \text{SAT}(\varphi, E(\varphi, \pi))$ and by the p-simulation $S \geq_p$ EF these proofs are also available in S . Hence, by contraposition and the restriction on φ , S derives $\neg \text{REF}_t(\text{REF}_s(\varphi, \tau), \pi)$, and it is easy to see that the total blow-up incurred by these additional derivations is at most $\text{poly}(n, m, s, t, \ell)$.

- (ii) This backwards direction follows from Lemma 9.4 above. Since S is reasonably strong, the conditions of Lemma 9.4 apply, giving us polynomial-size refutations of $\varphi[2]$. The system S is in particular closed under clause substitutions, so we can substitute the extension axioms introduced by $\varphi[2]$ to get a refutation of φ . $\square$

9.2 Automatability in terms of REF formulas

We will say that a proof system S is (weakly) *automatable on REF formulas* if there is an algorithm A that behaves like an automating algorithm whenever the input formula is a $\text{REF}_s(\varphi)$ formula for some φ and s , and is allowed to behave in any other way otherwise.

We remark that these are the REF formulas *for Resolution*, meaning that proof search in arbitrarily strong proof systems can be completely characterized by the proof search of Resolution lower bounds! The following theorems are the formal restatements of Theorem 1.9.

Theorem 9.6. *Let S be a reasonably strong proof system. Then, the following are equivalent:*

- (i) S is automatable;
- (ii) S is automatable on REF formulas.

Proof. One direction is trivial. For the other implication, let A be an automating algorithm that is only guaranteed to work on tautologies of the form $\neg \text{REF}_s(\cdot)$, and let φ be an unsatisfiable CNF formula with n variables and m clauses. We describe a new algorithm A' that correctly automates S on all inputs, meaning that if $\neg\varphi$ has an S -refutation in size t , then A' outputs a refutation in time $t^{O(1)}$.

By the formalization of the correctness of the extraction algorithm in Theorem 8.1, we obtained Theorem 9.5 from where it follows that if S has a refutation of φ in size t , then there exists an S -refutation of $\text{REF}_{p(n,m)}(\text{REF}_{n^3}(\varphi))$ for a fixed polynomial $p(n, m)$, in size $\text{poly}(n, m, t)$. This is precisely a REF($\cdot$) formula, so we can run $A(\text{REF}_{p(n,m)}(\text{REF}_{n^3}(\varphi)))$ to obtain a refutation π in S with at most a fixed polynomial blow-up. Now, by the backwards direction of Theorem 9.5, we know that S has a refutation of φ in size $\text{poly}(n, m, t)$ and, we can obtain this efficiently from π . Thus, we have an automating algorithm that given φ , which had a refutation in S in size t , outputs a refutation in size $t^{O(1)}$. $\square$

Since nothing in the argument above prevents us from talking about proofs in stronger systems, the result immediately carries over to weak automatability. For the following statement, we use the well-known fact that a proof system S is weakly automatable if, and only if, there exists an automatable proof system Q that simulates S . If we restrict ourselves to REF formulas, then it is easy to see that S is weakly automatable on REF formulas if, and only if, there is a proof system Q automatable on REF formulas and simulating S over REF formulas.

Theorem 9.7. *Let S be a reasonably strong proof system. Then, the following are equivalent:*

- (i) S is weakly automatable;
- (ii) S is weakly automatable on REF formulas.

Proof. The argument is identical. The only difference is that instead of an automating algorithm for S we have a proof system Q that simulates S on REF formulas and is automatable on REF formulas. Then, if $\neg\varphi$ can be refuted in size t in S , we can still guarantee (by Theorem 9.5) that there is a proof of $\text{REF}_{p(n,m)}(\text{REF}_{n^3}(\varphi))$, for a fixed polynomial $p(n, m)$, in size $\text{poly}(n, m, t)$. We look for these proofs with the automating algorithm for Q . Since Q itself may not be reasonably strong itself, we turn this Q -proof into a proof in the system $\text{EF} + \text{REFL}^Q$, which p -simulates Q , and for which Theorem 9.5 does apply. $\square$

The argument above crucially requires that S is at least as strong as Extended Frege. The reason is that we require the correctness statement of the extraction algorithm to be provable in S . If φ has a refutation of size t , then Theorem 9.5(i) guarantees that there is a size- $t^{O(1)}$ refutation of $\text{REF}(\text{REF}(\varphi))$, so the algorithm can focus on looking for these latter refutations instead. For an arbitrary proof system $S \geq \text{Res}$ that does not simulate EF, if S proves $\neg\varphi$ in size t , then we cannot guarantee that it also proves $\neg\text{REF}(\text{REF}(\varphi))$ in size $t^{O(1)}$.

This problem can be solved if S is Resolution itself; then, the semantics of the REF formula compensate for the weakness of the system, since now the REF formulas are about the proof system itself.

Theorem 9.8. *The following hold for Resolution:*

- (i) *Resolution is automatable if, and only if, it is automatable on REF formulas;*
- (ii) *Resolution is weakly automatable if, and only if, it is weakly automatable on REF formulas.*

Proof. The first item is easy, while the second one requires a bit more work.

- (i) This first item is a somewhat trivial byproduct of the hardness of automating Resolution [AM20]: if Resolution is automatable on REF formulas, this can be used to decide SAT and hence $\text{P} = \text{NP}$; but then every proof system, including Resolution, is automatable.
- (ii) Suppose Resolution is weakly automatable on REF formulas. This means there exists a proof system Q that simulates Resolution on REF formulas and is automatable on REF formulas. Now, suppose a CNF formula φ with n variables and m clauses has a Resolution refutation in size t . Then, run the automating algorithm for Q on formulas of the form $\text{REF}_{p(n,m,s)}(\text{REF}_s(\varphi))$ for $p(n, m, s)$ the polynomial from Theorem 9.5 for $s = 1, s = 2$, and so on, via dovetailing, until a refutation is found in Q . Note that, since there exists a refutation of φ in size t , that means that $\text{REF}_t(\varphi)$ is satisfiable and hence $\text{REF}_{p(n,m,t)}(\text{REF}_t(\varphi))$ is unsatisfiable but easy to refute in Resolution. Hence, the algorithm automating Q on REF formulas must find a refutation π in size $\text{poly}(n, m, t)$.

At this point we would like to turn this refutation π into a refutation of φ . We can turn π into a refutation in $\text{EF} + \text{REFL}^Q$, which p -simulates EF and is closed under *modus ponens*. Since Theorem 9.5 applies to these stronger systems, we just described an algorithm that finds a refutation of φ in time $\text{poly}(n, m, t)$ in the system $\text{EF} + \text{REFL}^Q \geq_p \text{Res}$, where t is the size of the shortest Resolution refutation. We can conclude that Resolution is weakly automatable. $\square$

By the classical result showing that Resolution is weakly automatable if and only if $\text{Res}(k)$ is weakly automatable for every constant $k \geq 1$ [AB04, Theorem 8], we have the following corollary.

Corollary 9.9. *For every $k \geq 1$, $\text{Res}(k)$ is weakly automatable if, and only if, $\text{Res}(k)$ is weakly automatable on REF formulas.*

9.3 Unprovability of Resolution lower bounds

The power of Theorem 9.5 can be applied to study the provability of Resolution lower bounds. In what follows, we will say that a formula $\neg \text{REF}_s(\varphi)$ constitutes a *true Resolution lower bound* if φ is unsatisfiable and does not have Resolution refutations of size s . Could there be a proof system that is polynomially bounded on all such formulas? And, in the first-order setting, is there perhaps a theory of arithmetic capable of proving all true Resolution lower bounds?

9.3.1 Propositional unprovability

We show that this is not the case: assuming S is strong enough and not polynomially bounded, there will be true Resolution lower bounds that S cannot derive in polynomial size.

We first exemplify this by giving explicit exponential lower bounds on REF formulas for bounded-depth Frege. The following is a formal restatement of Corollary 1.11.

Theorem 9.10 (Hard REF formulas for bounded-depth Frege). *For every $d \leq O(\log n / \log \log n)$, there are polynomials $p(n)$ and $q(n)$ such that the formulas in the sequence $\{\neg \text{REF}_{p(n)}(\text{REF}_{q(n)}(\text{PHP}_n))\}_{n \in \mathbb{N}}$ are all tautological but require size $\exp(\Omega(n^{1/(4d-2)}))$ to be proven in depth- d Frege systems.*

Proof. The formula PHP_n has $N = \Theta(n^2)$ variables and $\Theta(n^3)$ clauses. Consider first the formulas $\text{REF}_{q(n)}(\text{PHP}_n)$. Here, to apply the original bound on REF formulas (Theorem 1.6), $q(n)$ has to be at least a square in the number of variables of PHP_n , so we define $q(n) := N^2 = \Theta(n^4)$. Let the size parameter $p(n)$ of the outer REF formula be the polynomial p from Lemma 9.4 applied to N and $q(n)$. That the formulas $\text{REF}_{p(n)}(\text{REF}_{q(n)}(\text{PHP}_n))$ are tautologies follows from the fact that PHP_n is unsatisfiable, meaning that the lower bound on REF formulas guarantees that $\text{REF}_{q(n)}(\text{PHP}_n)$ is exponentially hard for Resolution and hence the second nesting of REF remains unsatisfiable. The statement then follows from Lemma 9.4: if depth- d Frege refutes $\text{REF}_{p(n)}(\text{REF}_{q(n)}(\text{PHP}_n))$ in size ℓ , then depth- d Frege refutes $\text{PHP}_n[2]$ in size $\text{poly}(n, \ell)$. Substituting these 2-extension axioms increases the depth of every line in the proof by at most one, giving a depth- $(d+1)$ Frege refutation of PHP_n in size $\text{poly}(n, \ell)$, but, by Theorem 2.1, depth- $(d+1)$ Frege requires size $\exp(\Omega(n^{1/(4d-2)}))$ to refute PHP_n . $\square$

There is nothing special about the pigeonhole principle in the argument above, and we can generalize this to any reasonably strong system where some lower bound is known. The following is a formal restatement of Theorem 1.10.

Theorem 9.11 (Propositional unprovability of Resolution lower bounds). *For every propositional proof system $S \geq \text{Res}$ implicationaly simulating Resolution and closed under contrapositions and clause substitutions, if S is not polynomially bounded, there exists a family of unsatisfiable CNF formulas $\{\psi_n\}_{n \in \mathbb{N}}$ on $N = \text{poly}(n)$ variables and of size $|\psi_n| = \text{poly}(n)$ such that*

- (i) *they require size at least $2^{N^{\Omega(1)}}$ to be refuted in Resolution;*
- (ii) *there is a polynomial $p(n)$ such that the formulas $\{\neg \text{REF}_{p(n)}(\psi_n)\}_{n \in \mathbb{N}}$ are tautological but do not have polynomial-size proofs in S .*

Proof. Since S is not polynomially bounded, there exists a sequence of unsatisfiable 3-CNF formulas $\{\varphi_n\}_{n \in \mathbb{N}}$, each over n variables, that cannot be refuted by S in polynomial size. This means that the formulas of the form $\text{REF}_{n^2}(\varphi_n)$ are unsatisfiable, or else the satisfying assignment would be a correct Resolution refutation which could be turned into a correct refutation in S . By the lower bound on REF formulas (Theorem 1.6), $\text{REF}_{n^2}(\varphi_n)$ requires size $2^{\Omega(n)}$ to be refuted in Resolution, meaning that $\text{REF}_{p(n)}(\text{REF}_{n^2}(\varphi_n))$ is unsatisfiable for every polynomial $p(n)$. If we choose p to be the polynomial in Lemma 9.4, it now follows from this same

lemma that S cannot possibly have polynomial-size proofs of the family $\{\neg \text{REF}_{p(n)}(\text{REF}_{n^2}(\varphi_n))\}_{n \in \mathbb{N}}$, or else there would be polynomial-size S -proofs of $\{\neg \varphi_n[2]\}_{n \in \mathbb{N}}$. Since S is closed under clause substitutions, we could substitute the 2-extensions of $\neg \varphi_n[2]$ and get polynomial-size proofs of $\{\neg \varphi_n\}_{n \in \mathbb{N}}$, a contradiction.

The formula family $\{\psi_n\}_{n \in \mathbb{N}}$ in the statement is then precisely given by $\psi_n := \text{REF}_{n^2}(\varphi_n)$. By Remark 2.6 the formula ψ_n has $N := \text{poly}(n)$ variables and polynomial size, which yields the rest of the parameters in the statement. $\square$

The informal statement in Theorem 1.10 is obtained in particular by taking $p(n)$ to be N^2 .

Remark 9.12. Since every proof system S can be p-simulated by $\text{EF} + \text{REFL}^S$, and the latter is closed under *modus ponens* and clause substitutions, that means that the previous statement applies to every proof system S . This implies that, unless $\text{NP} = \text{coNP}$, no proof systems can efficiently derive all true Resolution lower bounds. We note, however, that this follows already from the classical work of Iwama [Iwa97], who showed that the Proof Size Problem for Resolution (PSP_{Res}) is NP-complete. His many-one reduction mapped a 3-CNF formula φ over n variables to a new formula Φ over $O(n \log n)$ variables, such that φ is satisfiable if and only if Φ has a Resolution refutation of size $S(n) = O(n^3 \log n)$. In the case when φ is unsatisfiable, however, Iwama only proves a lower bound of $S(n) + g(n)$, for some function $g(n) = O(S(n)/n)$. This suffices for his purposes, but it is certainly not enough to get the superpolynomial gap needed for hardness of automatability obtained in [AM20]. In our statement the parameters achieve almost optimal range: the formulas are exponentially hard, but a slightly subquadratic lower bound is not efficiently provable. $\blacktriangleleft$

9.3.2 First-order unprovability of Resolution lower bounds

Our main result here is that a first-order version of the propositional unprovability results above hold unconditionally in the context of strong enough theories of arithmetic. The formal statement and proof follow.

Theorem 9.13. *Let T be a consistent first-order extending S_2^1 with a set of axioms recognizable in polynomial time and admitting $\mathbb{N}$ as a model. Then, there exists a sequence of unsatisfiable propositional formulas $\{\psi_{k,s}\}_{k,s \in \mathbb{N}}$ described uniformly by a polynomial-time algorithm given k and s in unary, where $\psi_{k,s}$ has $N = \text{poly}(k, s)$ variables, and such that*

- (i) *Resolution refutations of the formula $\psi_{k,s}$ require size $2^{N^{\Omega(1)}}$;*
- (ii) *there are constants $c > 0$ and $N_0 \in \mathbb{N}$ such that the lower bound expressed by the first-order sentence $\forall k \forall s \forall \pi (N > N_0 \wedge \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi) \rightarrow |\pi| > N^c)$ is independent of T .*

Proof. Given the theory T , consider the strong proof system of T , denoted P_T , as defined in Section 2.3.5. Since T is polynomial-time axiomatizable, the system P_T is a well-defined Cook–Reckhow proof system. For convenience, we see P_T as a refutation system, and we consider the propositional formulas encoding its reflection principle $\neg \text{REFL}_{k,s}^{P_T} := \text{REF}_s^{P_T}(\varphi, \tau) \wedge \text{SAT}_k(\varphi, \alpha)$, as defined in Definition 2.10, where k denotes the number of variables of φ and s is the size of the proofs considered. We assume for convenience that φ is always a 3-CNF formula and has hence $O(k^3)$ clauses. Let us also assume that the formula $\neg \text{REFL}_{k,s}^{P_T}$ itself is written as a 3-CNF formula and consists of $n := \text{poly}(k, s)$ variables. Consider now the propositional formula $\psi_{k,s} := \text{REF}_{n^2}(\neg \text{REFL}_{k,s}^{P_T})$ stating that there is a size- n^2 Resolution refutation of $\neg \text{REFL}_{k,s}^{P_T}$. It is easy to verify following Remark 2.6 that $\psi_{k,s}$ has $N := \text{poly}(n)$ variables and size $\text{poly}(n)$.

Observe, first, that the sequence $\{\psi_{k,s}\}_{k,s \in \mathbb{N}}$ can be generated uniformly in polynomial time given k and s in unary. Next, note that all the formulas $\psi_{k,s}$ are unsatisfiable: otherwise, there would be size- n^2 Resolution refutations of the reflection principle of P_T , and this would amount to Resolution polynomially simulating P_T . In more detail, suppose F is a 3-CNF formula over k variables with a size- s refutation in P_T . If Resolution can refute $\neg \text{REFL}_{k,s}^{P_T}(\varphi, \tau, \alpha)$ in size n^2 , then by restricting φ by F and τ by the size- s P_T -refutation

in question, we get a size- n^2 refutation of $\text{SAT}(F, \alpha)$. Resolution can then refute F in its usual “native” encoding via Proposition 2.9, so we get a Resolution refutation of F in size $n^{O(1)} = \text{poly}(k, s)$, concluding $\text{Res} \geq P_T$. This is a contradiction, because since $T \supseteq S_2^1$ we know that P_T simulates Extended Frege [Pud20, Section 4.2, Fact 2], and it is well-known that Resolution is strictly weaker than EF.

Finally, because all of the formulas $\neg \text{REFL}_{k,s}^{P_T}$ are unsatisfiable, the lower bound on REF formulas (Theorem 1.6) guarantees that for large enough $n = \text{poly}(k, s)$, the formula $\psi_{k,s} = \text{REF}_{n^2}(\neg \text{REFL}_{k,s}^{P_T})$ requires size $2^{\Omega(n^2/n)} = 2^{\Omega(n)}$ to be refuted in Resolution. Since $\psi_{k,s}$ has $N = \text{poly}(n)$ variables, the size lower bound in terms of N is $2^{N^{\Omega(1)}}$. In particular, that means that for every $c > 0$, there is some $N_0 \in \mathbb{N}$ such that the much weaker lower bound statement $\forall k \forall s \forall \pi (N > N_0 \wedge |\pi| \leq N^c \rightarrow \neg \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi))$ holds in the standard model $\mathbb{N}$, which is a model of T . Thus the lower bound is consistent with T .

We now provide a model where the sentence fails. By Gödel’s second incompleteness theorem, T cannot prove the soundness of P_T [Pud20, Section 4.2, Fact 3]. Thus, there is a model M of T where the reflection principle of P_T fails. We claim that there is $c > 0$ and a suitable $N_0 \in \mathbb{N}$ such that the sentence $\forall k \forall s \forall \pi (N > N_0 \wedge |\pi| \leq N^c \rightarrow \neg \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi))$ fails in this model as well. Indeed, since the reflection of P_T fails in M , there exist some nonstandard 3-CNF formula $\varphi_0 \in M \setminus \mathbb{N}$, a P_T -refutation $\tau_0 \in M \setminus \mathbb{N}$ and $\alpha_0 \in M \setminus \mathbb{N}$ such that $M \models \text{Ref}_{P_T}(\varphi_0, \tau_0) \wedge \text{Sat}(\varphi_0, \alpha_0)$. This formula φ_0 has some nonstandard number of variables $k_0 \in M \setminus \mathbb{N}$, and similarly τ_0 has some length $s_0 := |\tau_0| \in M \setminus \mathbb{N}$. We then have that in the model M , the formula $\neg \text{REFL}_{k_0, s_0}^{P_T}(\varphi, \tau, \alpha)$ is satisfiable.

Now, since M is a model of T and T extends S_2^1 , by Theorem 7.1 we have that the upper bound on REF formulas holds in M . That is,

$$\begin{aligned} M \models \forall \varphi \forall n \forall m \forall \alpha \forall t \Big(& \text{CNF}(\varphi, n, m) \wedge \text{Sat}(\varphi, \alpha) \\ & \rightarrow \exists \pi (|\pi| \leq c_P \cdot t(m + tn^2) \wedge \text{Ref}_{\text{Res}}(\text{REF}_t(\varphi), \pi)) \Big), \end{aligned} \quad (9.4)$$

where c_P is the constant in the Big-Theta term $\Theta(t(m + tn^2))$ bounding the size of upper bound construction in Theorem 7.1. (Technically speaking Theorem 7.1 proved the previous sentence in Resolution, but a simple inspection of the proof reveals that the construction is perfectly uniform and can be immediately formalized in S_2^1 —and, for that matter, even in much weaker theories.)

Hence, substituting φ for the 3-CNF formula $\neg \text{REFL}_{k_0, s_0}^{P_T}$ on n variables and $m \leq 8n^3$ clauses, and t for n^2 , we get

$$M \models \exists \pi (|\pi| \leq c_P \cdot (8n^5 + n^6) \wedge \text{Ref}_{\text{Res}}(\psi_{k_0, s_0}, \pi)) . \quad (9.5)$$

Since the number N of variables of ψ_{k_0, s_0} is $N = n^{\Theta(1)}$, the bound on $|\pi|$ is $c_P \cdot (8n^5 + n^6) \leq N^{O(1)}$ for large enough N , it follows that there is $c > 0$ and $N_0 \in \mathbb{N}$ such that the first-order sentence $\forall k \forall s \forall \pi (N > N_0 \wedge |\pi| \leq N^c \rightarrow \neg \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi))$ fails in M . $\square$

With a couple of extra tricks, we can extend the result to any theory T containing just Robinson Arithmetic rather than all of S_2^1 . We remark as well that there is nothing essential about T being polynomial-time axiomatizable, and one could generalize this to any recursively enumerable extension of $\mathbf{Q}$ via a padding trick [Cra53]. The following is a formal version of Theorem 1.12.

Corollary 9.14. *Let T be a consistent first-order extending $\mathbf{Q}$ with a set of axioms recognizable in polynomial time. Then, there exists a sequence of unsatisfiable propositional formulas $\{\psi_{k,s}\}_{k,s \in \mathbb{N}}$ described uniformly by a polynomial-time algorithm given k and s in unary, where $\psi_{k,s}$ has $N = \text{poly}(k, s)$ variables, and such that*

- (i) *Resolution refutations of the formula $\psi_{k,s}$ require size $2^{N^{\Omega(1)}}$;*
- (ii) *there is a constant $c > 0$ and $N_0 \in \mathbb{N}$ such that the lower bound expressed by the first-order sentence $\forall k \forall s \forall \pi (N > N_0 \wedge \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi) \rightarrow |\pi| > N^c)$ is unprovable in T , but true in $\mathbb{N}$.*

Proof. Define $T' := S_2^1 + \Pi_1\text{-Refl}_T$. Observe that T' satisfies all the conditions of Theorem 9.13. First, the theory is consistent because $\mathbb{N} \models T'$. This is because $\mathbb{N} \models S_2^1$ and, as T is consistent and extends Q , which refutes every false Π_1 -sentence, $\Pi_1\text{-Refl}_T$ is a true sentence. Second, T' is clearly recursively axiomatizable because S_2^1 is, and in particular the axioms can be recognized in polynomial time. Hence, Theorem 9.13 guarantees a sequence of unsatisfiable propositional formulas $\{\psi_{k,s}\}_{k,s \in \mathbb{N}}$ with the desired properties. In particular, the statement $\forall k \forall s \forall \pi (N > N_0 \wedge |\pi| \leq N^c \rightarrow \neg \text{Ref}_{\text{Res}}(\psi_{k,s}, \pi))$ is unprovable in T' , where c and N_0 are again given by Theorem 9.13. On the one hand, this means the sentence is not provable in T' . Thus it cannot be proven in T either, since, by construction, $T' = S_2^1 + \Pi_1\text{-Refl}_T$ is Π_1 -conservative over T and the sentence is a Π_1 -sentence. On the other hand, as argued in the proof of Theorem 9.13, the lower bound statement is true in $\mathbb{N}$ by the lower bound on REF formulas (Theorem 1.6). $\square$

Acknowledgements

We are indebted to Ján Pich for insightful initial discussions on the problem and particularly for the idea behind the proof of Theorem 1.3. A sketch of his original idea for the proof, based on a reduction from MCSP, is outlined in Section B.2 of the conference version of this paper [AAdRK25]. We thank Emil Jeřábek [Jeř25] for the technical insight on how to simulate contraposition inferences in Resolution, implicit in the proof of Theorem 7.1. We also thank Jonas Conneryd for useful comments and careful proofreading of a draft of this work, and Anupam Das, Stefan Grosser, Antonina Kolokolova, Jan Krajíček, Jakob Nordström, Kilian Risse, and Rahul Santhanam, as well as the anonymous FOCS reviewers, for helpful comments and suggestions.

This collaboration started at the Proof Complexity and Beyond workshop at Mathematisches Forschungsinstitut Oberwolfach, in March 2024. Part of this work was carried out during the Proof Complexity Workshop at the University of Oxford in September of 2024 and during the *Kaleidoscope de la complexité* spring school in April 2025 at the Centre International de Rencontres Mathématiques (CIRM) in Marseille, France. We also gratefully acknowledge that we have benefited greatly from being part of the Basic Algorithms Research Centre (BARC) environment financed by the Villum Investigator grant 54451.

Noel Arteche was supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. Albert Atserias was supported by the Spanish Research Agency through grant PID2022-138506NB-C22 (PROOFS BEYOND) and the Severo Ochoa and María de Maeztu Program for Centers and Units of Excellence in R&D (CEX2020-001084-M). Susanna F. de Rezende received funding from the Knut and Alice Wallenberg grant KAW 2023.0116, ELLIIT, and the Swedish Research Council grant 2021-05104. Erfan Khaniki was supported by the Royal Society University Research Fellowship URF\R1\211106 “Proof complexity and circuit complexity: a unified approach”.

References

- [AAdRK25] N. Arteche, A. Atserias, S. F. de Rezende, and E. Khaniki, “The proof analysis problem,” in *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2025, pp. 2558–2579. DOI: [10.1109/FOCS63196.2025.00133](https://doi.org/10.1109/FOCS63196.2025.00133)
- [AB04] A. Atserias and M. L. Bonet, “On the automatizability of resolution and related propositional proof systems,” *Inform. and Comput.*, vol. 189, no. 2, pp. 182–201, 2004. DOI: [10.1016/j.ic.2003.10.004](https://doi.org/10.1016/j.ic.2003.10.004)
- [AB09] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. DOI: [a10.1017/CBO9780511804090](https://doi.org/10.1017/CBO9780511804090)

- [ACG24] N. Arteche, G. Carenini, and M. Gray, “Quantum automating TC^0 -Frege is LWE -hard,” in *39th Computational Complexity Conference, CCC 2024, July 22-25, 2024, Ann Arbor, MI, USA*, 2024, 15:1–15:25. doi: [10.4230/LIPIcs.CCC.2024.15](https://doi.org/10.4230/LIPIcs.CCC.2024.15)
- [AD08] A. Atserias and V. Dalmau, “A combinatorial characterization of resolution width,” *J. Comput. System Sci.*, vol. 74, no. 3, pp. 323–334, 2008. doi: [10.1016/j.jcss.2007.06.025](https://doi.org/10.1016/j.jcss.2007.06.025)
- [Ajt94] M. Ajtai, “The complexity of the pigeonhole principle,” *Combinatorica*, vol. 14, no. 4, pp. 417–433, 1994. doi: [10.1007/BF01302964](https://doi.org/10.1007/BF01302964)
- [AKPS24] N. Arteche, E. Khaniki, J. Pich, and R. Santhanam, “From proof complexity to circuit complexity via interactive protocols,” in *51st International Colloquium on Automata, Languages, and Programming*, 2024, Art. No. 12, 20. doi: [10.4230/lipics.icalp.2024.12](https://doi.org/10.4230/lipics.icalp.2024.12)
- [All25] E. Allender, **P-uniform vs. DLOGTIME-uniform AC^0** , Theoretical Computer Science Stack Exchange, 2025. [Online]. Available: <https://csttheory.stackexchange.com/q/55461>
- [ALWZ21] R. Alweiss, S. Lovett, K. Wu, and J. Zhang, “Improved bounds for the sunflower lemma,” *Annals of Mathematics*, vol. 194, no. 3, pp. 795–815, 2021. doi: [10.4007/annals.2021.194.3.5](https://doi.org/10.4007/annals.2021.194.3.5)
- [AM11] A. Atserias and E. Maneva, “Mean-payoff games and propositional proofs,” *Inform. and Comput.*, vol. 209, no. 4, pp. 664–691, 2011. doi: [10.1016/j.ic.2011.01.003](https://doi.org/10.1016/j.ic.2011.01.003)
- [AM20] A. Atserias and M. Müller, “Automating resolution is NP-hard,” *Journal of the ACM (JACM)*, vol. 67, no. 5, pp. 1–17, 2020. doi: [10.1145/3409472](https://doi.org/10.1145/3409472)
- [AR08] M. Alekhnovich and A. A. Razborov, “Resolution is not automatizable unless $W[P]$ is tractable,” *SIAM Journal on Computing*, vol. 38, no. 4, pp. 1347–1363, 2008. doi: [doi: 10.1137/06066850X](https://doi.org/10.1137/06066850X)
- [AT24] A. Atserias and I. Tzameret, “Feasibly constructive proof of Schwartz-Zippel lemma and the complexity of finding hitting sets,” *Electronic Colloquium on Computational Complexity (ECCC)*, no. TR24-174, 2024. [Online]. Available: <https://eccc.weizmann.ac.il/report/2024/174/>
- [Ats13] A. Atserias, “The proof-search problem between bounded-width resolution and bounded-degree semi-algebraic proofs,” in *Theory and applications of satisfiability testing—SAT 2013*, ser. Lecture Notes in Comput. Sci. Vol. 7962, 2013, pp. 1–17. doi: [10.1007/978-3-642-39071-5_1](https://doi.org/10.1007/978-3-642-39071-5_1)
- [BDG+04] M. L. Bonet, C. Domingo, R. Gavaldà, A. Maciel, and T. Pitassi, “Non-automatizability of bounded-depth Frege proofs,” *computational complexity*, vol. 13, pp. 47–68, 2004. doi: [10.1007/s00037-004-0183-5](https://doi.org/10.1007/s00037-004-0183-5)
- [Bel20] Z. Bell, “Automating regular or ordered resolution is NP-hard,” *Electronic Colloquium on Computational Complexity (ECCC)*, no. TR20-105, 2020. [Online]. Available: <https://eccc.weizmann.ac.il/report/2020/105>
- [BP96] P. Beame and T. Pitassi, “Simplified and improved resolution lower bounds,” in *Proceedings of 37th Conference on Foundations of Computer Science*, 1996, pp. 274–282. doi: [10.1109/SFCS.1996.548486](https://doi.org/10.1109/SFCS.1996.548486)
- [BPR00] M. L. Bonet, T. Pitassi, and R. Raz, “On interpolation and automatization for Frege systems,” *SIAM J. Comput.*, vol. 29, no. 6, pp. 1939–1967, 2000. doi: [10.1137/S0097539798353230](https://doi.org/10.1137/S0097539798353230)
- [BPT14] A. Beckmann, P. Pudlák, and N. Thapen, “Parity games and propositional proofs,” *ACM Trans. Comput. Logic*, vol. 15, no. 2, May 2014. doi: [10.1145/2579822](https://doi.org/10.1145/2579822)
- [Bus86] S. R. Buss, *Bounded arithmetic*. Bibliopolis, Naples, 1986.

- [Bus97] S. R. Buss, “Bounded arithmetic and propositional proof complexity,” in *Logic of computation (Marktoberdorf, 1995)*, ser. NATO Adv. Sci. Inst. Ser. F: Comput. Systems Sci. Vol. 157, Springer, Berlin, 1997, pp. 67–121.
- [Bus98] S. R. Buss, “First-order proof theory of arithmetic,” in *Handbook of proof theory*, ser. Stud. Logic Found. Math. Vol. 137, North-Holland, Amsterdam, 1998, pp. 79–147. doi: [10.1016/S0049-237X\(98\)80017-7](https://doi.org/10.1016/S0049-237X(98)80017-7)
- [BW01] E. Ben-Sasson and A. Wigderson, “Short proofs are narrow—resolution made simple,” *J. ACM*, vol. 48, no. 2, pp. 149–169, 2001. doi: [10.1145/375827.375835](https://doi.org/10.1145/375827.375835)
- [BY24] S. Buss and E. Yolcu, “Regular resolution effectively simulates resolution,” *Inform. Process. Lett.*, vol. 186, Paper No. 106489, 4, 2024. doi: [10.1016/j.ipl.2024.106489](https://doi.org/10.1016/j.ipl.2024.106489)
- [CEI96] M. Clegg, J. Edmonds, and R. Impagliazzo, “Using the Groebner basis algorithm to find proofs of unsatisfiability,” in *Proceedings of the Twenty-eighth Annual ACM Symposium on the Theory of Computing (Philadelphia, PA, 1996)*, ACM, New York, 1996, pp. 174–183. doi: [10.1145/237814.237860](https://doi.org/10.1145/237814.237860)
- [Cob65] A. Cobham, “The intrinsic computational difficulty of functions,” in *Logic, Methodology and Philos. Sci. (Proc. 1964 Internat. Congr.)*, North-Holland, Amsterdam, 1965, pp. 24–30.
- [Coo75] S. A. Cook, “Feasibly constructive proofs and the propositional calculus (preliminary version),” in *Seventh Annual ACM Symposium on Theory of Computing (Albuquerque, N.M., 1975)*, Association for Computing Machinery, New York, 1975, pp. 83–97.
- [CP90] S. Cook and T. Pitassi, “A feasibly constructive lower bound for resolution proofs,” *Information Processing Letters*, vol. 34, no. 2, pp. 81–85, 1990. doi: [10.1016/0020-0190\(90\)90141-J](https://doi.org/10.1016/0020-0190(90)90141-J)
- [CR79] S. A. Cook and R. A. Reckhow, “The relative efficiency of propositional proof systems,” *J. Symbolic Logic*, vol. 44, no. 1, pp. 36–50, 1979. doi: [10.2307/2273702](https://doi.org/10.2307/2273702)
- [Cra53] W. Craig, “On axiomatizability within a system,” *J. Symbolic Logic*, vol. 18, pp. 30–32, 1953, ISSN: 0022-4812. doi: [10.2307/2266324](https://doi.org/10.2307/2266324)
- [dRGN+21] S. F. de Rezende, M. Göös, J. Nordström, T. Pitassi, R. Robere, and D. Sokolov, “Automating algebraic proof systems is NP-hard,” in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 2021, pp. 209–222. doi: [10.1145/3406325.3451080](https://doi.org/10.1145/3406325.3451080)
- [FFNR03] S. A. Fenner, L. Fortnow, A. V. Naik, and J. D. Rogers, “Inverting onto functions,” *Inform. and Comput.*, vol. 186, no. 1, pp. 90–103, 2003. doi: [10.1016/S0890-5401\(03\)00119-6](https://doi.org/10.1016/S0890-5401(03)00119-6)
- [FU96] X. Fu and A. Urquhart, “Simplified lower bounds for propositional proofs,” *Notre Dame journal of formal logic*, vol. 37, no. 4, pp. 523–544, 1996. doi: [10.1305/ndjfl/1040046140](https://doi.org/10.1305/ndjfl/1040046140)
- [Gar19] M. Garlík, “Resolution lower bounds for refutation statements,” in *44th International Symposium on Mathematical Foundations of Computer Science*, 2019, Art. No. 37, 13. doi: [10.4230/LIPIcs.MFCS.2019.37](https://doi.org/10.4230/LIPIcs.MFCS.2019.37)
- [Gar24] M. Garlík, “Failure of feasible disjunction property for k -DNF resolution and NP-hardness of automating it,” in *39th Computational Complexity Conference, CCC 2024, July 22–25, 2024, Ann Arbor, MI, USA*, vol. 300, 2024, 33:1–33:23. doi: [10.4230/LIPIcs.CCC.2024.33](https://doi.org/10.4230/LIPIcs.CCC.2024.33)
- [Gay21] A. Gaysin, “H-coloring dichotomy in proof complexity,” *Journal of Logic and Computation*, vol. 31, no. 5, pp. 1206–1225, Apr. 2021. doi: <https://doi.org/10.1093/logcom/exab028>
- [Gay23] A. Gaysin, *Proof complexity of CSP*, 2023. arXiv: [2201.00913](https://arxiv.org/abs/2201.00913).
- [Gay24] A. Gaysin, *Proof complexity of universal algebra in a CSP dichotomy proof*, 2024. arXiv: [2403.06704](https://arxiv.org/abs/2403.06704).

- [GKMP20] M. Göös, S. Koroš, I. Mertz, and T. Pitassi, “Automating cutting planes is NP-hard,” in *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, 2020, pp. 68–77. DOI: [10.1145/3357713.3384248](https://doi.org/10.1145/3357713.3384248)
- [GL10] N. Galesi and M. Lauria, “On the automatizability of polynomial calculus,” *Theor. Comp. Sys.*, vol. 47, no. 2, pp. 491–506, Aug. 2010. DOI: [10.1007/s00224-009-9195-5](https://doi.org/10.1007/s00224-009-9195-5)
- [Hak85] A. Haken, “The intractability of resolution,” *Theoretical Computer Science*, vol. 39, pp. 297–308, 1985, Third Conference on Foundations of Software Technology and Theoretical Computer Science, ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(85\)90144-6](https://doi.org/10.1016/0304-3975(85)90144-6)
- [Hås23] J. Håstad, “On small-depth Frege proofs for PHP,” in *2023 IEEE 64th Annual Symposium on Foundations of Computer Science—FOCS 2023*, IEEE Computer Soc., Los Alamitos, CA, 2023, pp. 37–49. DOI: [10.1109/FOCS57990.2023.00010](https://doi.org/10.1109/FOCS57990.2023.00010)
- [HP11] L. Huang and T. Pitassi, “Automatizability and simple stochastic games,” in *Automata, languages and programming. Part I*, ser. Lecture Notes in Comput. Sci. Vol. 6755, Springer, Heidelberg, 2011, pp. 605–617. DOI: [10.1007/978-3-642-22006-7_51](https://doi.org/10.1007/978-3-642-22006-7_51)
- [HP93] P. Hájek and P. Pudlák, *Metamathematics of first-order arithmetic*. Springer-Verlag, Berlin, 1993. DOI: [10.1007/978-3-662-22156-3](https://doi.org/10.1007/978-3-662-22156-3)
- [HR22] J. Håstad and K. Risse, “On bounded depth proofs for Tseitin formulas on the grid; revisited,” in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2022, pp. 1138–1149.
- [IR22] D. Itsykson and A. Riazanov, “Automating OBDD proofs is NP-hard,” in *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022)*, vol. 241, 2022, 59:1–59:15. DOI: [10.4230/LIPIcs.MFCS.2022.59](https://doi.org/10.4230/LIPIcs.MFCS.2022.59)
- [Iwa97] K. Iwama, “Complexity of finding short resolution proofs,” in *Mathematical foundations of computer science 1997 (Bratislava)*, ser. Lecture Notes in Comput. Sci. Vol. 1295, Springer, Berlin, 1997, pp. 309–318. DOI: [10.1007/BFb0029974](https://doi.org/10.1007/BFb0029974)
- [Jeř05] E. Jeřábek, “Weak pigeonhole principle, and randomized computation,” Ph.D. dissertation, Faculty of Mathematics and Physics, Charles University, Prague, 2005.
- [Jeř25] E. Jeřábek, *Modus ponens style inferences in Resolution*, Theoretical Computer Science Stack Exchange, 2025. [Online]. Available: <https://cstheory.stackexchange.com/q/55062>
- [Kha22a] E. Khaniki, “New relations and separations of conjectures about incompleteness in the finite domain,” *Journal of Symbolic Logic*, vol. 87, no. 3, pp. 912–937, 2022. DOI: [10.1017/jsl.2021.99](https://doi.org/10.1017/jsl.2021.99)
- [Kha22b] E. Khaniki, “Nisan-Wigderson Generators in Proof Complexity: New Lower Bounds,” in *37th Computational Complexity Conference (CCC 2022)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 234, 2022, 17:1–17:15. DOI: [10.4230/LIPIcs.CCC.2022.17](https://doi.org/10.4230/LIPIcs.CCC.2022.17)
- [Kha24] E. Khaniki, “Jump operators, interactive proofs and proof complexity generators,” in *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2024, pp. 573–593.
- [KM00] J. Köbler and J. Messner, “Is the standard proof system for SAT p-optimal?” In *International Conference on Foundations of Software Technology and Theoretical Computer Science*, Springer, 2000, pp. 361–372. DOI: [10.1007/3-540-44450-5_29](https://doi.org/10.1007/3-540-44450-5_29)
- [KP98] J. Krajíček and P. Pudlák, “Some consequences of cryptographical conjectures for S_2^1 and EF,” *Information and Computation*, vol. 140, no. 1, pp. 82–94, 1998. DOI: [10.1006/inco.1997.2674](https://doi.org/10.1006/inco.1997.2674)

- [KPT91] J. Krajíček, P. Pudlák, and G. Takeuti, “Bounded arithmetic and the polynomial hierarchy,” *Annals of Pure and Applied Logic*, vol. 52, no. 1, pp. 143–153, 1991. doi: [https://doi.org/10.1016/0168-0072\(91\)90043-L](https://doi.org/10.1016/0168-0072(91)90043-L)
- [KPW95] J. Krajíček, P. Pudlák, and A. Woods, “An exponential lower bound to the size of bounded depth Frege proofs of the pigeonhole principle,” *Random Structures Algorithms*, vol. 7, no. 1, pp. 15–39, 1995. doi: [10.1002/rsa.3240070103](https://doi.org/10.1002/rsa.3240070103)
- [Kra01] J. Krajíček, “On the weak pigeonhole principle,” *Fundamenta Mathematicae*, vol. 170, no. 1-2, pp. 123–140, 2001. doi: [10.4064/fm170-1-8](https://doi.org/10.4064/fm170-1-8)
- [Kra19] J. Krajíček, *Proof complexity*. Cambridge University Press, Cambridge, 2019. doi: [10.1017/9781108242066](https://doi.org/10.1017/9781108242066)
- [Kra22] J. Krajíček, “Information in propositional proofs and algorithmic proof search,” *J. Symb. Log.*, vol. 87, no. 2, pp. 852–869, 2022. doi: [10.1017/jsl.2021.75](https://doi.org/10.1017/jsl.2021.75)
- [Kra23] J. Krajíček, “A proof complexity conjecture and the incompleteness theorem,” *The Journal of Symbolic Logic*, pp. 1–5, 2023. doi: [10.1017/jsl.2023.69](https://doi.org/10.1017/jsl.2023.69)
- [Kra24] J. Krajíček, “Extended Nullstellensatz proof systems,” *Proceedings of the American Mathematical Society*, vol. 152, pp. 4881–4892, 2024. doi: doi.org/10.1090/proc/16709
- [Kra25] J. Krajíček, *Proof Complexity Generators*. Cambridge University Press, Cambridge, 2025. doi: <https://doi.org/10.1017/9781009611664>
- [Kra95] J. Krajíček, *Bounded arithmetic, propositional logic, and complexity theory*. Cambridge University Press, Cambridge, 1995. doi: [10.1017/CBO9780511529948](https://doi.org/10.1017/CBO9780511529948)
- [LLR24] J. Li, Y. Li, and H. Ren, “Metamathematics of Resolution lower bounds: A TFNP perspective,” *Electronic Colloquium on Computational Complexity (ECCC)*, no. TR24-190, 2024. [Online]. Available: <https://eccc.weizmann.ac.il/report/2024/190/>
- [MP20] M. Müller and J. Pich, “Feasibly constructive proofs of succinct weak circuit lower bounds,” *Annals of Pure and Applied Logic*, vol. 171, no. 2, p. 102 735, 2020. doi: [10.1016/j.apal.2019.102735](https://doi.org/10.1016/j.apal.2019.102735)
- [MP24] N. Mazon and R. Pass, “Gap MCSP Is Not (Levin) NP-Complete in Obfuscopia,” in *39th Computational Complexity Conference (CCC 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 300, 2024, 36:1–36:21. doi: [10.4230/LIPIcs.CCC.2024.36](https://doi.org/10.4230/LIPIcs.CCC.2024.36)
- [MPW19] I. Mertz, T. Pitassi, and Y. Wei, “Short proofs are hard to find,” in *46th International Colloquium on Automata, Languages, and Programming (ICALP 2019)*, vol. 132, 2019, 84:1–84:16. doi: [10.4230/LIPIcs.ICALP.2019.84](https://doi.org/10.4230/LIPIcs.ICALP.2019.84)
- [Mül21] M. Müller, “Typical forcings, NP search problems and an extension of a theorem of Riis,” *Annals of Pure and Applied Logic*, vol. 172, no. 4, p. 102 930, 2021. doi: doi.org/10.1016/j.apal.2020.102930
- [Nar22] M. Narusevych, *Models of bounded arithmetic and variants of pigeonhole principle*, 2022. arXiv: [2208.14713](https://arxiv.org/abs/2208.14713).
- [Nar24] M. Narusevych, *An independence of the MIN principle from the PHP principle*, 2024. arXiv: [2406.14930](https://arxiv.org/abs/2406.14930).
- [Oli25] I. C. Oliveira, “Meta-mathematics of computational complexity theory,” *Electronic Colloquium on Computational Complexity (ECCC)*, no. TR25-041, 2025. [Online]. Available: <https://eccc.weizmann.ac.il/report/2025/041/>

- [Pap24] T. Papamakarios, “Depth- d Frege systems are not automatable unless $P = NP$,” in *39th Computational Complexity Conference (CCC 2024)*, vol. 300, 2024, 22:1–22:17. DOI: [10.4230/LIPIcs.CCC.2024.22](https://doi.org/10.4230/LIPIcs.CCC.2024.22)
- [PBI93] T. Pitassi, P. Beame, and R. Impagliazzo, “Exponential lower bounds for the pigeonhole principle,” *Computational Complexity*, vol. 3, no. 2, pp. 97–140, 1993. DOI: [10.1007/BF01200117](https://doi.org/10.1007/BF01200117)
- [PP24] J. Park and H. T. Pham, “A proof of the Kahn–Kalai conjecture,” *J. Amer. Math. Soc.*, vol. 37, pp. 235–243, 2024. DOI: [10.1090/jams/1028](https://doi.org/10.1090/jams/1028)
- [PS22] J. Pich and R. Santhanam, “Learning algorithms versus automatability of Frege systems,” in *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, vol. 229, 2022, 101:1–101:20. DOI: [10.4230/LIPIcs.ICALP.2022.101](https://doi.org/10.4230/LIPIcs.ICALP.2022.101)
- [PS23] J. Pich and R. Santhanam, *Towards $P \neq NP$ from Extended Frege lower bounds*, 2023. arXiv: [2312.08163](https://arxiv.org/abs/2312.08163).
- [Pud03] P. Pudlák, “On reducibility and symmetry of disjoint NP pairs,” *Theoretical Computer Science*, vol. 295, no. 1-3, pp. 323–339, 2003. DOI: [https://doi.org/10.1016/S0304-3975\(02\)00411-5](https://doi.org/10.1016/S0304-3975(02)00411-5)
- [Pud20] P. Pudlák, *Reflection principles, propositional proof systems, and theories*, 2020. arXiv: [2007.14835](https://arxiv.org/abs/2007.14835).
- [Raz95] A. A. Razborov, “Bounded arithmetic and lower bounds in Boolean complexity,” in *Feasible mathematics, II (Ithaca, NY, 1992)*, ser. Progr. Comput. Sci. Appl. Logic, vol. 13, Birkhäuser Boston, Boston, MA, 1995, pp. 344–386.
- [ST21] R. Santhanam and I. Tzameret, “Iterated lower bound formulas: A diagonalization-based approach to proof complexity,” in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 2021, pp. 234–247. DOI: <https://dl.acm.org/doi/10.1145/3406325.3451010>
- [Tha16] N. Thapen, “A tradeoff between length and width in resolution,” *Theory Comput.*, vol. 12, Paper No. 5, 14, 2016. DOI: [10.4086/toc.2016.v012a005](https://doi.org/10.4086/toc.2016.v012a005)